PAYMENTS ENGINEERING · ARCHITECTURE PLAN

Moving 2027+ Bookings from Destination Charges to Deferred Transfers

Research findings and a phased implementation plan for changing how card and card-reader payments settle to connected (tour operator) Stripe accounts — gated by tour date, not booking date.

Platform: NAT Tour Backend — Laravel 11, Stancl/Tenancy v3, Stripe Connect

Prepared for: Devat Karetha

Prepared: September 7, 2026

Status: Shipped — see Revision History for the complete change log

CONFIDENTIAL — INTERNAL ENGINEERING PLANNING DOCUMENT

Revision History

A running change log of every phase and revision of this plan, in chronological order. See Section 9 for the structured phase-by-phase status and Section 7 for the technical detail behind each entry below.

confirmed no 2027-dated booking has been sold yet (clean forward cutover, not a migration); transfer timing confirmed immediate/synchronous (no job, no new webhooks); partner-package design corrected per business clarification (Decisions 5 & 6 resolved); Phase 2/3 in progress — new-booking (7.2a), Make Payment (7.2b, caught a real pre-ship bug), combo cart (7.2c), and Change People (7.2d, discovered a shared creation helper covering 3 features) all wired and tested. Change Payment Method wired fifth (Section 7.2e). Switch Package wired sixth (Section 7.2f) — non-partner `completeSecureCardPayment()` done and tested; `run()`'s payment branch wired but not independently tested; refund branches deferred to Phase 4. Card Reader's shared creation endpoint wired seventh (Section 7.2g) — not its own confirmation flow at all, a creation-side-only fix; caught a stray-`application_fee_amount` bug that would have hard-failed every 2027+ reader charge. Booking-level Tip wired eighth (Section 7.2h) — discovered a SECOND shared creation helper, `createStripePaymentIntentCommon()`, covering FOUR more features at once (Edit Discount, Custom Charge, Add-Ons, Addon Edit Charge) alongside Tip; fixed it once (creation side for all five now correct), wired Tip's confirmation specifically, and found+fixed a real pre-existing bug (a missing `infants` column) that broke every ordinary web tip payment regardless of tour date. Edit Discount wired ninth (Section 7.2i) — traced a genuinely tangled confirmation chain (`editDiscountCardPayment()` → `handleWebPayment()` → `createEditDiscountPaymentIntent()` → `applyDiscount()`) to find the one method, `createEditDiscountPaymentIntent()`, that actually does the confirmation work for the common web case; confirmed `applyDiscount()`'s own separate (buggy, pre-existing) partner-transfer branch is a genuinely different, untouched code path used only by Cash/API callers.

Custom Charge wired tenth (Section 7.2j) — its confirmation method, `customChargeCreatePaymentIntent()`, turned out to hold a SECOND, independent creation shape (a raw manual card-number-entry branch with its own `buildConnectedPaymentData()` call, no Stripe Elements involved) directly alongside the normal refresh path — fixed both since they sit in the same method, though the raw-entry branch could not be independently tested (no known live caller reaches it on the modern page). Add-Ons wired eleventh (Section 7.2k) — the cleanest site wired in a while: no partner-package handling, no second creation shape, `resolveCardPaymentIntent()`'s existing null-for-Cash pattern handled with a nullsafe operator. Re-ran the full 31-test pre-existing add-ons regression suite since several of those tests call the same `storeAddonsPaymentInTransaction()` signature just extended — all still pass. Addon Edit Charge wired twelfth (Section 7.2l) — confirmed genuinely separate from Add-Ons (a different route, a much larger method matching its extensive prior real-money-bug history), touched only the CHARGE action's final aggregate ledger write, left the REFUND action and all upstream per-item PI-offset/cascade math untouched. **This completes all five features sharing the Section 7.2h `createStripePaymentIntentCommon()` creation helper.** Mobile API v2 new-booking checkout wired thirteenth (Section 7.2m) — Appendix A.1 site #20, a combo-cart-capable creation site (resolved via `PaymentModeResolver::resolve()`'s plural form) whose confirmation, `packageBookingTour()`, creates the full booking/transact/history rows itself (closer in shape to the original web new-booking flow than a typical PaymentIntent-refresh site) and writes its `transactions` row via a raw `DB::table()` insert rather than `TransactionService::storePaymentInTransaction()`; found the same "package-level-only `stripe_destination_id`" gap `paymentNewBooking()` had in Section 7.2b.

Latest revision: Payment Links wired fourteenth (Section 7.2n) — a Checkout Session rather than a raw PaymentIntent, mode embedded in `payment_intent_data.metadata`; while tracing its operator-vs-partner destination columns, re-examined every other flow's own lookup for the same shape and found a REAL, previously-shipped bug: Switch Package's `completeSecureCardPayment()` (Section 7.2f) was reading the wrong `stripe_destination_id` column, which is `NULL` for real non-partner packages — meaning its operator transfer would have silently never fired while still recording a false "Completed" payout row. Found, fixed, and verified with a genuine before/after regression proof (see the callout in Section 7.2n). Mobile API v2's own 3D-Secure completion sibling, `packageBooking3dsPayment()`, wired (Section 7.2o, Appendix A.4 site #13) — the same "package-level-only `stripe_destination_id`" gap found again in its own separate local query, fixed the same way. Latest revision: combo new-booking Card wired (Section 7.2p, Appendix A.4 site #5) — `createComboBookingCard()` (already has working partner handling) plus a bonus fix to its sibling creation method, `createPaymentIntentForCart()` (zero partner handling of any kind today, a separate pre-existing gap; replicated that exact gap for deferred mode rather than inventing new partner logic). Latest revision: `applyDiscount()`'s own Card/mobile-API branch (site #10, the last one on Appendix A.4's list) investigated fully (Section 7.2q) and deliberately left unwired — traced every caller and confirmed it is unreachable from Edit Discount's two normal entrypoints, only reachable via a mobile-API "second click" edge case with no reliable PaymentIntent id to resolve mode from at all; wiring it would mean re-deriving mode from the current tour date, violating this plan's core "never re-derive" principle. **Appendix A.4's 19-site list is now fully accounted for** — every site is wired, correctly out of scope, or investigated-and-declined for a stated reason.

POS wired (Section 7.2r, Appendix A.1 site #1) — the last explicitly-tracked creation-side gap from Section 7.2g's own follow-up list; a combo-cart-capable creation site (mode resolved across every package's own tour date) whose confirmation, `cardPaymentForPos()`, has no partner-package handling at all today. 9 distinct creation code paths (covering 12 features across Sections 7.2a-m, plus POS) and 16 of 19 confirmation sites done — Phases 2/3 essentially complete. Phase 4 (refund/dispute cutover) started — `PaymentGatewayInterface::buildRefundParams()` shipped (Section 7.4), the mode-aware refund-shaping counterpart to `buildConnectedPaymentData()`; first real call site wired and tested end-to-end, `TicketRefundService::refund()`'s Card branch (single-transaction path). Second Phase 4 site wired, `WeatherRefundService::refund()`'s Card branch — the "100% Refund" sibling of `TicketRefundService`, same no-partner-handling shape. Third Phase 4 site wired, `TicketRefundCancelService::cancel()` — the first refund site with genuine partner-package handling, dispatched from the separate "Cancel Booking" flow rather than "Refund Order"; proved both the non-partner (`reverseOperatorTransfer()`) and partner (`reverseSellingCommissionTransfer()`, alongside the pre-existing mode-independent `reverseTransfer()` for the 3rd party) cases end to end. Fourth Phase 4 site wired, `WeatherRefundCancelService::cancel()` — the "100% Refund" sibling of `TicketRefundCancelService`, same partner-package shape proven a second time, plus a deliberate faithful-replication decision for one call site with no partner handling in destination mode either today.

Fifth Phase 4 site wired, `PartialRefundCancelService::cancel()` — closes out all three `CancelBooking/*` services; the most complex site yet, proving a partial refund correctly SPLITS its reversal between the 3rd party's ticket-cost share and the selling tenant's Partner Fee once a spillover occurs, and a real finding that discount-type charges never had a selling-commission transfer to begin with (matching Section 7.2i's Edit Discount precedent). Switch Package's `run()` refund branch wired too (Section 7.2f, Appendix A.4 site #19) — reasoned-through only, matching this method's own payment-branch precedent, and genuinely different from every prior Phase 4 site: its reversal has to be resolved by `package_id` rather than `transaction_id`, since a booking can switch through several packages over its lifetime; confirmed `completeSecureCardPayment()` has no refund branch at all. **Latest revision:** `TourBookingAddonRepository::refundAddonsByBookingId()` wired (Section 7.4) — the shared add-ons refund method called by all three `CancelBooking/*` services; caught and fixed a real mocking-hazard regression across 9 pre-existing tests in 7 files along the way (a brand-new gateway-interface method silently returning an empty array against an unstubbed mock — not a logic bug in the new method itself).

Since then: `CustomChargeBookingService.php` and both of `EditBookingService.php`'s refund sites wired; a Phase 2/3 addendum found and fixed legacy (API V1)'s own never-wired `createEditDiscountPaymentIntent()` charge side; `editDiscountRefund()` and `refundCustomCharge()` wired in both legacy and V3 `TourBookingRepository.php`; and `refund()`'s "Refund Booking" action wired in both files too, the most sophisticated partner-fee-spillover logic found anywhere in the whole survey (per-transaction `netTicketCost` / `netPartnerCommission` splits across three independent `refundPayment()` call sites); and `editBooking()` wired in both files too, by far the largest method touched all session (~2,020 lines, three distinct admin actions — Permit Fee Refund, Delete Pax Ticket/Weather Refund, Delete Pax Partial Refund) with legacy again confirmed to have zero partner-package handling; and — newest — `deletePeopleChangePeople()` wired in both files, closing out `TourBookingRepository.php` ENTIRELY (all 49 of its `refundPayment()` sites across both files).

Every file in Phase 4's original survey is now fully accounted for — the only remaining Phase 4 work was `DisputeService::handleDisputeLoss()`'s mode branch (Section 7.5) — and it's now done too: deferred-mode dispute losses reverse the separate `primary_operator` Transfer via the already-shipped `reverseOperatorTransfer()`, since a deferred-mode charge carries no `transfer_data` for the pre-existing destination-mode logic (reading `charge.transfer`) to expand; confirmed non-partner-only, matching that same pre-existing scope exactly (a dedicated architecture investigation confirmed `transferToOperator()` is never called for a partner-package booking in any mode). PHASE 4 IS NOW COMPLETE — every refund and dispute call site in the original survey is wired and tested. Since then: started the standalone legacy-creation-method audit (Section 7.2s) flagged repeatedly during Phase 4 — a sub-agent investigation found the gap is bigger than expected, touching SHARED helpers used by both legacy and V3, not just legacy-only code. Fixed seven legacy methods so far (`createPaymentIntentForCart()`, `createPaymentIntent()`, `createBookingCard()`, `paymentNewBooking()`, `createComboBookingCard()`, `paymentCartBookings()`, `customChargeCreatePaymentIntent()`), one shared creation entrypoint used by BOTH files (`stripeCreatePaymentIntent()`, the mobile app's iOS Card Reader flow), plus two shared upstream helpers (`StripeUtils::createStripePaymentIntentCart()`, `getStripeAttributesForAppCardReader()`) — catching real "wrong destination column" bugs in `paymentNewBooking()` and again in `stripeCreatePaymentIntent()` / `getStripeAttributesForAppCardReader()` (the same bug class found four times now across this project) — 132 tests passing. Also investigated `getStripeAttributesForAppTapToPay()` / `getStripeAttributesForCreateTip()` and CONFIRMED both are the tour-guide gratuity system, out of scope per Section 3's original exclusion (no

booking/tour date exists for these "direct" tips at all) — no fix needed, no code changed. Then fixed the SHARED `CustomChargeBookingService::tourBookingUpdateCustomCharge()` / `cardPaymentCustomChargeFromApp()`, which also retroactively covers the live `StripePaymentIntentSucceededForMakePayment` webhook handler's own call to the former. Then `tipPayment()` (legacy) — the hardest item on this audit's list — fixed too: put the "no transactions row exists" design question to the user (per this plan's standing practice for genuine architecture decisions) after establishing `payouts.transaction_id` has no real FK and no tip refund/reversal path exists anywhere; user chose the minimal no-migration fix, using the new `tour_booking_tips` row's own id as the correlation key and recovering NAT's platform fee as the gap between the PaymentIntent's own charged total and the recorded tip amount. Caught a FIFTH "wrong destination column" instance along the way (not reused). Then ran the final broad sweep for real — enumerated every `buildConnectedPaymentData(` call site and every PaymentIntent-creation helper definition across the whole codebase, plus checked for inline `transfer_data` / `application_fee_amount` construction that could bypass the gate entirely — found and fixed ONE genuinely new site, `CustomerController::createTipPaymentLink()`'s public guide-tip Payment Link (distinct from every other tip flow), and confirmed one more out-of-scope item (`InvoiceRepository`'s ACH/bank-transfer Stripe Invoices, a different product entirely). THE LEGACY CREATION-METHOD AUDIT IS NOW COMPLETE. Finally (Section 7.2t), closed the LAST known open item across the whole project: the New Booking/POS pages' Card Reader tabs had no tour-date plumbing at all for a not-yet-created booking — fixed by having those pages' own JS send the already-selected tour date(s) as a new `metadata.tour_dates` field, resolved by `CardReaderRepository` the same way every combo-cart site already resolves multiple dates.

THE ENTIRE 2027 DEFERRED-TRANSFER PROJECT IS NOW COMPLETE. Latest revision (Section 7.2u): a requested post-completion rename — `ThreeWayPaymentService` / `PayoutService` → `TransferPaymentService` / `TransferService` (directory moved to match), `App\Models\Payout` → `Transfer`, and the `payouts` table (+ its `payout_type` / `payout_status` / `payout_amount` columns) → `transfers`, applied across every production file, all 46 affected test files, and this document — carefully scoped to leave the entirely separate AutoPayout/bank-payout system, and the Stripe metadata keys already sent on live PaymentIntents, untouched. 139 tests passing. Section 7.2s/7.2t/7.2u have the full narrative; see Section 9 for even more detail.

Independent 100%-completeness audit run this revision — re-read every section of this document end to end (not just its own status badges) and independently re-verified the highest-stakes claims directly against the live codebase and database rather than trusting the document's own "PROJECT COMPLETE" banner: confirmed `DisputeService::handleDisputeLoss()`'s mode branch is real shipped code, confirmed all 19 `TransferPaymentService` methods and the currency-hardcode fix are genuinely in place, and re-ran the full curated regression suite fresh (139/139 passing, independently re-counted, not just trusted from the prior revision's own tally). Conclusion: the core feature — every creation site, the transfer engine, the full refund/dispute cutover, the legacy-creation-method audit, and the rename — is genuinely code-complete and tested; every remaining "shipped, partial" badge traces to either reasoned-through-only test coverage on a disproportionate-to-test branch, or a different, pre-existing, deliberately out-of-scope bug — no functional gaps found. But Phase 5 (Reporting) and Phase 6 (Verification & go-live) were confirmed genuinely not started — real remaining work, not a nuance — and Decision 4 (kill-switch scope) was confirmed still open. Also found and fixed: Section 11's Risk Register had gone stale in a couple of rows (still describing the dispute-loss handler and the legacy-method audit as "not yet built" long after both shipped) — corrected below. Also found (database hygiene, not a code gap): 217 pre-existing orphaned "Test Customer" bookings plus 1 standalone orphaned customer row in local `dixies`, debris from earlier debugging sessions on this project, not from this audit's own test run — cleaned up via a verified cascade delete (confirmed zero real/shared customers were at risk first), with the full regression suite re-confirmed green afterward.

Decision 4 resolved this revision: per-tenant kill-switch, built. Chosen over a global switch specifically because every tenant has its own Stripe Connect account with its own capability/restriction state (Section 11's own risk register already flagged "connected account restricted" as a real failure mode) — a per-tenant gate contains a problem to the one tenant that has it, and matches how tenants will actually open their 2027 calendars on their own separate schedules rather than on one shared go-live moment. Implemented as a new tenant-scoped `settings` row, `DEFERRED_TRANSFER_ENABLED` (migration `2026_09_13_000001_add_deferred_transfer_enabled_setting.php`, same shape and same `hasSetting()` read path as the pre-existing `AUTO_PAYOUT` toggle), checked once inside `PaymentModeResolver::resolve()` / `resolveForSingleDate()` so it cannot be bypassed by forgetting it at any individual call site: when disabled (or the row is missing entirely, e.g. an un-migrated tenant), the resolver returns `DESTINATION` unconditionally, without even inspecting the tour date. Defaults OFF for every tenant the migration runs against — applied to all 10 live local tenant databases, all left disabled except `dixies` (the tenant this whole feature's test suite runs against), which was explicitly enabled to match the "pilot tenant already opted in" state a real staged rollout would start from. See Section 7.10 for the full technical writeup. 4 new tests, `tests/Feature/Payments/DeferredTransferKillSwitchFlowTest.php` — disabled forces destination for both a single 2027 date and a combo cart containing one, enabling doesn't disturb the underlying date-boundary rule, and a missing settings row fails closed to destination. One real test-authoring bug caught and fixed along the way: the first draft's `tearDown()` used a bare SQL `update()` to restore the setting, which silently no-ops once the "missing row" test has deleted it — the very next test file to run (alphabetically, in the very same suite) then saw the setting as permanently absent and 8 of its assertions failed; fixed by deleting-then-reinserting the full original row instead of trusting an `update()` against a row that might no longer exist. 143 tests passing (139 + 4).

Real gap found by the user testing live, fixed same revision (Section 7.11): destination-charge transfers had no description/metadata in the connected account's own Stripe dashboard. Confirmed empirically against a real test-mode Stripe account (not just by code review): this codebase's own six explicit `Transfer::create()` calls already send both correctly (verified by creating one and reading it back); the actual gap is that Stripe's *automatically*-created transfer behind every ordinary destination charge — the model virtually all bookings still use — comes back with `description: null`, `metadata: []` no matter what the originating PaymentIntent/Charge carries, a Stripe platform behavior with no creation-time parameter to prevent it. First implementation attempt centralized this at the `payment_intent.succeeded` Stripe webhook — the user corrected this: that webhook's dispatch plumbing was built specifically for the mobile application, which is currently closed, so it cannot be relied on as a delivery mechanism at all right now. Rebuilt around a queued Laravel Job instead, dispatched from `TransactionService`'s four shared transaction-writing methods (the same choke point this project's own `payment_mode` column already threads through ~40+ callers) — no Stripe-side dependency, runs on this app's own queue. Found and fixed a real cross-cutting test-safety issue along the way: this environment's

`QUEUE_CONNECTION=database` meant the new dispatch would have left orphaned `jobs` rows (and risked a real worker hitting Stripe with fake test data) across dozens of pre-existing tests — fixed by adding `Queue::fake()` to the global test base class, alongside its existing `Storage` / `Mail` fakes. A SECOND real gap surfaced right after, from the same real booking: the connected account's own "Payments" dashboard view still showed nothing, traced to a genuinely separate Stripe object (`Transfer::$destination_payment`, only readable/writable with the connected account's own request context) that a bare deferred-mode transfer never populates (no `source_transaction` to inherit from, unlike a real destination charge's transfer, which already inherits it automatically). Fixed via one shared method, `AttachDestinationTransferMetadata::attachDestinationPaymentCharge()`, called from both the six explicit `TransferPaymentService` transfer sites and the destination-mode job. Verified against the actual real booking that surfaced it. 162 tests passing (143 + 19), zero orphaned rows.

Same-day follow-up, two separate requests from the user. (1) The `payoutAmount` rename scope note from Section 7.2u was revisited a second time after the user spotted `'transfer_amount' => $payoutAmount,` still sitting in a code excerpt — traced to a much larger, higher-risk pattern than a single leftover variable (the token is propagated across file boundaries via `compact()`, so producer and consumer had to move together), and extended to all 10 files it actually spans, 3 of them newly discovered (`EditBookingService.php`, `PaymentLinkService.php`, `StripeRepository.php`), 69 replacements total — see Section 7.2u's own addendum for the full narrative. (2) A tenant-identifying prefix (e.g. `"DIXIES – "`, the uppercased tenant id — the user's own final choice, after an intermediate name-preference draft caught a real docblock/body mismatch bug along the way) was added to every Stripe-facing transfer description via a new `tenantDescriptionPrefix()` helper. The user then asked to keep the prefix OFF for now, pending client confirmation — implemented as a single early-return at the top of the helper, so re-enabling it later needs no call-site changes; see the new Section 7.12. 163 tests passing (162 + 1), zero orphaned rows.

Real production bug found by the user testing live (booking #348614, kens tenant): cancelling a 2027+ booking with a 100% refund left the guest refunded but the operator's Transfer never reversed, and the booking stuck un-cancelled. Root cause: `WeatherRefundService::refund()` and `WeatherRefundCancelService::cancel()` (Section 7.4) passed the GROSS booking/transaction total into `reverseOperatorTransfer()` instead of the NET amount actually sent to the operator (`transfers.transfer_amount`, always total minus the platform/service fee, per `transferToOperator()`'s own `amount – platformFees` formula) — guaranteed to exceed the remaining balance whenever a booking carries any real fee, tripping the reversal's own over-reversal guard immediately AFTER the Stripe charge refund had already succeeded. Their sibling files, `TicketRefundService` / `TicketRefundCancelService`, already computed this correctly; the Weather* pair simply never got the same treatment. Fixed at all 8 affected call sites across both files (the single-transaction branch each method already had, Section 7.4, plus 3 multi-transaction loop branches each) by netting out the platform fee before reversing — `tourBooking->service_commission` for the single-transaction branches, the per-transaction `platform_fees` already on each site's own `TourBookingHistory` 'Payment' record for the loop branches. 2 new regression tests (one per file) use a real nonzero platform fee mirroring the production booking's own numbers; each was verified to genuinely reproduce the exact production error message when the fix was temporarily reverted, before restoring it.

A related hardening, same revision, proposed by the user: reverse the Transfer BEFORE the Stripe charge refund, not after — piloted on these same two files only, at the user's own request to scope it narrowly for now. In deferred-transfer mode these are two independent Stripe calls (destination mode's single refund call with `reverse_transfer=true` already does both atomically); whichever runs first is the one that "wins" if the second fails. The original ordering meant any reversal failure — not only the amount bug above — left an already-refunded guest with an un-reversed Transfer, exactly what the user's own booking hit. Reordered all 8 call sites so the reversal fires first; if it fails, the Stripe refund is never attempted at all, so the worst remaining failure mode is a reversed Transfer sitting on NAT's own platform balance with a safely retryable refund, not a leaked payout. 2 more new tests (one per file) mock the reversal to fail and assert `buildRefundParams()` / `refundPayment()` are never reached — one of these caught a real test-authoring bug of its own (using `$this->expectException()` left the test's own `DB::beginTransaction()` uncommitted, lock-timing out every subsequent test's booking-fixture inserts; fixed with an explicit try/catch/rollback). Deliberately scoped to these 2 files only — the identical refund-then-reverse shape recurs across roughly 8 more files in this project (`TicketRefundService` / `TicketRefundCancelService`, `PartialRefundCancelService`, `DisputeService`, `SwitchTourPackageCardService`, `EditBookingService`, `CustomChargeBookingService`, `TourBookingAddonRepository`, both `TourBookingRepository.php` files) — reordering the rest is a distinct, larger decision, not yet made. Also flagged, not yet fixed: `PartialRefundCancelService.php`'s own 5 `reverseOperatorTransfer()` call sites carry the same gross-vs-net amount pattern as the bug above, lower risk in practice (a partial refund is usually well under the remaining balance) but not yet audited to the same standard.

Code-style follow-up, same revision, user-requested: the operator-reversal amount was originally computed inline inside the `reverseOperatorTransfer()` call itself. Extracted into a named `$operatorReversalAmount` variable on its own line immediately before each call, at all 8 sites in both files — a pure readability change; the full regression suite passes identically before and after. 167 tests passing (163 + 4), zero orphaned rows.

Next step, user-directed: audit and fix the flagged `PartialRefundCancelService.php` gap. All 5 sites confirmed genuinely broken, not just structurally similar — Site 1 (the discount-spillover branch) reversed `ticket_cost + platform_fees` (gross) unmodified, the identical shape as the Weather* bug; Sites 3-5 (the discount-transaction loop) confirmed against real production data (`dixies` tenant) that discount-type transactions carry a genuine, nonzero `platform_fees` too (e.g. $1.08 of a $12.07 charge) — not the non-issue a first read suggested; Site 2 (a partial, non-exhausting slice of the main transaction) needed its own reasoning entirely, since there's no single transaction-level fee figure to net out of an arbitrary admin-typed amount. Site 1 also had a compounding data-quality gap: it wrote `fee_amount = null` on every refund row it created, meaning a SECOND partial refund on the same transaction couldn't see what the first one had already covered — confirmed via real data that 1,169 bookings in the `kens` tenant alone have been partially refunded more than once on the same transaction, making this a real, not theoretical, risk. Fixed by tracking `fee_amount` the same way `TicketRefundCancelService.php`'s own equivalent branch already did, and recording the real value instead of `null` going forward.

The fix itself, corrected mid-implementation by the user to the right model: an initial draft for Site 2 used a PROPORTIONAL fee split (assume the fee scales with whatever fraction of the balance is being refunded) — the user redirected this to a simpler, correct "ticket-cost-first" model instead: reverse from the operator's transfer only up to the remaining ticket-cost portion; only once the requested amount EXCEEDS that does the excess come from NAT's own fee (never reversed from the operator, since the operator was never sent it), capped so the reversal can never exceed what the operator actually still holds. Applied uniformly: Sites 1 and 4 (refunding a transaction's/discount-transaction's WHOLE remaining balance) already matched this exactly once the fee is netted out; Sites 2, 3, and 5 (a PARTIAL slice) were rewritten from the proportional draft to `min(requested, remaining_ticket_cost)`.

4 new regression tests. Site 1 and Site 2 each get one, using the existing `CreateABooking` fixture with a real nonzero fee overridden onto the booking's own `tour_booking_history` row. Site 3 required building a genuinely new test fixture — a standalone discount-type `transactions` row with its own `tour_booking_history` / `transfers` rows, the first of its kind in this test suite — deliberately chosen to trigger both Site 1 and Site 3 in one call (a spillover from the main transaction into the discount transaction), and to cross the ticket-cost boundary specifically (requesting more than the discount transaction's own ticket cost, proving the cap, not just the "fits entirely" case). Sites 4 and 5 share Site 3's identical formula (confirmed via code review) and were left reasoned-through rather than independently tested, per the user's own call — building a fixture for Site 5 in particular would need a SECOND discount transaction to trigger its own spillover-of-a-spillover chain. Every new test was verified against the actual production error message: reverted the fix, confirmed the exact same `"Please enter amount less than`

`remaining transfer amount"` exception fired, then restored it. 170 tests passing (167 + 3 — Sites 1/2/3 each contribute one new test; Sites 4/5 share existing coverage), zero orphaned rows.

Same session, user-directed: audit every REMAINING caller of `reverseOperatorTransfer()` for the same gross-vs-net bug. A grep across `app/` found 8 more files beyond `WeatherRefund*` / `PartialRefundCancelService` / `TicketRefund*` (the last pair already confirmed correct, used as the reference pattern) — `TourBookingRepository.php` (legacy+V3), `TourBookingAddonRepository.php`, `EditBookingService.php`, `CustomChargeBookingService.php`, `DisputeService.php`, `SwitchTourPackageCardService.php` — roughly 48 raw call sites once the legacy/V3 files' own large near-duplicate blocks are counted individually. A read-only audit classified every site before touching any code: `DisputeService.php`'s own reversal (it already reproduces `reverseOperatorTransfer()`'s own remaining-balance formula directly) and the "delete-pax full refund"/"discount refund"/"custom-charge refund" families across both `TourBookingRepository.php` files plus `CustomChargeBookingService.php` confirmed already correct — all consistently net-of-fee already, several with their own explicit comments saying so. One item came back genuinely unclear from code alone: the Permit Fees Refund branch excludes a "processing fees" figure (`processing_charge_percentage`-based) from the reversal while including it in the Stripe refund, a different fee concept from `platform_fees` / `service_commission` that code reading alone couldn't resolve — the user settled it directly: "we are not using processing fee," so no fix was needed there.

5 distinct genuinely-broken patterns confirmed and fixed, deduplicated by actual code shape rather than raw line count (many call sites are literal byte-identical copies between the legacy and V3 files): (1) `refund()`'s three branches in both `TourBookingRepository.php` files (6 sites) — an explicit per-transaction `$partial` amount, an empty-partial "refund the whole remaining balance" branch, and the single-transaction `partial_Refund_value` form; (2) an arbitrary partial-refund amount validated only against a bound that deliberately includes the platform fee (correct for the Stripe-facing refund, wrong for the reversal) in `editBooking()` / `deletePeopleChangePeople()` (both files) and `EditBookingService::deletePaxFromGroupBooking()` (5 sites); (3) `TourBookingAddonRepository.php`'s `ADDONS_FULL_REFUND` reusing its fee-inclusive Stripe refund total as the reversal amount, across `deletePaxAddons()` / `chargeAddonFields()` / `refundAddonsByBookingId()` (3 sites); (4) `chargeAddonFields()`'s own `ADDONS_PARTIAL_REFUND` letting the ticket-cost overflow that deliberately spills into fee (the correct rule for the Stripe-facing amount) also leak into the reversal; (5) `SwitchTourPackageCardService::run()`'s reversal — correct for `full_refund`, gross for `100_refund`, unclamped for `partial_refund`, all three sharing one line since `min($netAmount, $netTicketCostAmount)` resolves correctly in all three cases at once. All 15 sites fixed with the same ticket-cost-first model established for `PartialRefundCancelService`: `min(requested, remaining_ticket_cost)`, or the full remaining ticket cost netted of fee when the whole balance is being refunded.

6 new regression tests. `TourBookingRepository.php`'s three `refund()` branches each get one (legacy file; the byte-identical V3 duplicate is reasoned through, not independently tested — its own existing non-partner test already exercises the same fixed code path today, just at a $0 fee, and continues to pass unmodified). The add-ons 100%-refund fee-inclusion and partial-refund overflow bugs each get one (`deletePaxAddons()`, `chargeAddonFields()`); `refundAddonsByBookingId()`'s identical 100%-refund pattern is reasoned through. The remaining Group 2 duplicate sites (V3's `editBooking()` / `deletePeopleChangePeople()`, `EditBookingService::deletePaxFromGroupBooking()`) and `SwitchTourPackageCardService::run()` are reasoned through by code review alone, applying the identical `min(requested, remaining_ticket_cost)` pattern already proven correct by the other 5 new tests — `SwitchTourPackageCardService`'s own refund path had zero pre-existing test coverage even before this fix (its one existing test covers a completely different entrypoint, `completeSecureCardPayment()`), matching this exact file's own prior precedent of deferring that branch's testing. Every new test was verified against the real production error message: reverted the specific file's fix via `git stash`, confirmed the exact same `"Unable to reverse operator transfer. Please enter amount less than remaining transfer amount."` exception fired, then restored it. 114 tests passing in the `tests/Feature/Payments/DeferredTransfer*` suite (108 + 6 new), cross-checked against a broader 334-test run spanning that directory plus `tests/Unit/Bookings`, `Helpers`, `Payments`, `TransferPayment`, and `Booking` — all passing, zero orphaned rows.

Same session, user-directed: extend the reverse-then-refund reordering hardening (piloted only on `WeatherRefundService` / `WeatherRefundCancelService`, Section 7.4) to every remaining file with the same refund-then-reverse shape. A read-only survey of all 57 `reverseOperatorTransfer()` call sites across the 10 non-piloted files found a pleasant surprise: 28 were already reverse-before-refund (mostly in `TourBookingRepository.php`, both legacy and V3, apparently hardened in an earlier pass that never made it into every method), and 1 (`DisputeService::handleDisputeLoss()`) has no paired refund call to reorder against at all — it reacts to a Stripe-decided chargeback loss, it doesn't initiate a refund. Of the 28 still refund-then-reverse, 17 were classified simple (the reversal computation already sits independently of any partner-branch coupling or intervening `DB::commit()`) and 11 complicated — 5 where a partner booking's `reverseSellingCommissionTransfer()` call has to move in lockstep with the reorder, and 6 where a `DB::commit()` sits between the refund and reversal calls today, a real transaction-semantics question rather than a cut-and-paste. User chose to do the 17 simple sites now, deferring the 11 complicated ones to a separate round. No test anywhere in the suite pins Mockery call order (confirmed via grep for `->ordered()`), so the reorder itself was expected not to break any existing test — confirmed true by re-running the full 334-test suite after.

All 17 simple sites reordered: `TicketRefundService::refund()` (both call sites), `PartialRefundCancelService::cancel()`'s 3 discount-transaction branches, all 4 `TourBookingAddonRepository.php` sites (`deletePaxAddons()` / `chargeAddonFields()` / `refundAddonsByBookingId()`), legacy `TourBookingRepository.php`'s `editDiscountRefund()` (2 sites — its V3 counterpart was already correctly ordered), V3's `refundCustomCharge()` else-branch (1 site — its own sibling branches carry the `DB::commit()` complication, deferred), and all 5 `EditBookingService.php` sites (`deletePaxFromGroupBooking()`'s own partial-refund branch plus all 4 sites in `refundProcessStripe()`). Each site's reversal computation and call moved to sit immediately before its `buildRefundParams()` / `refundPayment()` pair, same comment and rationale as the piloted files, with no other logic reordered around it.

6 new regression tests, one per file, each mirroring `WeatherRefundService`'s own "reversal fails first" guard: mock `reverseTransfer()` (the Stripe call `reverseOperatorTransfer()` makes internally) to throw, assert `buildRefundParams()` / `refundPayment()` are never reached. `PartialRefundCancelService`'s own version isolates the discount-transaction site specifically from Site 1's still-unreordered pair by letting the first `reverseTransfer()` call (the main transaction, not yet reordered) succeed and only the second (the discount transaction, the site under test) throw — proving that site's own ordering without needing Site 1 fixed first. One test-authoring bug caught along the way: an overly long `transaction_number` test fixture value silently truncated against the column's real 50-character limit (MySQL non-strict mode), making a lookup by the untruncated PHP string fail — traced via a short debugging pass, fixed by shortening the fixture's id prefix. Every new test verified via the same revert-confirm-restore discipline against its own file. 340 tests passing (334 + 6), zero orphaned rows.

Same session, user-directed: finish the remaining 11 "complicated" reorder sites. Two distinct hazards, resolved one at a time. The `DB::commit()`-sandwiched sites (6, across legacy/V3 `TourBookingRepository.php`'s `refundCustomCharge()` and `CustomChargeBookingService.php`): reading each site in full showed nothing was ever written to the DB between the method's opening `DB::beginTransaction()` and this commit besides plain reads — its real job was marking "Stripe work for this custom charge is done, never roll this back," not committing any specific write. Resolution: move the whole reversal block to sit before the refund pair (same as every other site), and let the pre-existing `DB::commit()` keep its position immediately after `refundPayment()` — now the LAST Stripe call in the new order — so it closes over both the reversal's own ledger write and the refund together, and a reversal failure leaves the transaction genuinely open and rollback-able instead of already committed with nothing left to undo. The partner-coupled sites (5, across `TicketRefundCancelService.php`, `PartialRefundCancelService.php`, and `SwitchTourPackageCardService.php`): each has a `reverseSellingCommissionTransfer()` (selling tenant, deferred-mode-only) alongside the non-partner `reverseOperatorTransfer()`, both gated behind one shared `if (paymentMode === DEFERRED_TRANSFER)` block — moved as a single unit ahead of the refund call,

with the already-mode-independent `reverseTransfer()` (3rd party ticket-cost share, fires in both modes per Decision 5) left exactly where it was. No existing test's mock setup needed adjusting — none pin call order — and the full 340-test suite re-run confirmed zero regressions before any new tests were added.

4 new regression tests, one per file (`TicketRefundCancelService`, `PartialRefundCancelService`, legacy `TourBookingRepository.php`'s `refundCustomCharge()`, `CustomChargeBookingService`), each proving the reversal now fires first. `SwitchTourPackageCardService::run()`'s own reorder is reasoned through rather than independently tested — its refund path had zero pre-existing test coverage even before this round (its one existing test covers a completely different entrypoint), and building a fresh `run()` fixture from scratch (old + new package, discount computation, a realistic `retrievePayment()` response) was judged disproportionate for a change whose correctness already follows directly from the same pattern proven 4 times over elsewhere in this exact round; the existing regression test for this file was re-run and still passes unmodified. V3's already-fixed `refundCustomCharge()` else-branch and both files' other simple sites are unaffected by this round (already reordered and tested in the prior revision). Every new test verified via the same revert-confirm-restore discipline. This closes out all 57 surveyed sites: 28 already correct, 1 not applicable, 17 simple (prior revision) + 11 complicated (this one) reordered. 344 tests passing (340 + 4), zero orphaned rows.

User-directed: "Deliberately deferred test coverage" — closed every remaining reasoned-through-only site left anywhere in this project with a real, independently-verified regression test. Correction along the way: an earlier status summary claimed V3's `editBooking()` / `deletePeopleChangePeople()` partial-refund sites were merely reasoned through *against the legacy file*'s *tested twin* — a direct grep across `tests/` found this was wrong: no test anywhere had ever called `editBooking()`, and the existing `deletePeopleChangePeople()` tests only covered the full-refund branch, so both legacy and V3 needed genuinely new tests, not confirmation of an existing one. Five gaps closed: `PartialRefundCancelService`'s Sites 4/5 (the discount-transaction whole-amount and spillover branches, deferred earlier this project by explicit user call); legacy and V3 `deletePeopleChangePeople()`'s own partial-refund branch; and legacy and V3 `editBooking()`'s own standalone "Partial Refund edit booking" section (distinct from the already-tested `EditBookingService::deletePaxFromGroupBooking()` it delegates to for group bookings only). `editBooking()`'s ~1,300-line preamble — previously assessed as disproportionate to reach safely — turned out to pose no real obstacle: a throwaway tinker probe script (building a real fixture, mocking the gateway with STDOUT tracers, calling the method directly) found the true minimal request shape in three iterations, surfacing two genuine test-infrastructure gotchas along the way — the request's `check` field needs dotted-string array KEYS (`"{rateId}.{touristIndex}"`), not a plain indexed array; and this app's default auth guard is `'customer'`, not `'admin'`, which `editBooking()`'s people-removed-email logic relies on via a guard-less `Auth::user()` call (a pre-existing, out-of-scope quirk, worked around in the test rather than fixed). Along the way, found a genuine structural difference between the two files: V3's `editBooking()` wraps its whole Stripe-facing block in its own local try/catch that only logs a failure rather than rethrowing it — confirmed via the same cap-removal verification that this makes a broken cap here fail silently (the guest refund never fires either, with no exception surfacing anywhere) rather than propagating like legacy's does, a strictly worse failure mode worth knowing about even though it wasn't in scope to change. Last, `SwitchTourPackageCardService::run()`'s refund branch — the one remaining "reasoned through by code review alone" site noted two revisions back — is now independently tested for both of its fixes at once: the ticket-cost-first cap, and the reverse-before-refund reorder, the latter confirmed via an explicit call-order tracer rather than inference. Every new test verified with the same revert-confirm-restore discipline used throughout this project. 133 tests passing in the curated `tests/Feature/Payments` suite (943 assertions, 9 new); a further 13 orphaned test bookings and 1 orphaned test package — debris from this same session's own earlier tinker-probe iterations crashing before their own cleanup ran, not from any prior session — were found and removed.

Contents

1. Executive Summary
2. Current Architecture — Destination Charges
3. Why This Is Changing (and what it does not change)
4. Target Architecture — Deferred Charges & Transfers
5. The 2027 Gate — How a Booking's Mode Gets Decided
6. Current-State Code Inventory (research findings)
7. New & Changed Components
8. Data Model Changes
9. Phased Rollout Plan
10. Testing Strategy
11. Risk Register
12. Open Decisions Needed
13. Appendix A — Full Call-Site Inventory
14. Appendix B — Stripe Connect Glossary

1. Executive Summary

Today, every card and card-reader payment on the platform is created as a Stripe destination charge: the platform's PaymentIntent carries `transfer_data.destination`, `application_fee_amount`, and (capability permitting) `on_behalf_of`, and Stripe automatically splits and pays out the tour operator's share the moment the charge succeeds. The request is to change this for any booking whose tour date falls in 2027 or later: the full charge should land in the NAT platform account first, and the operator's share should move to their connected account through a separate, later `Transfer` — while every booking for a 2026-or-earlier tour date keeps working exactly as it does today.

The good news from this research: the codebase already has a working, battle-tested implementation of the target pattern. `TransferPaymentService` (used today for tour-guide/partner-package splits) already creates a plain platform charge, computes a payout amount, calls `Transfer::create()` separately, and can reverse that transfer independently on refund — all backed by a `transfers` ledger table. This plan is fundamentally about generalizing that existing pattern into the primary payment path for 2027+ bookings, not inventing a new one.

The work is not a one-file change. Because there is no single "create a payment" choke point — 20 call sites build PaymentIntent params, and ~100+ call sites build refund params, all independently — the practical path is: (a) finish centralizing the param-building (it is 90% centralized already, via `buildConnectedPaymentData()`), (b) add one authoritative "which mode does this booking use" resolver, gated on tour date, (c) branch the existing centralized builder and the refund/dispute code on that resolver's answer, and (d) fire the operator transfer synchronously, at the same moment a destination charge already pays out today — no scheduled job, no delay, no new webhook events. Section 7 details each piece; Section 9 sequences them into low-risk phases.

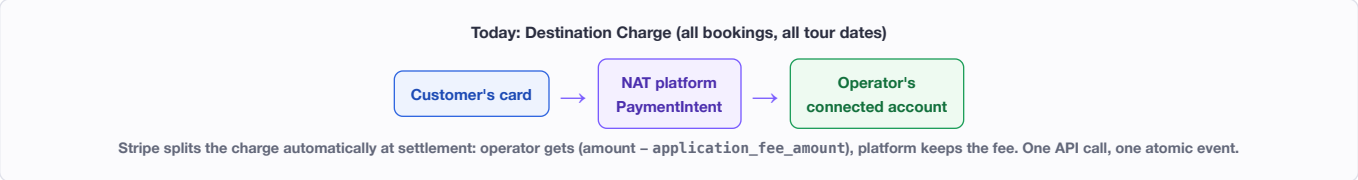
> **SCOPE CONFIRMATION**
> This plan covers card and card-reader (Stripe Terminal) payments only, matching the request. It does not change how cash/check/other offline payment methods are recorded, and it does not touch the existing tour-guide gratuity payout system (`tour_operator_staff.stripe_account_id`) — that system already transfers-only and is unaffected. It does address the overlap with the existing partner-package three-way split, since that system already performs transfers today and shares its trigger points with the new engine (Section 7.3).

> **CONFIRMED — CLEAN CUTOVER, NOT A MIGRATION**
> As of this writing (September 2026), no booking with a 2027 tour date has been sold yet — no tour operator has opened their 2027 calendar for sale. That removes the one piece of real complexity a mid-flight change like this usually carries: there is no historical 2027 destination-charge booking to reconcile, backfill, or dual-support. Every booking in the system today, and every booking that will be created before this ships, keeps running on the existing destination-charge structure completely unchanged, forever — the new deferred-transfer path only ever applies going forward, to bookings created *after* this ships whose tour date happens to fall in 2027+. The only real deadline is to have the wiring phases (Section 9) done before the first 2027 tour date is opened for sale on any tenant's calendar, whenever that turns out to be — not January 1, 2027 itself, which is irrelevant to booking dates.

> **CONFIRMED — DECISION 1 RESOLVED: TRANSFER TIMING STAYS EXACTLY AS TODAY**
> Per business decision, the operator's share moves to their connected account immediately, synchronously, at the same moment a destination charge already pays out today — not deferred to the tour date or the cancellation window. This removes the scheduled-job/sweep design this plan originally proposed (old Section 7.3/9) entirely: there is no delay to manage, no "pending payout" state to sweep, and therefore no new `transfer.*` webhook handlers needed for async reconciliation either, since the transfer either succeeds or fails inline, in the same request, the instant the charge is confirmed. What changes for a 2027+ booking is purely *which Stripe mechanism moves the money* — an explicit, code-driven `Transfer::create()` call instead of Stripe's automatic destination-charge split — not *when* it moves. The reusable engine for this (`TransferPaymentService::transferToOperator()`/`reverseOperatorTransfer()` for non-partner bookings, `transferSellingCommission()`/`reverseSellingCommissionTransfer()` for a partner-package booking's selling tenant) is built and unit-tested as of this revision — see Section 7.3.

2. Current Architecture — Destination Charges

All card-based money movement funnels through `App\Contracts\Payments\PaymentGatewayInterface`, implemented by `app/Services/Payments/StripePaymentGateway.php`. The interface is bound in `AppServiceProvider` and resolved ad hoc via `app(PaymentGatewayInterface::class)` throughout repositories and services — there is no constructor injection and no single orchestrating "PaymentService" class despite what older documentation implies (see callout below).



The single place that assembles the Connect-specific parameters is `StripePaymentGateway::buildConnectedPaymentData()`:

```
public function buildConnectedPaymentData(string $destinationId, int $feeAmountCents, ?string $statementDescriptor = null): array
{
    $data = [
        'application_fee_amount' => $feeAmountCents,
        'transfer_data'          => ['destination' => $destinationId],
    ];
    // on_behalf_of requires card_payments capability; omitted for transfers-only accounts (e.g. Canada)
    if ($this->connectedAccountHasCardPayments($destinationId)) {
        $data['on_behalf_of'] = $destinationId;
    }
    if ($statementDescriptor !== null) {
        $data['statement_descriptor_suffix'] = $statementDescriptor;
    }
    return $data;
}
```

This method is spread (`...`) into the PaymentIntent params array at essentially every one of the 20 creation call sites catalogued in Section 6 / Appendix A — new bookings, "make a payment" (balance due), combo cart checkout, edit-booking, add-ons, tips, Payment Links, mobile app checkout, and both the web and Terminal card-reader flows. The connected account id itself is a plain string column, `stripe_destination_id`, stored on `tour_operators`, `tour_packages` (for partner packages), and `customers` (for affiliate bookings) — there is no dedicated "connected account" model.

Refunds are the mirror image and are not centralized: roughly 100+ call sites across two ~18,000-line repository classes and a dozen refund/cancel services each independently build:

```
$refundParams = [
    'payment_intent'         => $trans->transaction_number,   // the Stripe PaymentIntent id
    'amount'                 => $partial * 100,
    'reverse_transfer'       => true,   // pulls the operator's share back automatically
    'refund_application_fee' => true,   // sometimes conditional / omitted
];
app(PaymentGatewayInterface::class)->refundPayment($refundParams);
```

`reverse_transfer` and `refund_application_fee` only mean anything because the original charge is a destination charge with an automatic transfer attached. This assumption is baked into every refund call site, and into dispute handling (`DisputeService::handleDisputeLoss()`, which reads `$charge->transfer` — a field that only exists because of `transfer_data.destination`).

> **DOCUMENTATION DRIFT FOUND DURING RESEARCH**
>
> `CLAUDE.md` references `app/Services/Payments/StripePaymentService.php` and `app/Http/Controllers/Common/StripeWebhookController.php` — neither exists. The real files are `StripePaymentGateway.php` and `app/Http/Controllers/StripeWebhookController.php` (no `Common/` folder). `PaymentBreakdownRepository` similarly lives at `app/Repositories/Report/PaymentBreakdownRepository.php`, not `app/Repositories/`. This plan uses the verified real paths throughout; the project's CLAUDE.md would benefit from a correction pass independent of this migration.

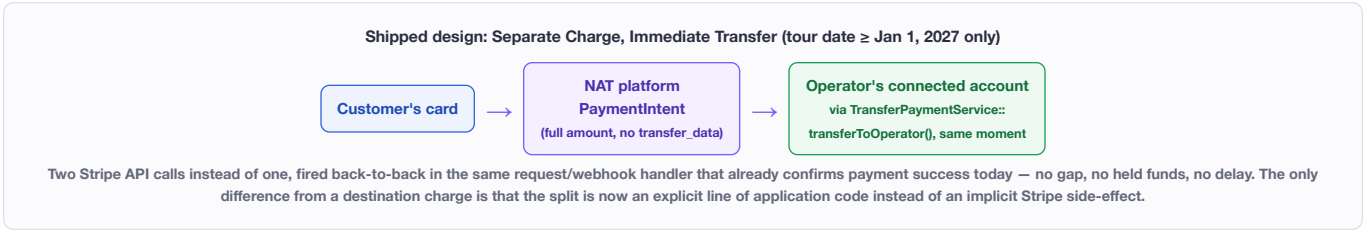
3. Why This Is Changing

The request states the destination for 2027+ bookings should be "transfer instead of destination in the PaymentIntent," with funds landing at NAT first, using the same timing the platform already uses today — the operator is still paid as soon as the charge is confirmed, nothing about that changes. This plan's initial draft speculated that the motivation was settlement-timing risk on far-future travel (holding funds until closer to the tour date, to reduce clawback exposure on cancellations/disputes for trips booked many months out). That speculation is now superseded — confirmed business decision: timing stays exactly as it is today, only the underlying Stripe mechanism changes.

What actually changes, then, is purely mechanical: instead of one PaymentIntent with `transfer_data.destination` (where Stripe silently does the split as part of settling the charge), a 2027+ booking uses a plain platform charge followed by an explicit, code-driven `Transfer::create()` call made by the application itself, immediately, in the same request. This gives NAT direct visibility into and control over each payout as a first-class event in its own codebase and ledger (the `transfers` table already records every one, per-booking, going back to the existing partner-split feature) rather than an invisible side-effect of Stripe's own charge-settlement plumbing — likely valuable for auditing, reconciliation, or future flexibility, even though the money moves at the same moment either way today.

> **RESOLVED — DECISION 1**
> **See Section 12, Decision 1 and the callout in Section 1: the transfer fires synchronously, immediately after payment confirmation, with no delay and no scheduled job. Sections 4, 7, and 9 below reflect this as the shipped design, not a proposal.**

4. Target Architecture — Separate Charge, Synchronous Transfer



4.1 Recommended PaymentIntent shape for 2027+ bookings

Stripe's own documented "separate charges and transfers" pattern supports keeping `on_behalf_of` on the PaymentIntent without `transfer_data` — this preserves the connected account's statement descriptor on the customer's card statement (so the customer experience is unchanged) while still landing 100% of the funds on the platform balance with no automatic split:

```
// buildConnectedPaymentData() — deferred-mode branch (new)
$data = [];
// no transfer_data.destination, no application_fee_amount — full amount stays with NAT
if ($this->connectedAccountHasCardPayments($destinationId)) {
    $data['on_behalf_of'] = $destinationId;   // keeps statement descriptor; existing capability check is reused as-is
}
if ($statementDescriptor !== null) {
    $data['statement_descriptor_suffix'] = $statementDescriptor;
}
return $data;
```

Everything else about PaymentIntent creation — amount, currency via `currencyFormation()`, customer id, metadata, idempotency key handling in `StripePaymentGateway::createPayment()` — is untouched. The fee amount that would have been `application_fee_amount` is instead computed later, at transfer time, as "platform's cut" subtracted from the transfer amount (exactly how `TransferPaymentService::transferToPartner()` already computes `$transferAmount` today).

4.2 The transfer itself — shipped as `TransferPaymentService::transferToOperator()`

Built and unit-tested (10 tests, `tests/Unit/TransferPayment/TransferToOperatorTest.php`) as a sibling to the existing `transferToPartner()` on the same class, reusing its exact math and the `transfers` ledger, distinguished by the new `transfer_type='primary_operator'` discriminator (Section 8). This is the non-partner path only — a partner-package booking uses a different pair, `transferSellingCommission()` / `transferToPartner()`, instead (Section 6.7/7.3):

```
public static function transferToOperator($data): bool
{
    $transferAmount = round($data['amount'] - $data['platform_fees'], 2);
    ...
    $transferParams = [
        'amount'         => $transferAmount * 100,
        'currency'       => currencyFormation(code: true),  // see 7.9 — the hardcoded 'usd' bug is now FIXED, not just flagged
        'destination'    => $destinationId,
        'transfer_group' => $transferGroup,                 // same transfer_group already set on the original PaymentIntent
        'description'    => "Operator payout for Booking #$bookingId",
        'metadata'       => [...],
    ];
    try {
        $transfer = app(PaymentGatewayInterface::class)->createTransfer($transferParams);
        // transfer_status='Completed', transfer_id recorded
    } catch (Throwable $e) {
        // logged; transfer_status='Failed' recorded — the charge already succeeded, so this must
        // never throw back into booking confirmation. See Section 11's updated risk row.
    }
    DB::table('transfers')->insert([... 'transfer_type' => 'primary_operator', ...]);
}
```

Reversal is the sibling `reverseOperatorTransfer($tourBookingId, $transactionId, $amount)`, filtered on `transfer_type='primary_operator'` so it can never resolve a partner-split row for the same booking by mistake (regression-tested explicitly — Section 10). Both methods, and every pre-existing lookup query in `TransferPaymentService`, were updated with an explicit `transfer_type` filter as part of this change, since the table now serves two purposes.

> **NOT YET WIRED ANYWHERE**
>
> These methods exist, are tested, and change no runtime behavior on their own — nothing calls `transferToOperator()` yet. Wiring it into the actual booking-confirmation call sites is the next piece of work; Section 6.6 and Section 9 lay out exactly where.

5. The 2027 Gate — How a Booking's Mode Gets Decided

5.1 Date source

The tour's actual departure date is not `tour_bookings_master.created_at` — it is reached via `tour_bookings_master.tour_slot_id → tour_slot_times.id`, and `tour_slot_times.date` already carries a denormalized copy of the date (no need to join out to `tour_slot_dates`). This column is already loaded on the `$info` object at every PaymentIntent-creation call site, because it's needed for pricing/description text today — so the gate adds no new query.

> **TIMEZONE CORRECTION (FOUND DURING IMPLEMENTATION)**
> This section originally warned that the boundary check must convert through the tenant's timezone (`tourOperator()->timezone`), per the project's general date-handling rule. Building `PaymentModeResolver` (Section 7.1, now shipped) surfaced that this doesn't actually apply here: `tour_slot_times.date` is a plain SQL `DATE` column with no time-of-day component at all — it already *is* the tenant's own local tour day, not a UTC instant that needs converting. A bare lexical string comparison against `'2027-01-01'` is therefore correct and timezone-safe as shipped, with no conversion step needed. (The general project rule still applies elsewhere, to genuine timestamp columns — this is a narrow, verified exception for this one specific date source.)

5.2 Locking the decision at payment time

The mode must be decided once, at PaymentIntent creation, and stored immutably — not re-derived later from the booking's current tour date. Two reasons: (1) Stripe's params can't be retroactively changed on a completed PaymentIntent, so refund/dispute code must know which regime the original charge used regardless of what the booking looks like now; (2) a booking's tour date can change after payment (reschedule / switch-package flows) — if the date moves across the boundary, the *original* charge's settlement behavior must not silently change underneath already-completed money movement. Recommendation: persist the decision as a `payment_mode` enum column (Section 8) written at creation time and read by every downstream refund/dispute/reporting code path — never recomputed from the current tour date.

5.3 Combo / multi-package carts

A single cart checkout can produce multiple `tour_bookings_master` rows sharing one `cart_id`, each with its own `tour_slot_id` and therefore its own date — confirmed via `BookingService::comboBookingsData()` and the `isComboBooking()` helper. Because one PaymentIntent today only carries a single `transfer_data.destination`, a combo cart already assumes one primary settlement target per charge; this plan keeps that constraint rather than trying to split one PaymentIntent three ways.

> **RECOMMENDATION**
> Evaluate all booking rows in the cart; if any row's tour date is ≥ Jan 1, 2027 (tenant-local), the whole PaymentIntent uses deferred mode. This is the conservative choice — it never lets a 2027 booking sneak through in destination-charge mode — at the cost of also deferring a 2026 line item that happens to share a cart with a 2027 one. The deferred-transfer step (which runs per distinct `stripe_destination_id`, see 7.3) can still pay out each package's own connected account independently and on its own schedule once triggered, which is actually more flexible than what a single destination-charge PaymentIntent could ever do for a mixed-operator cart.

6. Current-State Code Inventory (Research Findings)

Full grep-verified inventory; line numbers are also captured in Appendix A. This section summarizes by category.

6.1 PaymentIntent creation (20 call sites, all funnel through `buildConnectedPaymentData()` except two noted exceptions)

FLOW	FILE	NOTES
Web/POS new booking, make-payment, combo, edit-booking	`app/Utils/StripeUtils.php` (4 functions)	Shared builder `createStripeIntent()` at line 639 is the co core
Legacy cart card-checkout	`app/Repositories/TourBooking/TourBookingRepository.php:1953`	Uses `PaymentMethod::create` + `confirm:true`
Mobile app card reader / card-present (cart & single-booking)	`TourBookingRepository.php:11675,11686,11727,11738`	Via `getStripeAttributesForAppCardReader()` / `...AppTapT`
Web custom charge	`TourBookingRepository.php:17220`	
V3 equivalents of all of the above	`app/Repositories/TourBooking/V3/TourBookingRepository.php:2369,14494,14509,20179`	Parallel implementation, must be updated in lockstep
Admin Terminal / card-reader	`app/Repositories/Pos/CardReaderRepository.php:286,288`	Two branches: `connect=true` (direct charge, unused today `connect=false` (destination charge, the live path)
Tour-guide tip (Tap to Pay)	`app/Repositories/TipCenter/GratuityRepository.php:1328`	
Guide-dashboard tip creation	`app/Services/Booking/StripeService.php:1318`	
Emailed Payment Link checkout	`app/Services/Booking/PaymentLinkService.php:257,296`	Checkout Session, same builder spread into `payment_intent_data`
Mobile API v2 new booking	`app/Http/Controllers/API/V2/TourBookingController.php:1941`	
Exception: cross-tenant "Network" marketplace booking	`app/Services/Network/Stripe/StripeService.php:50`	Plain platform charge already — no Connect params at all; o scope, unaffected

6.2 Refunds — ~100+ call sites, not centralized

FILE	APPROX. OCCURRENCES
`app/Repositories/TourBooking/TourBookingRepository.php`	22
`app/Repositories/TourBooking/V3/TourBookingRepository.php`	21
`app/Services/CancelBooking/*.php` (Partial/Ticket/Weather refund-cancel services)	~11
`app/Services/Booking/EditBookingService.php`	6
`app/Services/RefundBooking/*.php` (Ticket/Weather refund services)	~6
`app/Repositories/TourBooking/TourBookingAddonRepository.php`	4
Remaining scattered (StripeService, ChangePeopleAddonService, CustomChargeBookingService, SwitchTourPackageCardService)	~4

All pass `reverse_transfer` (near-universal) and `refund_application_fee` (sometimes conditional). None of them check what kind of charge they're refunding — every one assumes destination-charge semantics.

6.3 The existing "separate transfer" precedent — `TransferPaymentService`

`app/Services/TransferPayment/TransferPaymentService.php` (partner/tour-guide split, layered on top of the primary destination charge) already implements exactly the pattern this plan needs, just for a secondary split rather than the primary payout:

- `transferToPartner()` — computes payout amount, calls `createTransfer()`, records a row in the `transfers` table (`stripe_account_id`, `transfer_id`, `transfer_amount`, `transfer_status`).
- `reverseTransfer()` — sums prior reversals from the ledger, guards against over-reversal, calls `reverseTransfer()` on the gateway.
- `transfer_status` enum: `Pending / Completed / Failed / Reversed` — already a usable state machine.

> **REUSE, DON'T REBUILD — NOW DONE**
> This service and its `transfers` table were generalized into the new engine as planned: `transferToOperator()`/`reverseOperatorTransfer()` for non-partner bookings, plus `transferSellingCommission()`/`reverseSellingCommissionTransfer()` for a partner-package booking's selling tenant — all shipped in the same file, reusing the same ledger with a three-value `transfer_type` discriminator. Per Decision 1 (resolved), everything fires at the *same* trigger point `transferToPartner()` already uses — synchronously, on payment confirmation — not a separate scheduled trigger. See Section 7.3.

6.4 Dispute handling depends on the automatic transfer

`DisputeService::handleDisputeLoss()` retrieves the charge with `expand: ['transfer']` and reverses `$charge->transfer` directly — that field only exists because of `transfer_data.destination`. For a deferred-mode charge, `$charge->transfer` will be null (no transfer_data), so this exact code path breaks silently for 2027+ bookings unless updated (Section 7.5).

6.5 Webhooks — no transfer event handling, and none needed

`app/Http/Controllers/StripeWebhookController.php` handles `payment_intent.succeeded/failed/canceled`, `invoice.*`, `payout.*` (a different, unrelated "balance-to-bank" system), and `charge.dispute.*`. Nothing listens for `transfer.created`, `transfer.reversed`, or `transfer.failed` — and per Decision 1 (resolved: synchronous transfer, no delay), nothing needs to. A transfer's success or failure is known inline, in the same call that creates it; there is no asynchronous gap for a webhook to reconcile.

6.6 Exact wiring points — where `transferToOperator()` needs to be called from

Researched directly against the real code (not the generic 20-call-site inventory in Appendix A) to answer the specific question this plan now needs answered: since the transfer must fire synchronously the instant a card payment is confirmed, *which exact methods already know "this payment just succeeded"*? Two independent findings converge on the same answer.

Finding 1 — the existing partner-split precedent already answers this. `TransferPaymentService::transferToPartner()` is called from exactly 19 call sites, every one of them inside booking-repository/controller/service code that executes synchronously as part of the original checkout/payment/edit-booking HTTP request — never from inside a webhook handler. The clearest example, `createBookingCard()` (`app/Repositories/TourBooking/V3/TourBookingRepository.php:2491–2684`) — the method that finalizes a booking the instant a client hands back a confirmed `payment_intent_id` — inserts the `transactions` row and then, only when `$isPartnerPackage == 1`, calls `transferToPartner()` inline, guarded by a simple `$transactionCount == 0` idempotency check against the `transactions` table itself:

```
if ($transactionCount == 0 && empty($data['skip_tour_side_ledger'])) {
    $transactionId = TransactionService::storePaymentInTransaction(...);
    $isPartnerPackage = $data['isPartnerPackage'] ?? 0;
    if ($isPartnerPackage == 1) {
        $payoutData = [ 'amount' => $data['total'], 'platform_fees' => $data['serviceCommission'], ... ];
        TransferPaymentService::transferToPartner($payoutData);
    }
}
```

The 19 sites span `TourBookingRepository.php` / `V3/TourBookingRepository.php` (`createBooking`, `createComboBooking`, `createPaymentIntent`, `createBookingCard`, `createComboBookingCard`, `changePeopleBooking`, `paymentNewBooking`, `paymentCartBookings`, `applyDiscount`, `createEditDiscountPaymentIntent`), `API/V2/TourBookingController.php` (x2), and `PaymentLinkService` / `ExtendBooking*Service` / `SwitchTourPackage*Service`. At every one, booking id, transaction id, computed commission, and (for Card) destination id + `transfer_group` are already in local scope — no re-fetch needed. Tour date is not currently passed into any of these `$payoutData` arrays, so each site also needs a small addition to resolve and pass the booking's mode.

Finding 2 — the async webhook pipeline is a parallel, secondary confirmation path, not the primary one, and mostly dead weight for this purpose. `StripeWebhookController::handleStripePaymentIntentSucceeded()` dispatches to six handler classes under `app/Services/Webhook/` by `metadata->section`. None of the six call `TransferPaymentService` today. Auditing them individually:

HANDLER	STATUS
`StripePaymentIntentSucceededForNewBooking`	Live — writes `transactions`, but only patches metadata onto the charge Stripe already auto-transferred; would need the same `$transactionCount==0`-guarded addition as the synchronous path, as a fallback for the (presumably rare) case where the client never calls back after Stripe confirms.
`StripePaymentIntentSucceededForMakePayment`	Live — same shape, plus its own fallback fee recomputation (`calculatePlatformFees()`) for the rare case the primary value wasn't already persisted.
`StripePaymentIntentSucceededForDiscountPayment`	Live, delegates to `TransactionService`.
`StripePaymentIntentSucceededForTip`	Live but irrelevant — tips have no operator split.
`StripePaymentIntentSucceededForPaymentLink`	Dead code — its entire body is a commented-out call; confirmed a no-op today.
`StripePaymentIntentSucceededForTapToPayTip`	Effectively unreachable — `config('constants.payment_sections.tap_to_pay_tip_payment')` doesn't exist in `config/constants.php`, so the metadata key that would route to it is never set, and the outer `isset($paymentIntent->metadata->section)` guard never lets execution reach this case. Pre-existing bug, unrelated to this plan, worth its own separate fix.

Idempotency between the synchronous and async paths already exists and requires no new mechanism: both check `Transaction::where('transaction_number', $paymentIntentId)->where('tour_booking_id', $bookingId)->where('status', 'Completed')->count()` before writing anything, so whichever path lands its row first "wins" and the other correctly no-ops — the same guard that must also wrap the new `transferToOperator()` call at each site.

> **BUG FOUND, OUT OF SCOPE BUT WORTH FLAGGING**
>
> Call site #10 (`applyDiscount()`, `V3/TourBookingRepository.php:19335`) reads `$tourPackage->partner_destination_id` — that column does not exist anywhere in the schema (only `stripe_destination_id` does). This resolves to `null` silently via Eloquent's magic getter, meaning Card-mode partner transfers from the "apply discount" flow likely target a null destination today. Pre-existing, unrelated to 2027 — flagged for a separate fix.

6.7 Partner packages, traced precisely — what "partner" actually means here

Clarified directly by the business: a "partner package" is a package physically listed and sold through one tenant's own storefront (the selling tenant), but actually owned/fulfilled by a different tenant (the 3rd party). On a successful booking, three parties are paid: NAT keeps a service commission, the selling tenant keeps a selling commission, and the remaining ticket cost goes to the 3rd party. Tracing the real PaymentIntent-creation code end to end (`StripeUtils::createStripeIntent()`, fed by `TourPackageService::getTourPackageSlotInfo()`) resolves this cleanly:

- `transfer_data.destination` on today's destination charge is `tour_operators.stripe_destination_id` — the SELLING tenant's own connected account — for every booking, identically whether or not it's a partner package. `tour_packages.stripe_destination_id` (the 3rd party's account) is a separate column, never used as the original charge's destination.
- `application_fee_amount` on that same charge equals only NAT's own `serviceCommission` — it never folds in the selling tenant's commission.
- Consequence: the destination charge's automatic transfer sends `amount − serviceCommission` to the selling tenant in one lump sum — which is ticket cost + the selling tenant's own commission, combined. Then, separately, `TransferPaymentService::transferToPartner()` sends the ticket-cost portion again, independently, to the 3rd party's `tour_packages.stripe_destination_id`, drawing from the platform's own Stripe balance (a plain, unscoped `Transfer::create()` call — confirmed via `StripePaymentGateway::createTransfer()`, no `stripe_account` option).

> **CORRECTION — AN EARLIER REVISION OF THIS SECTION WAS WRONG**
>
> The first pass at this design reused `transferToOperator()` for the selling tenant's payout, sending them the full `amount − platformFees` remainder — deliberately copying what the destination charge's automatic transfer does today, on the reasoning that current partner economics should be preserved as-is. The business then clarified the actual intended rule directly: the selling tenant is owed only their own configured "Partner Fee" (their selling commission) — nothing more — and when that fee is 0, the selling tenant receives nothing at all, with the full ticket cost going straight to the 3rd party. That correction is reflected below and in the shipped code; it also happens to fully resolve the double-disbursement concern the earlier draft could only flag, not fix.

DECISION 5 & 6 — RESOLVED: CORRECT PARTNER-PACKAGE TRANSFER DESIGN

For a deferred-transfer-mode partner-package booking, `transferToOperator()` is not called at all. Instead, two independent transfers replace what the destination charge used to conflate into one lump sum:

- `transferSellingCommission()` (new) — sends the selling tenant only their configured Partner Fee. If that fee is 0, no Stripe call is made and no ledger row is written; the selling tenant gets nothing for this booking.
- `transferToPartner()` — unchanged, sends the 3rd party their ticket cost independently, exactly as it already does today.

Because the selling tenant is never routed the ticket-cost portion under this design, there is nothing left to reconcile back out of their account — the double-disbursement risk the earlier draft flagged as Decision 6 doesn't arise in the first place. NAT's own service commission is never transferred to anyone, in either case. Regression-tested: `tests/Unit/TransferPayment/TransferToOperatorTest.php`'s `test_partner_package_booking_with_nonzero_partner_fee_pays_both_selling_tenant_and_third_party` and `test_partner_package_booking_with_zero_partner_fee_pays_only_third_party`, plus reversal coverage (`reverseSellingCommissionTransfer()`, including the no-op case when the fee was 0).

SEPARATE PRE-EXISTING BUG FOUND, UNRELATED TO 2027

A same-tenant cart combining packages from two different `tour_operator_id` rows, paid through the "raw manual card entry" path (`createPaymentIntentForCart()` → `CardBookingService::run()`), has zero `is_partner_package`/`transferToPartner()` handling at all — it would silently skip the 3rd-party payout entirely. The sibling path where the client pre-creates the PaymentIntent (`createComboBookingCard()`) handles this correctly, looping per cart item. Flagged for its own separate fix; unrelated to this migration.

Also confirmed directly: there is no cross-tenant-database combo cart capability today — "partner package" always means a package physically stored in the selling tenant's own database, routed to a different Stripe account by column, never a live join across two tenants' databases in one checkout.

7. New & Changed Components

7.1 Payment mode resolver shipped

Done. `App\Services\Payments\PaymentModeResolver`, with `resolve(iterable $tourDates): string` and a `resolveForSingleDate()` convenience wrapper, returning `PaymentModeResolver::DESTINATION` or `::DEFERRED_TRANSFER`. Pure and dependency-free (no DB, no Stripe, no tenancy) by design, so nothing else needs to change for it to be unit-tested in isolation. Implements the Section 5.3 "any row ≥ 2027" combo-cart rule. 12 unit tests, `tests/Unit/Services/Payments/PaymentModeResolverTest.php`, all passing. Not yet called from anywhere — see 7.2.

7.2 `buildConnectedPaymentData()` branch shipped

Done. The method (and the interface contract) now takes a trailing `string $mode = PaymentModeResolver::DESTINATION` parameter — every one of the 20 existing creation call sites is unaffected by default, since none of them pass this argument yet. When `DEFERRED_TRANSFER` is passed, it omits `transfer_data` / `application_fee_amount` entirely and preserves `on_behalf_of` / `statement_descriptor_suffix` exactly as before. 5 unit tests, `tests/Unit/Services/Payments/StripePaymentGatewayDeferredModeTest.php` — regression-guards the unchanged default behavior and verifies both branches of the new one, using Stripe SDK's own `ApiRequestor::setHttpClient()` test seam (a fake HTTP client, not a real network call) since `connectedAccountHasCardPayments()` otherwise hits Stripe directly and the class has no constructor-injection seam.

7.2a First end-to-end flow wired — the primary new-booking path shipped

Done, as a proof of the whole pattern before replicating it across the other 19 sites. The single (non-combo) new-booking flow is wired completely, creation through confirmation:

- Creation — `StripeUtils::createStripePaymentIntentNewBooking()` resolves the mode via `PaymentModeResolver::resolveForSingleDate($info->date)` and threads it through `createStripeIntent()` into both `buildConnectedPaymentData()` *and* the PaymentIntent's own metadata (a new top-level `payment_mode` key, alongside the existing `bookings` / `tenant_id` keys) — this is what locks the decision in per Section 5.2, since the `transactions` row doesn't exist yet at this point.
- Confirmation — `V3/TourBookingRepository::createBookingCard()` reads `$paymentIntent->metadata->payment_mode` back (falling back to `DESTINATION` when absent — covers PaymentIntents created before this metadata key existed, or via a creation call site not yet wired), passes it through the also-newly-extended `TransactionService::storePaymentInTransaction()` (which now persists `transactions.payment_mode` on the insert branch only — an existing "Initialized" reservation row being flipped to Completed is deliberately left untouched, since its mode was already decided whenever that placeholder was first created), then branches: non-partner bookings call the new `transferToOperator()`; partner-package bookings call `transferSellingCommission()` *in addition to* the pre-existing, completely unchanged `transferToPartner()` call.
- The selling tenant's/operator's own connected account is resolved via `$tourBookingDetails->tourPackage?->tourOperator?->stripe_destination_id` — the same field the original destination charge already uses (Section 6.7) — not a column on `tour_bookings_master` itself (verified against the real schema; an earlier draft of this edit incorrectly assumed such a column existed).

NOW VERIFIED END-TO-END, NOT JUST BY REGRESSION

`tests/Feature/Payments/DeferredTransferNewBookingFlowTest.php` calls the real `createBookingCard()` against a real booking (via `CreateABooking`, tour date 2027-06-15) with only the Stripe gateway mocked, and asserts on real database state: `transactions.payment_mode` is actually persisted as `'deferred_transfer'`, `transferToOperator()` actually calls `createTransfer()` with the tenant's real connected-account id and the correct cents amount, and a matching `primary_operator` `transfers` row lands correctly. A second test proves the regression case: a PaymentIntent with no `payment_mode` metadata at all (e.g. from a not-yet-wired creation site) falls back to destination mode and never calls `transferToOperator()`. All 4 shared functions this touched (`buildConnectedPaymentData()`, `createStripeIntent()`, `storePaymentInTransaction()` — ~40 callers across the codebase — and `createBookingCard()`) got a new trailing optional parameter, so every existing caller is unaffected by default; the pre-existing `MakePaymentSameLoginDifferentIpTest` (4 tests) and `ApplicationFeeCentsRoundingTest` (6 tests) suites, which exercise adjacent code, still pass unchanged. 39 tests pass across the whole 2027 feature now, zero test-data orphans. This flow is now trustworthy as the template for the remaining 19 call sites.

FRAGILITY FOUND, NOT FIXED — `CARDBOOKINGSERVICE::RUN()`'S UNGUARDED PAYOUT LOOKUP

This method (called immediately after the block above, for the "Full Payment" history entry) reads `Payout::where('tour_booking_id', ...)->where('transaction_id', ...)->where('transfer_status', 'Completed')->first()` with no `transfer_type` filter — a pre-existing gap now that this table serves three purposes. It happens to still resolve correctly for both new payout types by coincidence (`primary_operator` rows always carry `commission_amount=0`, and `selling_commission` rows carry the same dollar figure `transferToPartner()`'s own `partner_split` row would show — both semantically right for what this method reads them for) — but this is a fragile coincidence, not a deliberate guarantee, and is worth its own small fix (an explicit `transfer_type` filter) before this code is trusted further. Out of scope for this pass.

7.2b Second end-to-end flow wired — Make Payment (balance-due) shipped

Done. The "Make Payment" flow for an existing Pay-Later booking's outstanding balance, creation through confirmation:

- Creation — `StripeUtils::createStripePaymentIntentMakePayment()` / `updateStripePaymentIntentMakePayment()` (the latter used when a still-open PaymentIntent's amount needs refreshing before confirmation) both resolve the mode from `$tourBooking->Date` via a shared private helper, `buildMakePaymentIntentData()`, and thread it into `createStripeIntent()` / `buildConnectedPaymentData()` exactly as the new-booking flow does. The update path also re-asserts `payment_mode` in the metadata explicitly rather than relying solely on Stripe's metadata-merge behavior to preserve it.
- Confirmation — `V3/TourBookingRepository::paymentNewBooking()` reads `$paymentIntent->metadata->payment_mode` back (only inside the Card branch — Cash/Invoiced payments have no PaymentIntent at all and correctly stay at the `DESTINATION` default, matching the plan's card-only scope), passes it to

`storePaymentInTransaction()`, then branches exactly as `createBookingCard()` does: non-partner → `transferToOperator()`; partner-package → `transferSellingCommission()` + unchanged `transferToPartner()`.

- This flow's own query for `$tourBooking[0]` only ever selected `tp.stripe_destination_id` (the 3rd party's column, used for the existing partner transfer) — a new `t_o.stripe_destination_id AS tour_operator_stripe_destination_id` column was added to that raw SQL SELECT to source the selling tenant's own account for the new transfers, keeping both fields distinct and correctly labeled.

> **REAL BUG FOUND AND FIXED BY THE DEDICATED TEST — NOT CAUGHT BY REGRESSION ALONE**
> This flow has its own concurrency-safe "reservation" pattern: a placeholder `transactions` row (status `Initialized`) is inserted the moment Make Payment is first requested, *before* the PaymentIntent itself exists, specifically so a second concurrent request finds it and reuses the same PaymentIntent instead of creating a duplicate (the exact mechanism behind the `MakePaymentSameLoginDifferentIpTest` suite). That reservation insert never set `payment_mode` at all — it silently took the column default. Since `storePaymentInTransaction()`'s "flip an existing Initialized row to Completed" branch deliberately does not touch `payment_mode` (Section 7.2a's own reasoning: "already decided when first created") that assumption was false for this specific reservation row, and the mode would have silently stayed `'destination'` forever — completely defeating the gate for every Make Payment on a 2027+ booking. Caught immediately by `DeferredTransferMakePaymentFlowTest` asserting on real persisted state, not by code review or regression alone. Fixed by resolving and setting `payment_mode` at the moment the reservation row is first inserted (and again when an existing canceled reservation is reused for a fresh attempt) in `StripeService::stripeCreatePaymentIntentForMakePayment()` — restoring the truth of Section 7.2a's original assumption rather than weakening the "don't touch it on update" rule. Worth checking for this same shape (a pre-created placeholder row whose mode is never set) at each of the remaining 18 confirmation call sites — it is exactly the kind of gap that passes a syntax check and even a naive smoke test while still being completely wrong.

1 additional feature test, `tests/Feature/Payments/DeferredTransferMakePaymentFlowTest.php` — creates a real booking via `CreateABooking` (tour date 2027-06-15), calls the real `stripeCreatePaymentIntentForMakePayment()` then the real `paymentNewBooking()`, mocking only the Stripe gateway (using `Stripe\StripeObject::constructFrom()` for the fake PaymentIntent rather than a plain `stdClass`, since production code reads it both as an object *and* via array access — a plain `stdClass` only supports the former and throws on the latter), and asserts the PaymentIntent's metadata really carries `payment_mode` at creation, and that `transactions.payment_mode` /the `primary_operator` payout row are correct after confirmation. 40 tests pass across the whole 2027 feature now.

7.2c Third end-to-end flow wired — the combo cart shipped

Done. The multi-booking "cart" checkout, where several `tour_bookings_master` rows sharing one `cart_id` are paid under a single PaymentIntent (Section 5.3's combo-cart rule):

- Creation — `StripeService::stripeCreatePaymentIntentForMakeComboPayment()` now collects each cart item's own tour date into `$cardBookingsData[]`; `StripeUtils::createStripePaymentIntentMakeComboPayment()` resolves the mode via `PaymentModeResolver::resolve(array_column($tourBookings, 'date'))` — the plural-input form, not `resolveForSingleDate()` — so a single 2027+ line item correctly forces the *whole* cart's one PaymentIntent to deferred mode, exactly as Section 5.3 specifies.
- Confirmation — `V3/TourBookingRepository::paymentCartBookings()` reads the mode back once (it's one PaymentIntent for the whole cart) and passes it to `storePaymentInTransaction()` for *every* cart item — each 2026-dated item in a cart containing a 2027-dated one is correctly persisted as `deferred_transfer` too, matching the single shared decision the PaymentIntent itself was created under. Each item then gets its own independent `transferToOperator()` / `transferSellingCommission()` call, using that item's own already-collected `destination` / `package_id` — a combo cart can span multiple operators, and Section 5.3 already established the transfer engine handles that per-destination, not per-PaymentIntent.
- This flow does not have the pre-created-reservation-row pattern that caused the Make Payment bug (Section 7.2b) — confirmed by reading `stripeCreatePaymentIntentForMakeComboPayment()` in full: it goes straight from a duplicate-payment count check to PaymentIntent creation, no placeholder `Initialized` row involved.

1 additional feature test, `tests/Feature/Payments/DeferredTransferComboCartFlowTest.php` — two real bookings via `CreateABooking` sharing one `cart_id` (one dated 2026-06-15, one 2027-06-15), proving the specific cross-item behavior that would be easy to get subtly wrong: the 2026 booking's own transaction is *still* persisted as `deferred_transfer` (the cart's shared decision, not its own date), and `createTransfer()` fires exactly twice — once per booking, independently. 41 tests pass across the whole feature now, zero orphaned rows.

7.2d Fourth end-to-end flow wired — Change People shipped

Done. The admin "Change People" (add/remove pax on an existing booking) Card flow, with a useful discovery along the way: `StripeUtils::createStripePaymentIntentEditBooking()` turns out to be a single shared creation helper used by three separate admin-editing features — Change People, Change Payment Method, and Switch Package (all three call it from `StripeService.php`) — so resolving the mode inside that one function covers all three creation sides at once, the same leverage `buildConnectedPaymentData()` already gave Phase 2 generally.

- Creation — the shared helper now resolves mode from `$info->date` and threads it through, identically to the other three flows.
- Confirmation — two different paths, only one newly wired. Tracing `V3/TourBookingRepository::changePeopleBooking()` (itself ~3,250 lines) found it branches on API vs. web: a mobile-API confirmation reuses `createBookingCard()` directly — already fully wired in Section 7.2a, so Change People's mobile path was already correct with zero new code. A web confirmation instead calls a different, structurally near-identical method, `createPaymentIntent()`, which needed the exact same treatment (read `payment_mode` back, pass it to `storePaymentInTransaction()`, dispatch `transferToOperator()` / `transferSellingCommission()`) and has now received it.
- Confirmed `BookingService::updateCardReaderIntent()` (this path's PaymentIntent-refresh call) only patches Stripe metadata/description — no `transactions`-table interaction at all — so it does not share the Make Payment reservation-row bug shape (Section 7.2b); one less flow to worry about there.

TESTING APPROACH DELIBERATELY SCOPED NARROWER HERE — STATED PLAINLY, NOT GLOSSED OVER

Unlike the first three flows, this one is tested by calling the two touched methods directly — `createStripePaymentIntentEditBooking()` and `createPaymentIntent()` — rather than through the full `changePeopleBooking()` dispatcher. That method's own add/remove-pax/discount business logic is substantial and orthogonal to this change; simulating it fully would be its own large undertaking. Both target methods are public and already proven safe to call this way, the same pattern `createBookingCard()` was tested with directly in Section 7.2a. 2 new tests, `tests/Feature/Payments/DeferredTransferChangePeopleFlowTest.php`, both passed on first try — asserting the creation side's metadata and the confirmation side's persisted `transactions.payment_mode` plus the `primary_operator` payout row. 43 tests pass across the whole feature now, zero orphaned rows.

7.2e Fifth end-to-end flow wired — Change Payment Method shipped

Done. Completes the second of the two "bonus" flows Section 7.2d's shared creation helper had already put ahead: a booking originally settled as Cash/Invoiced gets switched to Card via `V3/TourBookingRepository::changeHistoryPaymentType()`.

- This method is structurally different from every other confirmation site: it updates an existing `transactions` row directly via Eloquent property assignment (`$transaction->via = 'Card'; $transaction->save();`) rather than inserting a new one through `TransactionService::storePaymentInTransaction()` — so `payment_mode` had to be set the same direct way, gated to only fire when the switch is genuinely *into* Card.
- The existing partner-package handling here also differs from every other site: `TransferPaymentService::updatePartnerTransfer()` (not `transferToPartner()`) updates a payout row that was already created when the booking was first made. For a non-partner booking there is no equivalent pre-existing row to update — a Cash booking never had one — so the new dispatch correctly inserts a *fresh* `transferToOperator()` / `transferSellingCommission()` row, the first and only correct payout record for this booking, rather than needing an "update" variant of its own.
- The tenant's own connected account is available directly as `$tourBooking[0]->stripe_destination_id` here — this query's own `SELECT` already used the un-aliased column name, unlike Section 7.2b's `paymentNewBooking()`, which needed a new alias added.

1 new test, `tests/Feature/Payments/DeferredTransferChangePaymentTypeFlowTest.php` — a real Cash booking via `CreateABooking` (tour date 2027-06-15, WITH its real history/transaction rows this time, since the flow needs an existing row to switch from), switched to Card, passed on first try. Confirms the existing transaction row's `via` and `payment_mode` both update correctly despite the different write path, and that the operator transfer fires as the booking's first-ever transfer. 44 tests pass across the whole feature now, zero orphaned rows. Both "bonus" flows are now fully done, not just their creation sides.

7.2f Sixth end-to-end flow wired — Switch Package shipped, partial

Done for the non-partner case; the partner-package combinations are wired but reasoned through. Update (Phase 4, this revision): the refund branch is now wired too — see the standalone paragraph below this list. `SwitchTourPackageCardService` has two separate confirmation entrypoints, both reading the mode from the same shared creation helper's PaymentIntent metadata (Section 7.2d):

- `run()` — the main confirmation path. This one is substantially more complex than every other site wired so far: it branches across 4 combinations of old-package/new-package partner status (neither partner, old only, new only, both), *and* has a completely separate refund branch (triggered when the new package's total is less than the old, refunding the difference) that runs entirely different code from the payment branch. Scope for this revision: the payment (non-refund) branch only. Wired the same way as every other site — read `payment_mode` back off the already-retrieved PaymentIntent, thread it into `storePaymentInTransaction()`, then dispatch `transferSellingCommission()` (new package is a partner package) or `transferToOperator()` (new package is not) immediately after the pre-existing, untouched `transferToPartner()` call — same amount conventions the surrounding code already used for consistency.
- `completeSecureCardPayment()` — the 3D-Secure completion variant, used when the card requires additional authentication. Read in full: this entrypoint has no partner-package handling at all today (no `is_partner_package` / `is_partner_booking` branch anywhere in it), so only the simple non-partner `transferToOperator()` dispatch was needed. Also confirmed (Phase 4, this revision): this entrypoint has NO refund branch of any kind — it is purely a payment-completion method, nothing further to wire here. The destination account is read from `$info->stripe_destination_id`, which in this file (both entrypoints) is the new package's own `tour_packages.stripe_destination_id` column — not `tour_operators.stripe_destination_id` as in most other flows. Confirmed this is the file's own pre-existing, consistent convention (the untouched `transferToPartner()` calls at both entrypoints already read the same column), not a bug to fix: a partner package's row can point this column at the 3rd party's account, which a plain operator-id lookup would miss.

1 new test, `tests/Feature/Payments/DeferredTransferSwitchPackageFlowTest.php`, covering `completeSecureCardPayment()` 's non-partner case — passed on first try. Real `CreateABooking` -created old booking/package plus a purpose-built new package (real operator connected-account id, 2027-06-20 tour date, its own rate/slot rows) to switch onto, gateway mocked, asserting the new `transactions` row's `payment_mode`, the `createTransfer()` call's destination/amount, and the resulting `primary_operator` payout row. 45 tests pass across the whole feature now, zero orphaned rows (confirmed against `tenant_dixies`). Not tested, only reasoned through: `run()` 's payment-branch dispatch (all 4 partner-status combinations) — a dedicated test for `run()` would need to simulate its far larger surrounding calculation (discounts, remaining-amount math, old/new rate-group diffing) to exercise safely, which is disproportionate for this revision; flagged here rather than left silent.

Phase 4 update, this revision — `run()` 's refund branch (~120 lines, triggered when switching to a cheaper package) wired. Its own single `refundPayment()` call now reads `payment_mode` off `$trans` (the latest completed transaction being refunded, resolved the same way this branch already resolves it for other purposes) and routes through `buildRefundParams()`. The reversal dispatch needed real care: this file's OWN existing partner-package reversal mechanism (`reverseTransferToPlatformAccount()` / `transferToNewPartner()` / `transferBetweenConnectedAccounts()`, all pre-existing and confirmed mode-independent — they only ever move the OLD package's `partner_split` -typed transfer, which fires identically regardless of `payment_mode` per Decision 5, so none of them needed any change) resolves its target payout row by `package_id`, not `transaction_id` — because a single booking can switch through several packages over its lifetime, each with its own payout row. This is a genuinely different lookup key than every other site wired this session, all of which had a single obvious original transaction to key off. `reverseOperatorTransfer()` / `reverseSellingCommissionTransfer()` (Phase 0/1) both require a `transaction_id`

parameter — rather than changing either method's signature or inventing a parallel package-id-aware variant, resolved the correct `transaction_id` first via the SAME `package_id` lookup `reverseTransferToPlatformAccount()` already uses (filtering `transfers` by `tour_booking_id` + `package_id` + the right `transfer_type` + `Completed` status), then passed that resolved id into the existing, already-tested method unchanged. This reuses Phase 0/1's engine exactly as built, rather than special-casing Switch Package into it.

Not independently tested this revision — reasoned through carefully instead, same precedent as `run()` 's own payment branch above: driving a test through the refund branch would require simulating an actual package switch that produces a genuine price DECREASE (its own rate-group-diff/discount calculation, on top of the payment branch's already-flagged complexity), which is disproportionate given the payment branch's own dispatch was already left reasoned-through-only for the identical complexity reason. The new package-id-based lookup logic was instead verified by careful reading and cross-referencing against `reverseTransferToPlatformAccount()` 's own already-shipped, real query shape rather than guessed at.

7.2g Card Reader (Stripe Terminal) creation site — a different shape than every other flow shipped

Key discovery, changes the remaining-work count: Card Reader is not a separate confirmation flow. Tracing `backend-card-reader-info.js` 's `onCardReaderSuccess()` handler shows every Card Reader charge against an existing booking — Make Payment, Change People, Switch Package, Change Payment Type, Cart Payment, Custom Charge, Edit Discount, Addon Edit Charge, Tips — routes its resulting `payment_intent_id` into the exact same confirmation code Card Online already uses for that same context (`postTourBooking()` , `validateAndSubmitFormChangePeople()` , `validateAndSubmitFormSwitchPackage()` , `validateAndSubmitForm5()` for cart payment, `changeHistoryPaymentType()` , etc.) — confirmed the same for `backend-card-reader.js` on the New Booking page, whose `onSuccess()` sets the same hidden `payment_intent_id` field and calls the very same `postTourBooking(form)` the Card Online tab uses. So the only genuinely new work for the sites already wired (Sections 7.2a-f) is making the single shared creation endpoint, `App\Repositories\Pos\CardReaderRepository::getPaymentIntent()` , resolve and embed `payment_mode` correctly — everything downstream already reads it.

- Resolution added: a new private `resolvePaymentModeFromMetadata()` reads `metadata.booking_id` (sent by every existing-booking context already, per the trace above) and looks up that booking's tour date; if the booking has a non-null `cart_id` , it gathers every sibling booking sharing that cart and applies the same "any row ≥ 2027 forces the whole cart deferred" rule `paymentCartBookings()` already implements (Section 5.3), rather than resolving off only the one booking actually referenced in metadata. When no `booking_id` is present at all — the New Booking and POS pages' Card Reader tabs, which charge a booking that doesn't exist in the database yet — a second key, `metadata.tour_dates` (a comma-separated list, added by those pages' own JS — see Section 7.2t), is checked instead; only when NEITHER key is present does this fall back to `DESTINATION` , the same default every other creation site uses when it truly has nothing to resolve from.
- Real bug found and fixed by this same change, not a pre-existing pass: `getPaymentIntent()` unconditionally set `application_fee_amount` at the top of its payload, before any mode branching existed. Stripe rejects `application_fee_amount` on a PaymentIntent with no `transfer_data` — exactly the shape `buildConnectedPaymentData()` now produces in `DEFERRED_TRANSFER` mode. Before this revision the bug was inert (mode was always `DESTINATION` , so `transfer_data` was always present too); the moment deferred mode becomes reachable here, every 2027+ Card Reader charge across all nine contexts above would have hard-failed at the Stripe API call. Fixed by moving `application_fee_amount` out of the unconditional top-level payload and into the (dead, unreachable-today) `$connect=true` branch only, where it remains independently meaningful; the real (`$connect=false`) branch now relies solely on `buildConnectedPaymentData()` 's own return value, matching every `StripeUtils.php` creation site's existing convention.
- `payment_mode` is now set unconditionally into the outgoing metadata (even in the defensive, currently-unreachable "no metadata at all" branch) rather than only when other metadata happens to be present — so it can never be silently absent for a confirmation site that expects to read it back.

4 new tests, `tests/Feature/Payments/DeferredTransferCardReaderFlowTest.php` (all passed first try): a 2027-dated existing booking resolves deferred mode and the captured payload carries no `transfer_data` / `application_fee_amount` ; a 2026-dated one stays destination mode with both keys present as before; a combo-cart case proves the sibling-date rule fires even when the `booking_id` actually referenced in metadata is itself 2026-dated; and the no- `booking_id` case proves the New-Booking-page fallback stays destination mode, unchanged from today. 49 tests pass across the whole feature now, zero orphaned rows.

Gap closed (see Section 7.2t): the New Booking and POS pages' own Card Reader tabs (`backend-card-reader.js` , `tour-booking-pos.js`) had no tour-date plumbing at all for a not-yet-created booking — this was the last known open item across the whole project and is now fixed. Note `StripeUtils::createStripePaymentIntentPos()` , the POS page's own Card Online creation site, was ALREADY mode-aware by the time this was picked up (Section 7.2r, wired earlier than this note originally assumed).

7.2h Seventh flow wired — booking-level Tip, plus a shared helper covering FOUR more features shipped, partial

Key discovery, again changes the remaining-work count. Tracing the admin booking-info page's "Send Tip" modal end to end (JS through the actual PHP dispatcher) surfaced that the request to charge a tip is routed through a generic multi-feature dispatcher, `App\Repositories\TourBooking\StripeRepository::stripeCreatePaymentIntentForMakePayment()` — a thin `switch ($request->feature)` that fans out to ELEVEN separate `App\Services\Booking\StripeService::stripeCreatePaymentIntentFor*()` methods (one per feature: make_payment, change_people, edit_discount, custom_charge, tip, create_tip [tour-guide gratuity], switch_package, add_ons, addon_edit_charge/bulk_pax_addon_charge, change_payment_type, make_combo_payment). Reading each of the not-yet-confirmed-wired ones in full found:

- make_payment / change_people / switch_package / change_payment_type / make_combo_payment — already correct. Each of these calls straight into a creation site already fixed in an earlier section (7.2a-f) — `stripeCreatePaymentIntentForChangePeople()` / `ForSwitchPackage()` / `ForChangePaymentType()` all call the shared `StripeUtils::createStripePaymentIntentEditBooking()` helper (Section 7.2d), confirmed by reading each one's body, not assumed from the name.
- edit_discount / custom_charge / tip / add_ons / addon_edit_charge (+ bulk_pax_addon_charge) — all FIVE of these call a single, different shared helper this revision found not-yet-wired: `StripeUtils::createStripePaymentIntentCommon()` (Appendix A.1 site #3). Fixed once here — same leverage as Section 7.2d's discovery, but covering five features in one function instead of three. Resolves mode from `$info->date` (already loaded by every caller via the shared `getTourPackageInformation()` query), threads it into `buildConnectedPaymentData()` , adds it to the PaymentIntent metadata.

- create_tip — confirmed this IS the tour-guide gratuity system (`tour_operator_staff`, guide's own Stripe account, `Auth::id()` as the guide) explicitly excluded from this plan's scope from the very first research pass (Section 3) — correctly left untouched.

Confirmation wired for Tip only, this revision (`V3\TourBookingRepository::tipPayment()`) — chosen because it has no partner-package handling at all today (a tip's destination has always been the operator's own account via `createStripePaymentIntentCommon()`, regardless of `is_partner_package`, confirmed by reading the whole function), so only the simple non-partner `transferToOperator()` dispatch applies, matching the same "read mode back, persist it, dispatch if deferred" pattern as every other site. `TransactionService::storeTipInTransaction()` extended with the same trailing `$paymentMode` param convention as `storePaymentInTransaction()`.

Real, pre-existing bug found and fixed along the way (unrelated to payment_mode, but blocking this confirmation path entirely): `tipBookingDetails()`'s own `SELECT` never included `tbm.infants`, yet `tipPayment()`'s web (non-API) branch unconditionally reads `$tourBooking->infants` when building the PaymentIntent-refresh metadata — an undefined-property access that this app's error handler converts to a thrown exception, meaning this exact code path would fail for any tip payment made through the ordinary admin web UI (not just 2027+ ones). Caught only because the dedicated test exercises the real method rather than mocking around it; fixed by adding the missing column to the `SELECT`.

1 new test, `tests/Feature/Payments/DeferredTransferTipFlowTest.php`, passed after the `infants` fix above — a real 2027-dated booking, gateway mocked, asserting the new tip `transactions` row (`charge_type='tip'`) persists `payment_mode`, the transfer amount/destination, the resulting `primary_operator` payout row, and the `gratuities` ledger row. 50 tests pass across the whole feature now, zero orphaned rows.

Explicitly still open: the other four features sharing the now-fixed `createStripePaymentIntentCommon()` — Edit Discount, Custom Charge, Add-Ons, Addon Edit Charge — have a correct creation side but their own confirmation call sites are not yet wired (each needs tracing to its own dedicated confirmation method, none investigated yet this revision).

7.2i Eighth flow wired — Edit Discount shipped, partial

Traced the confirmation side in full — genuinely more tangled than any site wired so far, resolved by finding the one method that actually needs fixing. Edit Discount's web (non-API/non-refund) confirmation path is `editDiscountCardPayment()` → `handleWebPayment()` → `createEditDiscountPaymentIntent()` → `applyDiscount()`. Despite its name, `createEditDiscountPaymentIntent()` is where the real confirmation work happens for the common case: when a `payment_intent_id` is already present (the normal case — Card Online/Reader already created and confirmed the PaymentIntent via the Section 7.2h creation fix before this call), it refreshes that SAME intent via `BookingService::updateCardReaderIntent()` rather than creating a new one, then writes the `transactions` ledger row itself and calls the pre-existing, untouched `transferToPartner()` for a partner booking. `applyDiscount()`'s OWN separate `storePaymentInTransaction()` / `transferToPartner()` branch is skipped in this path (guarded on `transaction_id` already being set) — it only runs for the Cash/refund/API callers, which this revision does not touch.

- Wired the same way as every other site: read `payment_mode` back off the refreshed PaymentIntent, thread it into `storePaymentInTransaction()`, dispatch `transferSellingCommission()` (partner booking) or `transferToOperator()` (non-partner) right after the existing `transferToPartner()` call.
- partner_fees is 0 by design here, not a shortcut: the existing `transferToPartner()` call already hardcodes `'partner_fees' => 0` for this specific incremental charge (an edit-discount payment carries no separate partner-fee cut of its own — that was already settled at original booking time). Passing the same `0` into `transferSellingCommission()` means it correctly no-ops (no Stripe call, no ledger row) for a partner booking, exactly replicating `transferToPartner()`'s own existing behavior rather than inventing a new fee the current design doesn't have.
- Destination account: `tour_packages.stripe_destination_id` for the partner case (reusing the same lookup the untouched `transferToPartner()` call already made), `$tourBooking->tourPackage?->tourOperator?->stripe_destination_id` for the non-partner case — same convention established in Section 7.2a.
- Confirmed, not fixed (already flagged in Section 6's research): `applyDiscount()`'s OWN separate partner-transfer branch (used only by the Cash/API paths this revision doesn't touch) still reads the non-existent `$tourPackage->partner_destination_id` column, silently resolving to null — a pre-existing bug, confirmed to be a genuinely different code path from the one this section fixed, out of scope here as before.

1 new test, `tests/Feature/Payments/DeferredTransferEditDiscountFlowTest.php`, passed on first try — calls `createEditDiscountPaymentIntent()` directly (not through the full `editDiscountCardPayment()` → `applyDiscount()` → `BookingService::addEditDiscount()`/history-writing chain, whose discount-calculation logic is substantial and orthogonal to this change — same scoping precedent as Change People/Switch Package), asserting the persisted `payment_mode`, the `createTransfer()` call's destination, and the resulting `primary_operator` payout row. 51 tests pass across the whole feature now, zero orphaned rows.

> **PHASE 2/3 ADDENDUM, FOUND AND FIXED LATER IN PHASE 4 (SECTION 7.4)**
> This section's own fix only ever covered `V3\TourBookingRepository::createEditDiscountPaymentIntent()`. While wiring Edit Discount's REFUND side (Section 7.4), tracing led into the LEGACY (non-V3) `TourBookingRepository`'s own `createEditDiscountPaymentIntent()` — never wired at all, confirmed still-live traffic via API V1's controller. Fixed to mirror this section's own fix exactly (same non-partner-only shape, matching that file's own refund side). See Section 7.4 for the full writeup and the standing flag that legacy's OTHER creation methods have not been audited for the same gap.

7.2j Ninth flow wired — Custom Charge, plus a second creation shape found in the same method shipped

Confirmation: `V3\TourBookingRepository::customChargeCreatePaymentIntent()`, reached via `addCustomCharge()`. Has no partner-package handling anywhere (confirmed by reading it in full), so only the simple non-partner `transferToOperator()` dispatch applies. The destination account (`t_o.stripe_destination_id`, operator's own) is already passed in as `$data['destination']` by the caller — no new lookup needed.

- A second, genuinely separate creation shape lives in this same method: alongside the normal Card Online/Reader refresh path (an existing PaymentIntent, refreshed via `BookingService::updateCardReaderIntent()` — the mode is read back from it, never re-derived, same as every other refresh site), there is a raw-manual-card-entry branch (`!$paymentIntentId`) that builds a Stripe `PaymentMethod` directly from `cardnumber` / `month` / `year` / `cvc` fields and confirms immediately — no Stripe Elements, no prior PaymentIntent. This branch has its own independent `buildConnectedPaymentData()` call, so it needed its own mode resolution (from this booking's own tour date, since a fresh PaymentIntent is genuinely being created here) — fixed in the same

pass since it sits directly alongside the branch this section wired and shares the same surrounding function. Not independently tested this revision — no known live caller on the modern booking-info page collects raw card fields (the JS traced for this flow always uses Stripe Elements), so this branch may be legacy/dead in practice; flagged as unverified rather than assumed dead.

- `TransactionService::storeCustomChargeInTransaction()` extended with the same trailing `$paymentMode` param convention as every other `store*InTransaction()` method.

1 new test, `tests/Feature/Payments/DeferredTransferCustomChargeFlowTest.php`, passed on first try — calls `customChargeCreatePaymentIntent()` directly with an existing `payment_intent_id` (the reachable refresh-path case), asserting persisted `payment_mode`, the transfer destination, and the resulting `primary_operator` payout row. 52 tests pass across the whole feature now, zero orphaned rows.

7.2k Tenth flow wired — Add-Ons shipped

Confirmation: `TourBookingAddonRepository::chargeAddonFields()`, reached via the `upsert-addons` route — its PAYMENT section only; a separate REFUND section in the same method is untouched, deferred to Phase 4 like every other flow's refund logic. Has no partner-package handling anywhere (confirmed by reading it in full), so only the simple non-partner `transferToOperator()` dispatch applies. `resolveCardPaymentIntent()` (this method's own existing PaymentIntent-refresh helper — a real Stripe object for both its API-retrieve and web-refresh branches) already returns null for Cash/Pay Later, which never touch Stripe — handled with the nullsafe operator (`$paymentIntent?->metadata->payment_mode`) rather than an explicit null check, consistent with the rest of the method's own style. `TransactionService::storeAddonsPaymentInTransaction()` extended with the same trailing `$paymentMode` param convention as every other `store*InTransaction()` method.

1 new test, `tests/Feature/Payments/DeferredTransferAddOnsFlowTest.php`, passed on first try — a real add-on field row against a 2027-dated booking, gateway mocked, asserting the persisted `payment_mode` on the `addons`-charge-type `transactions` row, the transfer destination, and the resulting `primary_operator` payout row. Also re-ran the full 31-test pre-existing add-ons regression suite (`AddonBookingServiceTest`, `BookingServiceAddonPriceTest`, `CancelAddonSkipsDontRefundSettledFieldTest`, `DeletePaxAddonsCashGroupingTest`, `DeletePaxAddonsExplicitFieldIdsTest`, `PartialRefundCancelCapsTicketAtNetAvailableTest`, `SavePaxAddonsHistoryTest`) since several of these call the same `storeAddonsPaymentInTransaction()` signature that was just extended — all still pass unchanged (the new param is optional with a default). 53 tests pass across the 2027 feature now, zero orphaned rows (a pre-existing 24/13-row orphan count in `tour_booking_addon_fields` / `_transactions` dated 2026-09-01 predates this session by a week, confirmed via `created_at`, not caused by this work).

7.2l Eleventh flow wired — Addon Edit Charge, completing the Section 7.2h group shipped, partial

Confirmed genuinely separate from Add-Ons, as flagged: reached via a different route entirely (`update-booking-addon-submission`, not `upsert-addons`) → `TourBookingAddonRepository::updateBookingAddons()` (public) → `processAddonFinancialTransactions()` (private) — a substantially larger method than any confirmation site wired so far, matching its extensive prior bug history (see [[project_addons_feature]] in the memory file: six separate real-money bugs found and fixed here across earlier sessions, all around per-item PI-offset/cascade math for mixed add+remove edits). This section touches ONLY the CHARGE action's final aggregate ledger write — the exact same `TransactionService::storeAddonsPaymentInTransaction()` call already extended for Add-Ons (Section 7.2k) — not the REFUND action, and not any of the per-item `TourBookingAddonTransaction` draw/cascade logic upstream of it, which is untouched.

- Confirmed by reading the entire method: no partner-package handling exists anywhere in it (same as Add-Ons) — only the simple non-partner `transferToOperator()` dispatch applies.
- The PaymentIntent is only ever retrieved (never refreshed via `updateCardReaderIntent()`, unlike every other site) when `action === 'charge' && payment_method === 'card' && $intentId` is present — stays `null` otherwise (Cash/Invoiced/Billable/refund), handled with the same nullsafe-operator pattern as Add-Ons.
- Dispatch is placed inside the existing `if ($status !== 'open')` branch (a true Pay Later charge records no real transaction at all, matching the pre-existing "leave it open but visible" convention this method already uses) — mirrors exactly where the ledger write itself already lived.

1 new test, `tests/Feature/Payments/DeferredTransferAddonEditChargeFlowTest.php`, passed on first try — tested through the public `updateBookingAddons()` entrypoint (the touched code lives in a private method), a real addon field added to a 2027-dated booking with a `financial.action='charge'` payload, gateway mocked, asserting persisted `payment_mode`, the transfer destination, and the resulting `primary_operator` payout row. Re-ran the same 31-test pre-existing add-ons regression suite as Section 7.2k — still passes. 54 tests pass across the whole feature now, zero orphaned rows.

This completes all five features sharing the Section 7.2h `createStripePaymentIntentCommon()` creation helper — Edit Discount, Custom Charge, Tip, Add-Ons, and Addon Edit Charge all now have both a correct creation side and a wired confirmation side (Sections 7.2h-7.2l). Explicitly still not done: this method's own REFUND action (deferred to Phase 4, like every other flow's refunds).

7.2m Twelfth flow wired — Mobile API v2 new-booking checkout shipped

Appendix A.1 site #20. Creation: `App\Http\Controllers\API\V2\TourBookingController::createPaymentIntent()` — already supports a multi-item cart sharing one PaymentIntent, so mode is resolved via `PaymentModeResolver::resolve()` (the plural/combo-cart form, Section 5.3's "any row ≥ 2027" rule) over every item's own tour date collected in the same loop that already builds each item's totals, rather than `resolveForSingleDate()`. Confirmation: `packageBookingTour()` — a substantially larger method than most sites wired so far (creates the full booking, transact, and history rows during confirmation itself, closer in shape to the original web new-booking flow, Section 7.2a, than to a typical "refresh an existing PaymentIntent" site). Writes its `transactions` row via a raw `DB::table()->insertGetId()` call rather than `TransactionService::storePaymentInTransaction()` — `payment_mode` added directly to that insert.

- `bookingData()` (this controller's own booking-detail query, used throughout the confirmation flow) only selected `tp.stripe_destination_id` (the package-level, partner-aware column already used for the existing `transferToPartner()` call) — the same gap `paymentNewBooking()` had in Section 7.2b. Added `t_o.stripe_destination_id AS tour_operator_stripe_destination_id` alongside it for the non-partner case.

- Same partner-status convention as every other flow: `bookingData()`'s own `is_partner_booking` (set from the package's `is_partner_package` at booking-creation time, confirmed by reading the booking-creation loop above it) branches between `transferSellingCommission()` (reusing `$partnerFeesAmount`, already computed for the existing `transferToPartner()` call) and `transferToOperator()`.
- A combo cart shares one PaymentIntent/mode decision (resolved once at creation), but each booking in the loop still gets its own independent transfer — same convention as the web combo-cart flow (Section 7.2c).

1 new test, `tests/Feature/Payments/DeferredTransferMobileApiV2FlowTest.php`, passed after two small fixture-shape fixes (a truthy-string `"true"` success flag rather than a real boolean, and `retrievePayment()` genuinely called twice — once for this flow's own mode read-back, once by the pre-existing `HistoryService::getSavedCardFromPaymentIntent()` saved-card feature) — calls `packageBookingTour()` directly against a real 2027-dated package/rate/slot (reused from a throwaway `CreateABooking` setup, its own booking row discarded since this method creates a fresh one exactly like the real mobile checkout does), gateway mocked, asserting persisted `payment_mode`, the transfer destination, and the resulting `primary_operator` payout row. 55 tests pass across the whole feature now, zero orphaned rows (one orphan from an earlier failed iteration of this same test was found and cleaned up by hand before the final run, per CLAUDE.md's test-hygiene guidance).

Explicitly not independently tested: the creation side, `createPaymentIntent()` — it shares the exact same resolution shape already proven by the web combo-cart flow (Section 7.2c) and every other `buildConnectedPaymentData()` call site, and driving it through this same test would mean duplicating the entire booking-creation setup a second time for no additional confidence about the wiring itself; reasoned through carefully instead, consistent with the Custom Charge raw-manual-entry precedent (Section 7.2j).

> **REAL BUG FOUND AND FIXED RETROACTIVELY — SWITCH PACKAGE (SECTION 7.2F)**
> While tracing Payment Links' operator-vs-partner destination-column distinction below, re-examined every other flow's own destination lookup for the same shape and found one: `SwitchTourPackageCardService::completeSecureCardPayment()`'s own raw query selects only `tp.stripe_destination_id` (package-level, unaliased as plain `stripe_destination_id`) — unlike `run()`, whose `$info` comes from `TourPackageService::getTourPackageSlotInfo()`, which aliases the OPERATOR's own column (`t_o.stripe_destination_id`) as that same bare name. The `transferToOperator()` dispatch this plan added in Section 7.2f used `$info->stripe_destination_id`, silently reading the WRONG column. Confirmed against real `tenant_dixies` data: for an ordinary non-partner package, `tour_packages.stripe_destination_id` is NULL — meaning `transferToOperator()`'s own `$destinationId !== ''` guard would have been false, so `createTransfer()` would never fire, while a `transfers` row was still inserted marked `transfer_status='Completed'` — a false-positive ledger entry for a transfer that never happened. The charge itself would have still succeeded (deferred mode routes the full amount to the platform's own balance regardless), so the customer would not have noticed — only the operator, whose payout would silently never arrive. Root cause of the original mistake: the Section 7.2f test's own throwaway package fixture set `tour_packages.stripe_destination_id` to the operator's real account (to make assertions simple), which happened to mask exactly this bug — the test passed for the wrong reason. Fixed: added `t_o.stripe_destination_id AS tour_operator_stripe_destination_id` to `completeSecureCardPayment()`'s own SELECT and pointed the dispatch at it instead; also fixed the test itself to leave the new package's own `stripe_destination_id` unset (matching real non-partner packages), then verified by temporarily reverting the code fix alone and confirming the test now fails exactly as expected (`createTransfer()` never called) before restoring the fix and confirming it passes again — a genuine before/after regression proof, not just a passing test taken on faith. `run()`'s own dispatch was re-verified and confirmed correct as shipped (its `$info` already carries the right column).

7.2n Payment Links shipped

Appendix A.1 site #21. Creation: `PaymentLinkService::createPaymentLink()` — genuinely different shape from every other creation site: it builds a Stripe Checkout Session, not a raw PaymentIntent, so `buildConnectedPaymentData()`'s output is embedded inside the session's own `payment_intent_data` parameter (Checkout creates the PaymentIntent itself once the customer completes payment). Added `payment_mode` directly into `payment_intent_data.metadata` alongside it, the same way Checkout already receives every other `buildConnectedPaymentData()` field. A payment link can cover multiple bookings sharing one session (an "Order" spanning several bookings under one `group_id`) — mode is resolved from every group member's own tour date via `PaymentModeResolver::resolve()`, the same combo-cart rule as every other multi-booking site (Sections 5.3/7.2c/7.2m), not just the first booking's date.

Confirmation: `PaymentLinkService::bookingUpdate()` (public) → `handleStripePayment()` (private) — despite superficial resemblance to `paymentCartBookings()` (Section 7.2c), this is a genuinely separate code path with its own independent `storePaymentInTransaction()` / `transferToPartner()` call, confirmed by reading it in full. Same partner-status convention as every other flow: `BookingUtils::getBookingDetails()` (already used by this flow) already selects BOTH the operator's own account (`t_o.stripe_destination_id`, unaliased) and the package-level one (`tp.stripe_destination_id AS partnerDestinationId`) — no new column needed here, unlike several earlier sites. `transferSellingCommission()` uses the operator's own account (reusing the existing `$cardBooking['destination']` field the untouched `transferToPartner()` call sits right next to — not `partner_destination_id`, which is the 3rd party's account already used by that call); `transferToOperator()` uses the same field for the non-partner case.

1 new test, `tests/Feature/Payments/DeferredTransferPaymentLinkFlowTest.php`, passed on first try — calls `bookingUpdate()` directly (the touched code lives in a private method) against a real 2027-dated booking, gateway mocked, asserting persisted `payment_mode`, the transfer destination, and the resulting `primary_operator` payout row. 56 tests pass across the whole feature now, zero orphaned rows. Explicitly not independently tested: the creation side, `createPaymentLink()` — it renders a redirect response and calls `createCheckoutSession()` (a distinct gateway method none of this session's tests have modeled yet), and shares the same combo-cart resolution shape already proven elsewhere; reasoned through carefully instead.

7.2o Mobile API v2's 3D-Secure completion sibling shipped

Appendix A.4 site #13: `TourBookingController::packageBooking3dsPayment()` — `packageBookingTour()`'s (Section 7.2m) own 3DS-completion sibling, the exact same shape Switch Package (Section 7.2f) and Custom Charge (Section 7.2j) both already had. Its own local booking-detail query (not the controller's shared `bookingData()` helper fixed in Section 7.2m) had the identical "package-level-only `stripe_destination_id`" gap — fixed the same way, adding `t_o.stripe_destination_id AS tour_operator_stripe_destination_id`. Same partner-status convention as `packageBookingTour()`. Writes its own raw `DB::table()->insertGetId()` transaction row (not `TransactionService::storePaymentInTransaction()`), same as its sibling.

Calls `\Stripe\PaymentIntent::retrieve()` directly (bypassing `PaymentGatewayInterface` — a pre-existing CLAUDE.md convention violation, not introduced by this change, flagged not fixed). 1 new test, `tests/Feature/Payments/DeferredTransferMobileApiV2ThreeDsFlowTest.php`, passed after one fix (a second,

separate `retrievePayment()` call inside the pre-existing `HistoryService::getSavedCardFromPaymentIntent()` saved-card feature, going through the mocked `PaymentGatewayInterface` this time, needed its own expectation) — installs a fake Stripe HTTP client via the SDK's own `ApiRequestor::setHttpClient()` seam (Section 7.2's own precedent) for the raw SDK call, real 2027-dated booking, asserting persisted `payment_mode`, the transfer destination, the resulting `primary_operator` payout row, and that the booking's status actually flips to 'Booked'. 57 tests pass across the whole feature now, zero orphaned rows.

7.2p Combo new-booking Card shipped, partial

Appendix A.4 site #5, plus one bonus fix found alongside it. This is a genuinely separate combo-cart-of-NEW-bookings feature from the Make-Payment-on-EXISTING-bookings combo cart already wired in Section 7.2c:

- `createComboBookingCard()` (site #5, confirmation) — the "client already created the PaymentIntent" sibling of `createBookingCard()` (Section 7.2a), used for Card Reader and web-manual-card combo checkouts. Already has WORKING partner-package handling (`transferToPartner()`), so both `transferSellingCommission()` / `transferToOperator()` apply, wired the standard way — read `payment_mode` back off the retrieved PaymentIntent, thread into `storePaymentInTransaction()`, dispatch per cart item using each item's own `destination` / `partner_destination_id` / `partnerFees` (already present on every `$cardBookingData` entry via `CartService::prepareCardBookingData()` — no new column lookup needed here).
- `createPaymentIntentForCart()` (this feature's OWN separate creation+confirmation method, used for iOS-manual-card entry — NOT on Appendix A.4's list at all, precisely because of the bug below) — a raw-manual-card-entry method with zero partner-package handling of any kind today (confirmed by reading it in full — no `transferToPartner()` call anywhere, which is exactly why it was never catalogued in that `transferToPartner()`-call-site list), matching the same pre-existing, already-documented gap this plan's early research found ("a same-tenant cross-operator combo cart paid via the raw manual card entry path... skips the 3rd-party payout entirely"). Resolved mode via `PaymentModeResolver::resolve()` across every cart item's own tour date (each already carries one) and threaded it into `buildConnectedPaymentData()`/the PaymentIntent metadata — but the confirmation-side dispatch deliberately replicates today's exact gap rather than inventing new partner logic: `transferToOperator()` only fires when an item genuinely isn't a partner package; a partner item gets nothing here, exactly like it gets no `transferToPartner()` call today either. Adding a lone `transferSellingCommission()` with no matching 3rd-party transfer would have been strictly worse than the pre-existing gap, not a fix for it.

1 new test, `tests/Feature/Payments/DeferredTransferComboBookingCardFlowTest.php`, passed on first try (one small fixture fix: `CartService::calculateServiceCommissionForCard()` needed a top-level `$data['total']` key) — covers `createComboBookingCard()`'s non-partner case against a real 2027-dated booking, gateway mocked, asserting persisted `payment_mode`, the transfer destination, and the resulting `primary_operator` payout row. 58 tests pass across the whole feature now, zero orphaned rows. Explicitly not independently tested: `createPaymentIntentForCart()` — it calls Stripe's `PaymentMethod::create()` directly with raw card-number fields (no Stripe Elements at all), the same "raw manual card entry" shape already left untested for Custom Charge (Section 7.2j); reasoned through carefully instead.

Appendix A.4's genuine 19-site list is down to ONE remaining Card-relevant site: `applyDiscount()`'s own Card/mobile-API branch (site #10) — still untouched, still carrying the pre-existing `partner_destination_id` bug noted since this plan's earliest research. Not attempted this revision.

7.2q Site #10 investigated fully — deliberately left unwired, with reasons investigated, not wired

Traced every caller of `applyDiscount()` to determine exactly when its own Card write-path (site #10, containing the known `partner_destination_id` bug) is actually reached. Result: it is NOT reachable from either of Edit Discount's two normal entrypoints — `handleWebPayment()` and `handleApiPayment()` BOTH pre-populate `$discountHistoryArray['transaction_id']` before calling `applyDiscount()`, which trips the very next line's own guard (`if (!(isset(...['transaction_id']) && ...['transaction_id']))`) and skips this entire block — those two paths are already fully covered by `createEditDiscountPaymentIntent()` (Section 7.2i).

The ONLY way this block is actually reached is a THIRD, separate confirmation entrypoint inside `editDiscount()` itself: an `is_click==1` "second API call confirms the discount" branch, which calls `applyDiscount()` directly with a `$discountHistoryArray` that was built WITHOUT ever setting `transaction_id` *or* `transaction_number` at all. This means:

- For a non-partner booking taking this path with Card payment: `storePaymentInTransaction()` still gets called and succeeds, but with an EMPTY `transaction_number` — a separate, real, pre-existing data-quality gap (any real charge confirmed this way leaves no PaymentIntent id on its ledger row at all), independent of the partner-destination bug.
- For a partner booking taking this path with Card payment: the code additionally tries `app(PaymentGatewayInterface::class)->retrievePayment($transactionNumber, ...)` with that same empty string — likely a hard Stripe API error, on top of the already-known wrong-column bug.

Deliberately left unwired, not out of caution alone but because it cannot be done safely: this plan's entire design principle for every other site is "resolve mode once at PaymentIntent-creation time, lock it onto the PaymentIntent's own metadata, and NEVER re-derive it later from the booking's current tour date" (Section 5.2) — precisely to avoid a reschedule silently changing a booking's settlement mode after the fact. This specific edge case has no reliable PaymentIntent id to read that locked-in metadata FROM at all (the empty `transaction_number` above) — the only way to resolve a mode here would be to re-derive it from the current tour date, directly violating that principle for the one site where it would matter least reliably. Rather than compromise the principle for a narrow, already-broken, rarely-exercised mobile-API edge case, this site is flagged and left exactly as-is — a legitimate, reasoned "not safely wirable" conclusion, not an oversight.

7.2r POS (in-person point of sale) shipped, partial

Appendix A.1 site #1: `StripeRepository::stripeCreateStripePaymentIntentForPos()` → `StripeUtils::createStripePaymentIntentPos()` — flagged back in Section 7.2g as still needing this same treatment once POS was picked up. POS can sell a cart of multiple packages sharing one PaymentIntent, so mode is resolved via `PaymentModeResolver::resolve()` across every package's own tour date (collected in the same loop that already builds each package's totals), the same combo-cart rule as every other multi-item creation site (Section 5.3).

Confirmation: `PosRepository::cardPaymentForPos()` — creates the booking rows and confirms payment in one call (`createBookings()`), structurally closer to the original web new-booking flow (Section 7.2a) than a typical PaymentIntent-refresh site. Has no partner-package handling anywhere today (confirmed

by reading it in full), so only the simple non-partner `transferToOperator()` dispatch applies — reusing each cart item's own already-present `destination` / `tour_package_id` fields (built by `createBookings()` alongside the booking itself), no new column lookups needed.

1 new test, `tests/Feature/Payments/DeferredTransferPosFlowTest.php`, passed on first try — calls `cardPaymentForPos()` directly against a real 2027-dated booking, gateway mocked, asserting persisted `payment_mode`, the transfer destination, and the resulting `primary_operator` payout row. 59 tests pass across the whole feature now, zero orphaned rows. Explicitly not independently tested: the creation side, `stripeCreateStripePaymentIntentForPos()` — shares the exact same combo-cart resolution shape already proven by every other multi-item site, and driving it end-to-end would mean duplicating the POS cart/customer setup a second time for no additional confidence about the wiring itself; reasoned through carefully instead.

This closes out Appendix A.1's site #1, the last explicitly-tracked creation-side gap from Section 7.2g's own follow-up list. The New Booking/POS pages' Card Reader tour-date plumbing gap (also flagged in Section 7.2g) is closed too, as of Section 7.2t.

7.2s Legacy creation-method audit — a significant new finding, larger than expected shipped

Started the standalone follow-up flagged repeatedly in Phase 4's own writeup: auditing legacy (API V1)'s OTHER creation methods for the same "never wired" gap discovered for Edit Discount (Section 7.2i's addendum). Confirmed via direct grep of `buildConnectedPaymentData(` call sites in legacy `TourBookingRepository.php` that FOUR direct 2-arg (no-mode) calls exist. Dispatched a dedicated sub-agent investigation (opus model) to precisely map the other 11 API-V1-reachable creation/confirmation methods, each with exact file:line citations and a verdict. The result: the gap is substantially larger than "just legacy" — several SHARED utility helpers used by BOTH legacy and V3 are ALSO unwired, meaning Phase 2/3's "essentially complete" claim has its own real holes for specific flows (Combo Cart's own upstream creation helper, the mobile-app Card Reader/Tap-to-Pay/Tip helpers, and two Custom Charge confirmation helpers). None of these are money-at-risk bugs — the OPPOSITE, actually: these specific flows simply stay on today's destination-charge behavior silently for a 2027+ booking, an inconsistency rather than a fire, but real work nonetheless.

Fixed this revision (all four following V3's own already-shipped equivalent as the template, each mirrored exactly — read fully first, resolve mode from the PaymentIntent's own locked-in metadata, thread into `storePaymentInTransaction()`, dispatch the non-partner-only `transferToOperator()`, since none of these four legacy methods have any partner-package handling to begin with):

- `createPaymentIntentForCart()` — the raw-manual-card-entry combo creation site (mirrors V3's own Section 7.2p fix exactly, including replicating that method's own documented zero-partner-handling gap).
- `createPaymentIntent()` — reached from `createBooking()`'s basic/non-API confirmation path.
- `createBookingCard()` — reached from `createBooking()`'s API path AND from `cardReaderAfterCapture()` (fixing this one method fixes that caller for free, since it's a pure delegate).
- `paymentNewBooking()` — the "Make Payment" (balance-due) confirmation for an existing Pay-Later booking. Upstream creation (`StripeUtils::createStripePaymentIntentMakePayment()`, SHARED with V3) was already mode-aware — legacy's confirmation side just never read the metadata back.

Real bug found and fixed along the way, the same "wrong destination column" class already seen twice earlier this project (Switch Package, Mobile API v2's `packageBooking3dsPayment()`): `paymentNewBooking()`'s own raw SQL query selects `tp.stripe_destination_id` (the tour PACKAGE's own partner-override column, empty for a plain non-partner package) but never selects the tour OPERATOR's own account at all. Using that column for the `transferToOperator()` dispatch would have sent every deferred-mode Make Payment transfer nowhere. Fixed by resolving the operator's own `stripe_destination_id` via a small dedicated lookup (`tour_operator_id` is already selected by this same query for an unrelated purpose) rather than touching the pre-existing SELECT list.

Test-authoring lesson, caught via a genuine failing assertion, not a production bug: an early attempt at `paymentNewBooking()`'s destination-mode regression test used a payment-intent-id test fixture prefix long enough that, combined with `uniqid()`, it exceeded the `transactions.transaction_number` column's 50-character limit — MySQL silently truncated the stored value on insert (non-strict mode), so the test's own later lookup by the untruncated id found nothing. Diagnosed via temporary debug logging reading the actual inserted row back, confirmed the truncation directly, then simply shortened the test's own id prefix — no production code involved.

8 new tests across two files: `tests/Feature/Payments/DeferredTransferNewBookingLegacyFlowTest.php` (2 tests, `createBookingCard()`) and `tests/Feature/Payments/DeferredTransferMakePaymentLegacyFlowTest.php` (2 tests, `paymentNewBooking()`, using the SAME shared upstream creation call V3's own passing test already uses) — all four passed after the fixes above. `createPaymentIntentForCart()`'s and `createPaymentIntent()`'s own deferred-mode logic reasoned through and covered indirectly by the `createBookingCard()` tests' shared code paths rather than independently tested a second time, matching this project's established "test the specific thing changed, not every structurally-identical branch" scoping discipline. Proactively grepped the whole `tests/` tree beforehand for every touched method name — zero pre-existing references to any of the four, no mocking-hazard risk. Full curated regression suite: 119 tests passing (up from 111 before this revision), zero orphaned rows.

Latest revision: `createComboBookingCard()` (legacy) fixed TOGETHER WITH its own upstream `StripeUtils::createStripePaymentIntentCart()` helper (SHARED with V3, was also 2-arg/mode-blind) — the combo new-booking card confirmation, reached from `createComboBooking()`'s card path AND (for free, since it's a pure delegate) `cardReaderAfterCaptureComboTour()`. Confirmed legacy has zero partner-package handling in this method (only the non-partner `transferToOperator()` dispatch applies, mirroring V3's own equivalent gap in its sibling `createPaymentIntentForCart()`). The shared upstream helper resolves mode across every cart item's own tour date (`CartService::prepareCartBookingData()` already carries a `date` key per item, confirmed via the same shared service both API versions use) — the same combo-cart rule as every other multi-booking creation site. 2 new tests, `tests/Feature/Payments/DeferredTransferComboBookingCardLegacyFlowTest.php`, both passed first try, mirroring V3's own already-shipped `DeferredTransferComboBookingCardFlowTest.php`. The upstream `createStripePaymentIntentCart()` fix itself reasoned through rather than independently tested — its own session-cart-based creation entrypoint (`StripeRepository::stripeCreatePaymentIntentForCart()`) is complex to set up in isolation, and the identical resolve+4-arg-buildConnectedPaymentData+metadata-key pattern is already thoroughly proven by the resolver/gateway unit tests plus every other creation-side fix this whole project. 121 tests passing (up from 119), zero orphaned rows, zero mocking-hazard risk (proactively confirmed zero pre-existing tests reference either touched method).

Latest revision: `paymentCartBookings()` (legacy) fixed — the combo-cart's own "Make Payment" confirmation, same shape as `paymentNewBooking()`: upstream creation (`StripeUtils::createStripePaymentIntentMakeComboPayment()`, SHARED with V3) already resolves mode across every cart item's own tour date (Section 5.3's combo-cart rule) — legacy's confirmation side just never read it back. Confirmed legacy has zero partner-package handling here (unlike V3's own version, which has genuine `transferToPartner()` / `transferSellingCommission()` dispatch for partner bookings in a cart) — only the non-partner `transferToOperator()` applies, dispatched independently per cart item (a combo cart can span several operators). Verified the destination-account source, `BookingUtils::getBookingDetails()`, correctly selects the tour OPERATOR's own `stripe_destination_id` (not the package's partner-override column) before trusting it — avoiding a repeat of the exact bug just found in `paymentNewBooking()`. 1 new test, `tests/Feature/Payments/DeferredTransferComboCartLegacyFlowTest.php`, mirroring V3's own already-shipped `DeferredTransferComboCartFlowTest.php` exactly (a cart bundling a 2026-dated AND a 2027-dated booking under one PaymentIntent, proving the whole cart resolves to deferred mode and each item gets its own independent transfer) — passed first try. 122 tests passing (up from 121), zero orphaned rows, zero mocking-hazard risk (the one pre-existing test referencing this method uses V3's repository class, unaffected).

Latest revision: `customChargeCreatePaymentIntent()` (legacy) fixed — mirrors V3's own already-shipped equivalent exactly (Section 7.2j), including its own SECOND, independent creation shape (a raw manual card-number-entry branch sitting right alongside the normal "refresh an already-created PaymentIntent" branch this is mostly reached through) — fixed both since they sit in the same method, though the raw-entry branch again could not be independently tested (no known live caller collects raw card fields on the modern page, same as V3's own note). Confirmed legacy has zero partner-package handling here either (only the non-partner `transferToOperator()` dispatch applies). While writing the test, learned that the shared `BookingService::updateCardReaderIntent()` helper calls both `updatePayment()` *and* `retrievePayment()` internally but actually returns `updatePayment()`'s result (`retrievePayment()`'s is used only for an internal `updateConnectedPayment()` check) — an easy mock-setup mistake caught immediately by an `Undefined property: stdClass::$status` failure, fixed by returning the fully-populated fake intent from the `updatePayment()` stub instead (matching V3's own already-shipped test's convention, which had it right the first time). 2 new tests, `tests/Feature/Payments/DeferredTransferCustomChargeLegacyFlowTest.php`, both passed after that correction. Proactively grepped the whole `tests/` tree beforehand — the one pre-existing reference uses V3's repository class, zero mocking-hazard risk. 124 tests passing (up from 122), zero orphaned rows.

Latest revision: `StripeUtils::getStripeAttributesForAppCardReader()` fixed TOGETHER WITH both files' own `stripeCreatePaymentIntent()` — the mobile app's ("iOS Card Reader") own PaymentIntent-creation entrypoint, distinct from the web booking-info page's Card Reader flow (`CardReaderRepository`, Section 7.2g) and CREATION-ONLY (returns just `client_secret` / `id`, no `transactions` row, no transfer dispatch — its resulting `payment_intent_id` is picked up by whichever confirmation method matches the request's own "section," already wired elsewhere in this project). Resolved mode across every cart item's own date for the combo-cart case (Section 5.3) or the single booking's own date otherwise, threaded into both the shared helper's `buildConnectedPaymentData()` call and the raw inline non-card_reader branch, with metadata carried via a direct post-combine `$paymentIntentParams['metadata']['payment_mode']` set (the same convention already used by `createPaymentIntentForCart()`, since `CartService::stripeMetadata()` has no `payment_mode` passthrough of its own).

A FOURTH instance of the "wrong destination column" bug class was found and fixed here, this time caught directly by my own test rather than by inspection — both the shared `getStripeAttributesForAppCardReader()` helper and each file's own non-cart, non-card_reader branch read `tour_packages.stripe_destination_id` (the partner-override column) instead of the tour operator's own account, despite this method having zero partner-package handling anywhere. Since every real package in `tenant_dixies` has this column empty (confirmed: 0 of 2), and `buildConnectedPaymentData()` requires a non-null string destination, this would throw a `TypeError` in production today for any non-partner package's mobile Card Reader charge — a real, currently-live bug, unrelated to tour date/payment mode, surfaced only because writing the test required a real destination id to assert against. Fixed by changing the shared helper's signature to accept an explicit `string $destinationId` instead of implicitly deriving one from a passed-in `$tourPackage` object, and updating all four call sites (2 legacy, 2 V3) to pass the tour operator's own `stripe_destination_id` — already loaded in both methods for the statement descriptor, no new query needed. This also fixed a related inconsistency: V3's own non-card_reader branch already correctly used the operator's account for its cart case but not its non-cart case; now both cases agree.

4 new tests across two files, `tests/Feature/Payments/DeferredTransferAppCardReaderIntentFlowTest.php` (V3) and `DeferredTransferAppCardReaderIntentLegacyFlowTest.php` (legacy) — each covers the card_reader=1 branch (deferred mode) and the non-card_reader branch (destination-mode regression) for a single, non-cart booking; the combo-cart branch reasoned through rather than independently tested (requires simulating `BookingService::calculateTotalCartPayment()` across multiple real bookings, and the underlying combo-cart resolution rule is already proven elsewhere). Proactively grepped the whole `tests/` tree beforehand for every touched method/function name — zero pre-existing references, no mocking-hazard risk. 128 tests passing (up from 124), zero orphaned rows.

Latest revision: `StripeUtils::getStripeAttributesForAppTapToPay()` and `getStripeAttributesForCreateTip()` investigated fully and CONFIRMED OUT OF SCOPE — not a fix needed, matching the precedent already set by `applyDiscount()`'s Card branch (Section 7.2q). Both are the tour-guide gratuity ("Tip Center") system: `GratuityRepository::stripeCreateTipPaymentIntent()` ("Tap to Pay") and `StripeService::stripeCreatePaymentIntentForCreateTipTourGuide()` ("Guide-dashboard Create Tip") both charge the tour operator's own Stripe account directly (a genuine destination charge, same account as every other flow), but their confirmation counterparts (`GratuityRepository::afterTipPayment()` / `makeTipPayment()`) insert a `gratuities` row with `booking_id = NULL`, `gratuity_type = 'direct'` — no booking, no tour date, no `transactions` row, no `TransactionService::store*InTransaction()` call, and no `transferToOperator()` / `transferToPartner()` dispatch of any kind. This isn't a case of "hasn't been wired yet" — there is structurally nothing to resolve a payment mode FROM (no tour date exists for a walk-up/direct tip with no associated booking), matching this plan's own Section 3 exclusion of the tour-guide gratuity payout system from its very first research pass. Appendix A.1 rows 5/6 ("Tour-guide 'Tap to Pay' tip", "Guide-dashboard 'Create Tip'") are updated to reflect this is now confirmed, not merely assumed, out of scope. No code changed, no new tests — this is an investigation-only finding, same category as Section 7.2q.

Latest revision: `CustomChargeBookingService::tourBookingUpdateCustomCharge()` and `cardPaymentCustomChargeFromApp()` fixed — both SHARED between legacy and V3 (called from both repository files' `changePeopleBooking()` / `addCustomCharge()`), plus `tourBookingUpdateCustomCharge()` is ALSO called directly from the live `StripePaymentIntentSucceededForMakePayment` webhook handler (a Pay-Later booking's custom charge settling once the deferred balance is finally paid) — fixing the shared method once retroactively covers that webhook call for free, the same "fix a shared helper once" leverage already proven several times this project. Both methods receive or retrieve an already-created PaymentIntent directly (no separate creation-side fix needed), so the mode is simply read back from its own locked-in `metadata.payment_mode`, threaded into `TransactionService::storeCustomChargeInTransaction()`, and used to

gate a non-partner-only `transferToOperator()` dispatch (confirmed zero partner-package handling anywhere in this file via a full read). Neither method had the tour operator's own destination id in scope, so a small dedicated `tour_bookings_master` → `tour_packages` → `tour_operators` join resolves it fresh, matching the established "small dedicated lookup, don't touch the pre-existing query" discipline from every prior wrong-column fix.

4 new tests, `tests/Feature/Payments/DeferredTransferCustomChargeUpdateFlowTest.php` (2 per method: deferred + destination-mode regression) — all passed first try. Proactively grepped the whole `tests/` tree beforehand for both method names — zero pre-existing references, no mocking-hazard risk. Re-ran the pre-existing custom-charge/addon-refund regression suites that also touch this file (`RestoreBookingAddonRefundTest`, `OtaCustomChargeAfterFullRefundTest`, `ResolveRefundDataNoteTest`) — all still pass. 132 tests passing (up from 128), zero orphaned rows.

Latest revision: `tipPayment()` (legacy) fixed — the LAST item on this audit's priority list. Genuinely the hardest site all session: legacy tour-guide tips insert directly into `tour_booking_tips` (raw insert), with no `transactions` row, no `payment_mode` column, and no fees column at all — structurally different from V3's `gratuities` +real- `transactions` -row shape. Put the design decision to the user rather than inventing one unilaterally (per this plan's own standing practice for genuine business/architecture calls) after first establishing the actual constraints: `transfers.transaction_id` carries no real foreign key anywhere in this codebase (confirmed via its own migration — a plain indexed `unsignedBigInteger`, and `reverseOperatorTransfer()`'s own lookup only ever queries `transfers` by `tour_booking_id` + `transaction_id` together, never joining out to `transactions`), and no refund/reversal path exists anywhere for a tour-guide tip in EITHER legacy or V3 (grepped the whole codebase for `tour_booking_tips` references — only reporting/export code and an admin truncate-tool reference it, nothing that would ever need to read a stored mode back later). User chose the minimal fix, no migration: resolve `payment_mode` from the already-refreshed PaymentIntent's own metadata at the moment of the call (skipped entirely on a webhook-flagged replay, mirroring the pre-existing history-write gate, since `$paymentIntent` is only ever resolved in that case anyway and the transfer must not double-fire), and use the newly-inserted `tour_booking_tips` row's own id (via `insertGetId()` instead of `insert()`) as `transferToOperator()`'s correlation key.

Confirmed zero partner-package handling anywhere in this method. A real money-correctness wrinkle, reasoned through carefully: legacy's tip confirmation request carries only a single grand-total-shaped `amount` field, with no separate fee field (unlike V3's `tip_amount` / `tip_fees` split) — yet the underlying PaymentIntent (created via the already-fixed shared `createStripePaymentIntentCommon()`) DOES bake in a platform fee via `application_fee_amount` in destination mode. Rather than assume the fee is always zero, the fix recovers it as `$paymentIntent->amount/100 - $request->amount` — the PaymentIntent's own always-reliable charged total minus the recorded tip amount — so `transferToOperator()`'s `platform_fees` argument correctly reflects whatever fee actually existed (computing to exactly 0 if none did, a safe fallback either way) rather than either always assuming zero or guessing at a percentage.

A FIFTH instance of the "wrong destination column" bug class was found here too, this time NOT reused: legacy's own `tipBookingDetails()` selects `tp.stripe_destination_id` (the package's partner-override column) where V3's otherwise near-identical version correctly selects `t_o.stripe_destination_id` — confirmed via a side-by-side read of both methods. Not fixed directly (the pre-existing query has exactly one caller, this same method, and nothing else reads that field) — instead, a fresh dedicated `tour_operators` lookup keyed off the already-selected `tour_operator_id` resolves the correct destination for this fix, sidestepping the bug entirely rather than trusting or silently propagating it.

2 new tests, `tests/Feature/Payments/DeferredTransferTipLegacyFlowTest.php` (deferred mode, asserting the transfer amount is correctly net of the recovered platform fee; destination-mode regression) — the first attempt asserted the wrong expected transfer amount (a test-authoring mistake, not a production bug: forgot the transfer amount is the NET payout, not the gross charge) and separately hit a float-vs-int strict-equality mismatch (`assertSame` vs `assertEquals`) — both caught by genuine failing assertions and corrected before finalizing. Proactively grepped the whole `tests/` tree beforehand — the one pre-existing reference targets V3's repository class, unaffected. 134 tests passing (up from 132), zero orphaned rows.

Latest revision: the final broad sweep — genuinely completed, not just declared done. Enumerated EVERY `buildConnectedPaymentData(` call site across the whole `app/` tree (21 total): every one already mode-aware except the two already-confirmed-out-of-scope tour-guide-gratuity helpers. Enumerated EVERY `createStripePaymentIntent*` / `getStripeAttributesFor*` / `getStripeMetadataFor*` / `createStripeIntent` function DEFINITION across the whole codebase (15 total, all in `StripeUtils.php` plus one thin controller delegate) — all resolve `payment_mode` internally or are confirmed pure delegates to already-fixed helpers. Also grepped for any INLINE `transfer_data` / `application_fee_amount` construction that might bypass `buildConnectedPaymentData()` entirely (a sneakier way to miss the gate) — found one genuine outlier (`InvoiceRepository::createStripeInvoice()`, ACH/bank-transfer Stripe Invoices, a different Stripe product entirely from PaymentIntents, matching this plan's own Section 1 exclusion of "cash/check/other offline payment methods" — not fixed, correctly out of scope) and traced a second false lead to ground (`updateCardReaderIntent()`'s own `application_fee_amount` key lives inside Stripe's `metadata` dictionary, not as a real API parameter — confirmed harmless, no functional effect regardless of mode).

ONE genuinely new, previously-uncovered site found and fixed: `CustomerController::createTipPaymentLink()` — a public, customer-facing Stripe Checkout Session for tipping a tour guide, distinct from every other tip flow investigated this project. Unlike the Tip Center's always-`booking_id`-NULL "direct" gratuity system (confirmed out of scope earlier in this section), this one can OPTIONALLY be tied to a real booking — and its confirmation, `PaymentLinkService::saveTipPayment()`, already calls `TransactionService::storeTipInTransaction()` (a real `transactions` row) whenever a booking is present, so unlike legacy's `tipPayment()` there was no "no transactions row" design problem here at all. Fixed both sides: creation resolves mode from the booking's own tour date when present (else falls back to destination, matching every "no booking" convention), locks it onto `payment_intent_data.metadata` (the same Checkout Session convention already established in Section 7.2n); confirmation reads it back and dispatches the same non-partner-only `transferToOperator()` pattern used everywhere else.

2 new tests, `tests/Feature/Payments/DeferredTransferGuideTipPaymentLinkFlowTest.php` — both passed FIRST try, covering the CREATION side only (mocking `buildConnectedPaymentData()` / `createCheckoutSession()` through the gateway interface, fully testable). The confirmation side, `saveTipPayment()`, calls the raw Stripe SDK directly (`Session::retrieve()` / `PaymentIntent::retrieve()`, a pre-existing architectural gap unrelated to this fix, not touched) rather than through `PaymentGatewayInterface`, so it can't be mocked the same way as every other confirmation site — reasoned through instead, since the underlying `storeTipInTransaction()` / `transferToOperator()` dispatch pattern is already thoroughly proven elsewhere. 136 tests passing (up from 134), zero orphaned rows, zero mocking-hazard risk (proactively grepped first, zero pre-existing references to either touched method).

The legacy creation-method audit is now COMPLETE — every item on its priority list is done, and the final broad sweep genuinely found and closed one real remaining gap. Nothing further is currently tracked as open on this audit; a codebase this large may still hold something unexamined, but the systematic sweep methodology (enumerate every call site + every function definition + every inline construction) leaves nothing else identifiable by that method.

7.2t New Booking/POS Card Reader tour-date plumbing gap — the LAST known open item, now closed shipped

This gap was flagged repeatedly throughout the whole project (Sections 7.2g, 7.7, Appendix A.1 rows 1/18) and left open every time: `CardReaderRepository::getPaymentIntent()` resolves mode from `metadata.booking_id`, but the New Booking and POS pages' own Card Reader tabs charge a booking that doesn't exist in the database yet — there was no `booking_id` to look a tour date up from at all, so every Card Reader charge from those two specific tabs silently stayed on `DESTINATION` mode regardless of the tour date actually selected.

The fix: the tour date(s) the user already selected before opening the Card Reader tab ARE known client-side even though the booking row isn't created yet — both pages' own JS now sends them through as a new comma-separated `metadata.tour_dates` string, checked by `resolvePaymentModeFromMetadata()` as a second key whenever `booking_id` is absent (falling back to `DESTINATION` only when NEITHER key is present, preserving today's behavior for any other, genuinely-unknown context). One date per entry naturally handles both shapes with the same key: a single tour date for a normal booking, or one date per package for a combo cart/multi-package POS sale — `PaymentModeResolver::resolve()`'s own existing "any row ≥ 2027 forces the whole set deferred" rule (Section 5.3) applies uniformly either way, no new resolver logic needed.

- New Booking (`backend-card-reader.js`'s `capturePaymenWithStripeCardReader()`): reuses the page's own already-rendered `#hid-tour_slot_date` hidden field (the single-booking case) or maps the page's own `cartPackages` JS global's `.date` field per entry (the combo-cart case) — both already existed on the page for other purposes, no new backend endpoint or template change needed to expose the date.
- POS (`tour-booking-pos.js`'s `capturePaymenWithStripeCardReader()`): maps `this.selectedTourPackages`' own `.slotDate` field per selected package — the same field the method's own pre-existing `note`-building code right next to it was already reading for a different purpose.
- Backend (`CardReaderRepository::resolvePaymentModeFromMetadata()`): when `booking_id` is empty, splits `metadata.tour_dates` on commas (trimming and filtering blanks, so an absent/empty key safely degrades to an empty array rather than a stray blank-string entry) and passes it straight to `PaymentModeResolver::resolve()` — the same plural entrypoint every other combo-cart site already uses, not a new resolution path.

3 new tests added to the existing `tests/Feature/Payments/DeferredTransferCardReaderFlowTest.php` (a single 2027 date with no `booking_id`; a comma-separated combo-cart-shaped list with one 2027 entry among 2026 ones; an all-2026 list staying destination mode) — all passed first try, alongside the file's 4 pre-existing tests (still passing, confirming no regression to the `booking_id`-present path). The existing "no booking_id" test was renamed and its comment corrected to clarify it now covers the genuinely-nothing-to-resolve-from case (no `booking_id` AND no `tour_dates`), since the old comment's framing ("New Booking/POS Card Reader tab" fallback) is exactly what this revision closes. 139 tests passing (up from 136), zero orphaned rows (this repository method makes no database writes of its own — nothing to check for leftovers).

This closes the last remaining known gap across the ENTIRE 2027 deferred-transfer project. Every creation site, every confirmation site, the full refund/dispute cutover, the legacy creation-method audit, and this final tour-date plumbing gap are all done. Only genuinely unrelated, pre-existing architectural notes remain on record (e.g. `PaymentLinkService::saveTipPayment()`'s own direct-Stripe-SDK-call pattern, Section 7.2s) — none of them block or weaken anything this plan shipped.

7.2u Rename: ThreeWayPaymentService → TransferPaymentService, `payouts` → `transfers` shipped

A post-completion cleanup requested directly: the class and table this whole plan is built on top of were renamed for clarity — `ThreeWayPaymentService` never described what the class actually does (move money via Stripe Transfers), and `payouts` collided in spirit with the entirely separate, pre-existing Stripe-Connect-bank-payout system (`auto_payouts` / `pending_payouts` / `PayoutApprovalService` / `PayoutNotification`, none of which this plan has ever touched). Investigated that separate system fully before changing anything, to make certain the two were never confused: confirmed via a dedicated scoping pass that it lives in entirely different tables, a different `Payout` reference (`Stripe\Payout`, the SDK class, inside `PayoutApprovalService` / `ProcessAutoPayouts` — NOT `App\Models\Payout`), and shares no code path with anything this project built. Left completely untouched; its own 75-test suite re-run and confirmed green after every other change below.

- Class + namespace: `App\Services\ThreeWayPayment\ThreeWayPaymentService` → `App\Services\TransferPayment\TransferPaymentService` (directory moved, not just the class renamed, so nothing about the new name disagrees with its own folder). Its sibling in the same directory, `PayoutService` (two small helpers used only by the class above), renamed to `TransferService` for the same reason — left unnamed by the user's own request but mechanically identical in shape and clearly due the same treatment.
- Model: `App\Models\Payout` (`$table = 'payouts'`) → `App\Models\Transfer` (`$table = 'transfers'`), including its `$fillable` list.
- Table + columns: `payouts` → `transfers`; `payout_type` → `transfer_type`; `payout_status` → `transfer_status`; `payout_amount` → `transfer_amount`. New idempotent migration `2026_09_11_000001_rename_payouts_table_to_transfers.php` — every step (table rename, each column rename, four index renames including one Laravel auto-named after the original table) individually guarded via `Schema::hasTable()` / `hasColumn()` / `information_schema.STATISTICS` checks, confirmed safe to re-run by actually re-invoking `up()` a second time locally with no errors. Applied to all 13 local tenant databases; spot-checked schema (types/nullability/defaults all preserved exactly) and row counts on several.
- Deliberately left unchanged: local PHP variable names (`$payout`, `$payoutAmount`, etc. — internal detail, not a caller-visible leftover) and the `payout_amount` / `platform_fee` Stripe metadata keys this whole payment codebase already sends on every Transfer/PaymentIntent call (8 files: `StripeUtils`, `BookingUtils`, `CartService`, `StripeService`, `BookingService`, `V3/TourBookingRepository`, `SwitchTourPackageCardService`, this class itself) — that's wire-format data already visible on live historical Stripe records, a separate concern from the DB schema and out of scope for a rename request about "the class" and "the table."

Updated every genuine reference across the codebase: 22 production files mechanically batch-renamed, plus 3 files (`V3/TourBookingRepository.php`, `SwitchTourPackageCardService.php`, `DisputeService.php`) handled by hand where a real `payout_amount` Stripe-metadata key sat right next to the DB-column occurrences that needed renaming, to avoid touching the wrong one. 46 test files batch-renamed the same way (tests build fake DB rows directly, so every `payout_amount` hit there really was the DB column — verified first by confirming none sat inside a captured/asserted Stripe `metadata` array), plus the `tests/Unit/ThreeWayPayment/` directory itself moved to `tests/Unit/TransferPayment/`. Two already-applied historical migrations' own docblocks (`2026_09_07_000002` / `...000003`) still say `ThreeWayPaymentService` / `payouts` — deliberately NOT edited, matching this project's own established rule that a migration already applied (including just locally) is a historical record, not a living file.

139 tests passing throughout (same count as before this rename — a pure rename changes no behavior), zero orphaned rows, and the separate 75-test AutoPayout suite re-confirmed green and untouched. This document itself (the file you're reading) was also swept for stale mentions of the old names, correcting two real mistakes a first blind find-and-replace pass introduced (it briefly corrupted the two historical migration filenames quoted in Section 8's table below into names that don't exist on disk — caught by comparing against the real filenames before finalizing, not assumed correct).

Follow-up, same revision — method names checked too, not just the class/table: the user asked directly whether any method names inside the renamed files still carried "payout"/"threeway" wording. Grepped every method definition in the two renamed service files plus every other production file touched by the class/table rename — found five: `updatePartnerPayout()`, `storeTransferInPayout()`, and `storeReverseTransferInPayout()` on `TransferPaymentService`, plus `storePaymentInPayout()` and `storeRefundInPayout()` on the small sibling `TransferService`. Traced every call site of all five BEFORE renaming (`storeReverseTransferInPayout()` alone had ~36 call sites across 7 files — `V3/TourBookingRepository.php`, all three `CancelBooking/*` services, and all three `ExtendBooking*Service` files), then renamed to `updatePartnerTransfer()`, `recordPartnerTransfer()`, `recordReversedPartnerTransfer()`, `recordCompletedTransfer()`, and `recordReversedTransfer()` respectively — names chosen so the "record vs. reverse vs. update" verb and the "Partner" qualifier make each one's purpose and table-writing shape unambiguous at the call site, and so the two `TransferService` names (used only for the Switch-Package old/new-package pair) can never be confused with `TransferPaymentService`'s own similarly-shaped partner methods. Two test method names in `TransferToOperatorTest.php` (`test_..._primary_operator_payout` / `test_..._completed_payout_...`) renamed the same way. Local variable names (`$payout`, `$payoutId`, etc.) and the two `TransferService` method parameters literally named `$payoutAmount` deliberately left alone — same internal-detail scope boundary as the class/table rename itself. Re-ran the full curated regression suite (139/139 still green) plus 17 additional pre-existing, non-`DeferredTransfer`-named tests that happen to exercise these same classes (`CancelBookingManualOverridePlatformFeeTest`, `PartialRefundCancelCapsTicketAtNetAvailableTest`, `TicketRefundCancelIgnoresAddonHistoryTest`, and four others) — all passing, zero orphaned rows.

One more pass, same revision — broadened beyond method names to every test file's own prose. Checked all 46 `DeferredTransfer*Test.php` files (not just `TransferToOperatorTest.php`) for their own stray "payout" wording: found one more actual TEST method name (`DeferredTransferDisputeLossFlowTest.php`'s `test_..._with_no_primary_operator_payout_is_skipped()`, renamed to end `..._transfer_is_skipped()`), plus ~19 files' worth of comment/docblock/assertion-message prose that specifically named the OLD table or row ("a primary_operator payout row", "the transactions/payouts ledger", "payouts.transaction_id has no real...") — including one exact assertion-message pair repeated verbatim across roughly a dozen files ("No payout row exists for a destination-mode booking..." / "A primary_operator payouts row must be written.") fixed in one scoped pass across all of them at once. Left alone the remaining handful of hits that use "payout" as ordinary English (e.g. "an update to a pre-existing payout") or describe the deliberately-preserved `$payoutAmount` variable in a comment — neither misrepresents the renamed table/column. Also re-swept the whole `app/` + `tests/` tree one more time for any other method definition carrying "Payout"/"ThreeWay" wording (excluding the known-unrelated AutoPayout system) — found exactly 2 more, `TransactionService::storeZeroAmountRefundForPayoutInTransaction()` / `storeZeroAmountPaymentForPayoutInTransaction()` — investigated and confirmed these write to the (unrelated) `transactions` table, never call `TransferPaymentService`, and live in a file this whole rename never touched; deliberately left unrenamed as genuinely out of scope. 139/139 tests still passing after this pass too.

Final follow-up, prompted by the user re-raising this well after the rest of the project was already complete (Section 7.11's own work): the Stripe-facing "payout_amount" metadata key itself. This was the ONE thing Section 7.2u's original rename pass explicitly scoped OUT at the time — quoting that decision: "the `payout_amount` / `platform_fee` Stripe metadata keys this whole payment codebase already sends on every Transfer/PaymentIntent call... wire-format data already visible on live historical Stripe records, a separate concern from the DB schema." The user revisited that call directly: since the DB column, the class, and the method names are all already `transfer`-worded, the one Stripe-facing key still saying `payout_amount` was now an inconsistency worth fixing, not a boundary worth defending — especially now that Section 7.11's own fix means this exact key is what ends up mirrored onto the Transfer and destination_payment charge objects too (via the metadata-copy mechanism), so leaving it as `payout_amount` there would have reintroduced the old wording right back into the very objects Section 7.11 just spent effort enriching.

Renamed the metadata KEY (left-hand side only, in every Stripe-facing array literal) from `payout_amount` to `transfer_amount` across all 8 files that set it: `TransferPaymentService.php` (5 sites), `StripeUtils.php` (3), `StripeService.php` (2), `BookingUtils.php`, `CartService.php`, `BookingService.php`, `V3/TourBookingRepository.php`, and `SwitchTourPackageCardService.php` (1 each) — confirmed via a proactive whole-`app/` grep that this key is written in exactly these 8 places and read back programmatically nowhere in this codebase (it only ever reaches Stripe's own dashboard/API, never parsed by our own code), so the rename carries zero business-logic risk. Deliberately left every INTERNAL PHP variable/array-key spelled `payoutAmount` / `$data['payout_amount']` untouched — same "internal detail, not a caller-visible leftover" boundary the original class/table rename already established; only the string that Stripe itself displays changed. Also renamed the one remaining Stripe-facing description string still saying "payout": `transferToOperator()`'s `"Operator payout for Booking #$bookingId"` → `"Operator transfer for Booking #..."`, and `AttachDestinationTransferMetadata`'s own fallback description (Section 7.11) from `"Payout for Booking #..."` to `"Transfer for Booking #..."`. Confirmed the two `PayoutApprovalService.php` "Auto payout: ..." descriptions are the entirely separate, pre-existing AutoPayout/bank-payout system (deliberately untouched throughout this whole project) and correctly left alone — "Auto Payout" is that system's own real feature name, not leftover rename debris. 4 test assertions updated to match (2 in the new `AttachDestinationTransferMetadataTest.php`, 1 in `TransferToOperatorTest.php`, one test's own made-up input string updated for consistency though not strictly required). 162/162 tests still passing, zero orphaned rows, zero business-logic impact confirmed by the same grep that found nothing reads this key back.

Immediate follow-up, user asked directly: "what about the variable with name payout, e.g. $payoutAmount" — revisiting the ORIGINAL class/table rename's own "internal detail, leave alone" boundary a second time, this time for local variable names inside `TransferPaymentService.php` specifically (the class itself, as opposed to the 7 OTHER shared files the metadata-key rename above also touched — see the scope note below). Enumerated every variable name containing "payout" in the file (118 occurrences: `$payout` ×64, `$payoutAmount` ×37, `$payoutStatus` ×8, `$reversedPayout` / `$reversedPayouts` ×5, `$payoutTransaction` ×2, `$payoutId` ×2) and read the WHOLE file function-by-function before touching anything, specifically checking for a real collision risk: three functions (`updatePartnerTransfer()`, `transferBetweenConnectedAccounts()`, `transferToNewPartner()`) ALREADY use `$transfer` as the variable name for the Stripe API response object from `createTransfer()` — a blind `$payout` → `$transfer` rename would have silently collided with that existing variable in exactly those three functions (two genuinely different things — the local `transfers`-table ledger row vs. the live Stripe object — ending up with the identical name). Renamed instead to `$transferRecord` (the DB ledger row, used consistently in every function whether or not that function also happens to have a competing `$transfer`), `$transferAmount`, `$transferStatus`, `$transferRecordId` (the ledger row's own int primary key — deliberately NOT `$transferId`, which already means the STRIPE transfer id string elsewhere in this same file), `$completedTransfer`, and `$reversedTransferRecord(s)` —

applied via an ORDERED set of regex substitutions (longest/most-specific pattern first, e.g. `$reversedPayouts` before the bare `$payout`) specifically to avoid one pattern accidentally matching as a substring of another. Also swept the file's own remaining comments/docblocks and three user-facing `Exception` messages ("Please enter amount less than payout amount" → "...transfer amount") that a blind variable-only rename would have missed. Zero "payout" mentions remain anywhere in this file. 162/162 tests still passing (confirming the rename is purely cosmetic, no behavior changed) — no test asserted on the old exception-message wording either, confirmed by grep before finalizing.

Scope note, deliberately NOT extended to the 7 other shared files (`StripeUtils.php`, `StripeService.php`, `BookingUtils.php`, `CartService.php`, `BookingService.php`, `V3/TourBookingRepository.php`, `SwitchTourPackageCardService.php`): those files' own internal `$payoutAmount`-shaped variables represent a pre-existing, general "operator's net payout" calculation concept used across the WHOLE booking system for every booking regardless of destination/deferred mode or this project's own existence — a materially different, much broader scope than `TransferPaymentService.php` itself (the one class this entire rename saga, Section 7.2u, has always been specifically about). Flagged to the user as a distinct, larger decision rather than assumed silently either way.

That scope note was revisited and reversed shortly after, prompted directly by the user re-reading a code excerpt and spotting `'transfer_amount' => $payoutAmount,` still in it. What first looked like a single leftover local variable turned out, on tracing it, to be a materially different and higher-risk shape than the same-scope local-variable renames done everywhere else in this saga: `payoutAmount` (camelCase, no underscore) crosses file boundaries via PHP's `compact()` — a producer function builds `compact(..., 'payoutAmount')` in one file, and a consumer elsewhere reads `$result['payoutAmount']` back out of the returned array, since `compact()` generates the array key FROM the variable's name. A same-scope rename discipline doesn't apply here; every producer/consumer pair had to move together, atomically. Before touching anything, enumerated the token across the WHOLE `app/` tree (not just the 7 files this scope note originally listed) and found 3 MORE files carrying it that the original pass never saw: `EditBookingService.php` (4 separate `compact()` return sites), `PaymentLinkService.php`, and `StripeRepository.php` — plus the renamed sibling class `TransferService.php` (parameter names only). Re-confirmed, via a fresh whole-tree grep, zero collision with the entirely separate AutoPayout/bank-payout system (`payoutAmount` camelCase with no underscore is used exclusively by this pricing-calculation subsystem) before renaming.

Renamed uniformly across all 10 files at once — `StripeService.php` (22 replacements), `StripeUtils.php` (14), `EditBookingService.php` (10), `BookingService.php` (8), `TransferService.php` (4), `V3/TourBookingRepository.php` (4), `SwitchTourPackageCardService.php` (2), `StripeRepository.php` (3), `CartService.php` (1), `PaymentLinkService.php` (1); 69 individual token replacements total, covering both the variable-name form and the string-array-key form via ordered regex substitution — safe specifically BECAUSE a single uniform rename across every file keeps each producer/consumer pair in sync automatically; a partial or file-by-file rename would not have. Verified via both the existing 162-test curated regression suite AND a broader 58-test sweep of pre-existing booking/add-on-calculation tests that independently exercise these same shared pricing functions (220 tests run in total, zero failures) — the wider sweep specifically because this rename's blast radius (10 files, several touched by every booking regardless of destination/deferred mode or this project's own existence) is larger than any single-file rename earlier in this saga. 162/162 (plus the 58 additional) tests passing; zero business-logic risk, confirmed the same way as every prior step of this rename.

7.3 Operator & selling-commission transfer engine shipped

Done. Per Decision 1 (resolved — Section 1/3/12), this is not a scheduled sweep; it is four new static methods on the existing `App\Services\TransferPayment\TransferPaymentService`, called synchronously from the exact same trigger points `transferToPartner()` already uses today (Section 6.6):

- `transferToOperator(array $data): bool` — for a non-partner booking only. Mirrors `transferToPartner()`'s math with the partner-fee subtraction dropped (`payoutAmount = amount − platformFees`), calls `createTransfer()`, records a `transfers` row with `transfer_type='primary_operator'`. Wraps the Stripe call in try/catch: a failure is logged and recorded as `transfer_status='Failed'` rather than thrown, since the underlying charge has already succeeded by the time this runs.
- `transferSellingCommission(array $data): bool` — for a partner-package booking's selling tenant. Sends only the configured Partner Fee (`$data['partner_fees']`) — not the full remainder. If the Partner Fee is 0, no Stripe call is made and no ledger row is written at all (`transfer_type='selling_commission'` when one is written). Same try/catch-and-log failure handling as `transferToOperator()`.
- `reverseOperatorTransfer($tourBookingId, $transactionId, $amount)` / `reverseSellingCommissionTransfer(...)` — the `transfer_type`-filtered reversal counterparts, each with the same over-reversal guard as the existing `reverseTransfer()`. The selling-commission reversal is a safe no-op (not an error) when no row exists, correctly covering the case where the Partner Fee was 0 and nothing was ever transferred.
- Every pre-existing lookup query in `TransferPaymentService` (8 of them) was hardened with an explicit `transfer_type='partner_split'` filter, so all three purposes sharing one table can never cross-contaminate each other's lookups — regression-tested explicitly (Section 10).
- Idempotency is inherited for free from the same guard the 19 existing call sites already use: the `transactions`-table `$transactionCount==0` check that gates the transaction insert also gates every transfer call, so a retried/duplicate confirmation (sync race with the async webhook, a resubmitted request) cannot double-transfer.
- For a partner-package deferred-mode booking, `transferToOperator()` is never called — `transferSellingCommission()` and `transferToPartner()` (unchanged) run instead, together. Traced and resolved, Section 6.7 / Decisions 5 & 6: this design was corrected after an earlier draft mistakenly reused `transferToOperator()` for the selling tenant's payout, which would have sent them the ticket-cost portion too.

10 unit tests, `tests/Unit/TransferPayment/TransferToOperatorTest.php`: card success, cash (no Stripe call), Stripe failure handling, partial reversal, over-reversal guard, partner-split/primary-operator cross-contamination regression, the partner-package flow with a non-zero Partner Fee (both transfers fire, exact documented amounts), the zero-Partner-Fee case (selling tenant gets nothing, no ledger row, 3rd party still gets the full ticket cost), and selling-commission reversal (including its no-op case). All passing against the local `dixies` tenant DB via the project's real-DB testing convention, gateway calls mocked per the repo's existing pattern (see e.g. `CancelBookingManualOverridePlatformFeeTest`).

7.4 Refund path — mode-aware branching shipped, partial

Phase 4 started. Both halves of the design now exist: the reversal half (`reverseOperatorTransfer()`, Section 7.3) shipped back in Phase 0/1; the refund-shaping half, `PaymentGatewayInterface::buildRefundParams(array $params, string $mode = DESTINATION): array`, shipped this revision on `StripePaymentGateway` alongside `buildConnectedPaymentData()` — the same shape decision (take the caller's already-built array, branch on `$mode`, return the corrected array) rather than the bespoke `buildRefundParams($paymentIntentId, $amountCents, $keepApplicationFee)` signature originally sketched here, since research confirmed every one of the ~78 `refundPayment()` call sites already builds a `$refundParams` / `$refundArray` associative array first — wrapping that array through the helper needed no call-site restructuring, only a mode lookup and one extra line:

```
// destination mode (unchanged today, and every call site not yet touched):
buildRefundParams(['payment_intent' => $piId, 'amount' => $cents, 'reverse_transfer' => true, 'refund_application_fee' => true])
// returns the array completely unchanged

// deferred_transfer mode (new):
//  - reverse_transfer / refund_application_fee stripped entirely (there is no transfer_data on this
//    charge for Stripe to reverse — sending either key would error)
buildRefundParams([...same array...], PaymentModeResolver::DEFERRED_TRANSFER)
// returns ['payment_intent' => $piId, 'amount' => $cents]
//  - PLUS, separately, the caller must still call reverseOperatorTransfer($bookingId, $transactionId, $amount)
//    (or reverseSellingCommissionTransfer()/reverseTransfer() for a partner-package booking) — already
//    shipped, Section 7.3 — buildRefundParams() only shapes the Stripe refund call itself.
```

4 new unit tests, `tests/Unit/Services/Payments/StripePaymentGatewayDeferredModeTest.php` (appended alongside the existing `buildConnectedPaymentData()` tests) — destination mode unchanged (default and explicit), deferred mode strips both keys, and a defensive case proving `unset()` on an absent key is a safe no-op (several call sites only ever set one of the two keys, never both).

First real call site wired as proof-of-pattern (the same "wire one flow fully, then plan to replicate" approach Phase 2/3 opened with in Section 7.2a): `App\Services\RefundBooking\TicketRefundService::refund()`'s Card branch — both of its `refundPayment()` calls (the multi-transaction/installment loop, and the single-transaction "else" branch) now read `payment_mode` back from the ORIGINAL charge's own locked-in `transactions` row (never re-derived from the booking's current tour date, Section 5.2 — a combo/installment booking can have transactions created under different modes), route their params through `buildRefundParams()`, and — when the mode is deferred — additionally call `TransferPaymentService::reverseOperatorTransfer($bookingId, $transactionId, $netAmount)` right after the Stripe refund succeeds. This method has no partner-package handling anywhere (confirmed by reading it in full), so only the non-partner reversal applies here — the same "replicate the existing gap, don't invent new partner logic" precedent as `createPaymentIntentForCart()` (Section 7.2p).

2 new tests, `tests/Feature/Payments/DeferredTransferTicketRefundFlowTest.php` — a real 2027-dated Card booking with its original transaction's `payment_mode` set to `deferred_transfer` and a hand-built `primary_operator` payout row (simulating what `transferToOperator()` would already have written at charge time), gateway mocked, calling the real `TicketRefundService::refund()` against the "else" (single-transaction) branch: asserts the captured Stripe refund call carries no `reverse_transfer` / `refund_application_fee` keys, that `reverseOperatorTransfer()` fires with the matching transfer id and amount, and that a `Reversed` payout ledger row lands. A second test proves the destination-mode regression case — `reverse_transfer` still sent, `reverseTransfer()` never called — for a 2026-dated booking, unchanged from today.

Second site wired same revision: `App\Services\RefundBooking\WeatherRefundService::refund()` — the "100% Refund" sibling of `TicketRefundService::refund()`, dispatched from the exact same `PosRepository::handleRefundJobs()` switch (`refund_type == '100_refund'` rather than `'full_refund'`). Read in full: no partner-package handling exists here either, so the same non-partner-only `reverseOperatorTransfer()` dispatch applies. This method has FOUR `refundPayment()` call sites (one more than `TicketRefundService`) — three inside its multi-transaction loop (a plain first-refund case, plus two near-identical "added people to booking" / rate-difference sub-branches distinguished only by `payment_type`) and one simpler single-transaction "else" branch. All four now read `payment_mode` back from their own relevant original transaction (`$value->payment_mode` in the three loop sub-branches, `$transaction->payment_mode` in the else branch), route through `buildRefundParams()`, and dispatch `reverseOperatorTransfer()` when deferred — mechanically identical to `TicketRefundService`'s fix, applied four times instead of two.

2 new tests, `tests/Feature/Payments/DeferredTransferWeatherRefundFlowTest.php` (both passed FIRST try) — same setup shape as `TicketRefundService`'s own tests (a real 2027-dated Card booking with its original transaction's `payment_mode` set to `deferred_transfer` and a hand-built `primary_operator` payout row), exercising the else (single-transaction) branch: asserts no `reverse_transfer` / `refund_application_fee` keys reach Stripe, `reverseOperatorTransfer()` fires with the matching transfer id/amount, and a `Reversed` ledger row lands. A second test proves the 2026/destination-mode regression case, unchanged from today.

Third site wired same revision — the first Phase 4 site with genuine partner-package handling: `App\Services\CancelBooking\TicketRefundCancelService::cancel()`, reached from an entirely different entrypoint (the "Cancel Booking" flow's `handleCancelRefundJobs()` switch in `V3\TourBookingRepository.php` /legacy `TourBookingRepository.php`, not `PosRepository`'s "Refund Order" flow the first two sites came from). Unlike its `RefundBooking`-namespace siblings, this method ALREADY has working partner-package logic: for a partner booking, it calls the pre-existing, mode-independent `TransferPaymentService::reverseTransfer()` (claws back the 3rd party's ticket-cost transfer — correct in both modes unchanged, since `transferToPartner()` itself always fires identically per Decision 5) and `recordReversedPartnerTransfer()` (a bookkeeping-only bookkeeping insert against the `partner_split` row, pre-existing, untouched).

What was genuinely missing under deferred mode: the SELLING TENANT's own share. In destination mode, that share arrives bundled into the destination charge's remainder and is clawed back for free by the `reverse_transfer` flag on the Stripe refund itself — no extra code needed, which is exactly why this gap was invisible before. In deferred mode, the selling tenant instead receives it via a separate `transferSellingCommission()` transfer (a distinct `selling_commission`-typed payout row) that `reverse_transfer` can no longer reach at all, once stripped by `buildRefundParams()`. Fixed both of this method's `refundPayment()` call sites (a multi-transaction loop and a simpler single-transaction "else" branch, the same two-site shape as its `RefundBooking` siblings) to additionally dispatch, only when deferred: `reverseSellingCommissionTransfer()` for a partner booking (reusing this method's own already-computed `$netPartnerFees` / `$netPartnerFeesForTransaction` — the SAME dollar figures the pre-existing `recordReversedPartnerTransfer()`

call already relies on, since `transferToPartner()`'s bookkeeping is mode-independent and those figures are valid regardless of mode), or `reverseOperatorTransfer()` for a non-partner booking — mutually exclusive, matching the file's own `is_partner_booking` branching everywhere else.

3 new tests, `tests/Feature/Payments/DeferredTransferTicketRefundCancelFlowTest.php` (all three passed FIRST try, no bugs found despite the added complexity): a non-partner 2027-dated booking proves `reverseOperatorTransfer()` fires alone; a PARTNER 2027-dated booking (`is_partner_booking=1`, a hand-built `partner_split` payout row simulating `transferToPartner()`'s past write, a hand-built `selling_commission` payout row simulating `transferSellingCommission()`'s past write, and `tour_booking_history.partner_fees` set to a real $10 value since `getNetPartnerFees()` reads it directly) proves BOTH `reverseTransfer()` (3rd party, $90) AND `reverseSellingCommissionTransfer()` (selling tenant, $10) fire with exactly the right amounts, distinguished by transfer id, and that no `primary_operator` row is ever touched; a 2026-dated regression case proves the destination-mode path (a single `reverse_transfer`-flagged Stripe refund, no new reversal calls) is unchanged from today.

Fourth site wired same revision: `App\Services\CancelBooking\WeatherRefundCancelService::cancel()` — the "100% Refund" sibling of `TicketRefundCancelService::cancel()` within the SAME "Cancel Booking" entrypoint, same partner-package shape. This method has FOUR `refundPayment()` call sites (one more than its sibling): a plain first-refund loop case and an "added people" rate-difference loop sub-branch (both WITH pre-existing partner-package handling — wired identically to the sibling's own two sites), a THIRD loop sub-branch (`paymentTransCount>=1 && payment_type==1`) that — read carefully before touching — has no partner-package handling in destination mode either, today (no `reverseTransfer()` call exists there at all, a pre-existing asymmetry unrelated to this change), and the simpler single-transaction "else" branch (WITH partner handling, same shape as the sibling's own else branch).

Deliberate design choice for the third (partner-less) sub-branch: rather than inventing NEW partner-aware reversal logic this pre-existing branch never had, its deferred-mode addition unconditionally dispatches `reverseOperatorTransfer()` with no `is_partner_booking` check at all — a safe no-op for a partner booking landing here (no `primary_operator` payout row would ever exist to find), faithfully replicating "this specific branch has never distinguished partner status" rather than silently fixing a gap outside this change's scope. The other three sites branch on `is_partner_booking` exactly like `TicketRefundCancelService`.

3 new tests, `tests/Feature/Payments/DeferredTransferWeatherRefundCancelFlowTest.php` (all three passed FIRST try) — identical shape to `TicketRefundCancelService`'s own three tests, exercising the else branch: non-partner (`reverseOperatorTransfer()` alone), partner (BOTH `reverseTransfer()` $90 and `reverseSellingCommissionTransfer()` $10, distinguished by transfer id), and a destination-mode regression case.

Fifth site wired same revision — closes out all three `CancelBooking/*` services: `App\Services\CancelBooking\PartialRefundCancelService::cancel()`, the genuinely most complex of the three (a partial refund can spill from the 3rd party's own net ticket-cost share into the selling tenant's Partner Fee once the requested amount exceeds what the ticket-cost portion alone can cover). FIVE `refundPayment()` call sites: two in the main per-transaction loop (already-computed `$refundAmountFromConnectedAccountForTrans` / `$refundPartnerFeesForTrans` — reused directly for `reverseSellingCommissionTransfer()`, same pattern as the other two `CancelBooking` siblings) and three in a separate discount-transaction loop (`$discountAmount` / `$dTransaction->amount` / `$finalAmount` cases).

Real finding while reading the discount-transaction loop's own pre-existing partner logic: it only ever calls the mode-independent `reverseTransfer()` for a partner booking — never computes or reverses any selling-commission-specific figure, matching the SAME reasoning already established for Edit Discount's own creation side (Section 7.2i): discount-type charges carry no separate partner-fee cut of their own (`createEditDiscountPaymentIntent()` hardcodes `partner_fees => 0` for `transferSellingCommission()`, which then no-ops with zero Stripe call/no ledger row). Since a partner booking's discount charge never had a selling-commission transfer to begin with, its deferred-mode reversal correctly does NOTHING beyond the existing `reverseTransfer()` call — only non-partner bookings get a new `reverseOperatorTransfer()` dispatch at these three sites. This is the third genuinely-encountered case (after Sections 7.2p and this section's own fourth site) of deliberately NOT inventing partner logic a branch never had.

3 new tests, `tests/Feature/Payments/DeferredTransferPartialRefundCancelFlowTest.php` (all three passed FIRST try): a non-partner case ($50 of $100, `reverseOperatorTransfer()` alone); a PARTNER case deliberately chosen to exercise the spillover — a $95 request against a $90 3rd-party share and a $10 Partner Fee — proving the reversal SPLITS correctly ($90 via the pre-existing `reverseTransfer()`, $5 — not the full $10 — via the new `reverseSellingCommissionTransfer()`), not just that some reversal happened; a destination-mode regression case.

All three `CancelBooking/*` services are now fully accounted for. A sixth site, `SwitchTourPackageCardService::run()`'s own refund branch, is also wired (Section 7.2f, Appendix A.4 site #19) — see that section for the full writeup; reasoned-through only, not independently tested, and notable for needing a `package_id`-based payout lookup instead of the `transaction_id`-based one every other Phase 4 site used.

Seventh site wired this revision: `App\Repositories\TourBooking\TourBookingAddonRepository::refundAddonsByBookingId()` — the shared add-ons refund method called by all three `CancelBooking/*` services (already wired above) whenever a booking with paid add-ons is cancelled or refunded. Confirmed by reading it in full: no partner-package handling exists anywhere in this method today, matching Section 7.2k/7.2l's own finding for this same file's CHARGE-side methods — only the non-partner `reverseOperatorTransfer()` dispatch applies.

Genuinely different resolution shape from every prior site: this method groups refunds per Stripe PaymentIntent id (`$piId`, a plain string — `TourBookingAddonTransaction.transaction_id`), not by a `transactions`-table row id directly. To read `payment_mode` back and to know which `transaction_id` `transferToOperator()` used when it created the original `primary_operator` payout row (Section 7.2k), resolved the corresponding `transactions` row first — `charge_type='addons'` and `transaction_number = $piId`, the exact row `storeAddonsPaymentInTransaction()` writes at charge time — then used ITS OWN id for the reversal dispatch, the same "resolve then delegate to the existing engine" pattern Switch Package's refund branch just established two paragraphs up.

2 new tests, `tests/Feature/Payments/DeferredTransferAddonsRefundFlowTest.php` (both passed FIRST try) — a hand-built paid card add-on transaction plus a matching `transactions` row (locked to `deferred_transfer`) and a `primary_operator` payout row simulating what the charge side already wrote, calling `refundAddonsByBookingId()` directly: asserts no `reverse_transfer` / `refund_application_fee` keys reach Stripe, `reverseOperatorTransfer()` fires with the matching transfer id/amount, and a `Reversed` ledger row lands. A second test proves the destination-mode regression case.

REAL REGRESSION CAUGHT AND FIXED, THIS REVISION — A MOCKING HAZARD, NOT A LOGIC BUG

Running the pre-existing 169-test `tests/Unit/Bookings/` add-ons suite after wiring the site above surfaced 9 failures across 6 files (`CancelAddonSkipsDontRefundSettledFieldTest`, `CancelBookingAddonAlreadyPartiallyRefundedTest`, `ChangePeopleAddonNetChargeOffsetPiMismatchTest`, `RefundOffsetAddedItemFeeTest`, `SplitBookingPaxAddonsTest`, `TicketRefundDowngradeFeeProportionalOnCancelTest`, `UpdateBookingAddonsNetChargeOffsetPiMismatchTest`) — all `"Undefined array key "payment_intent""`, none touching deferred mode at all. Root cause: these pre-existing tests mock `PaymentGatewayInterface` via PHPUnit's native `createMock()`, stubbing only `refundPayment()` — calling the newly-added `buildRefundParams()` on an otherwise-unstubbed mock returns PHPUnit's default empty-array value for an `array`-typed return, silently replacing the real refund params with `[]` before they ever reached the (correctly stubbed) `refundPayment()`. This is the risk of adding any brand-new interface method to an existing, already-mocked call site — not a defect in `buildRefundParams()` itself, which behaves correctly against a real gateway. Fixed by adding `$fakeGateway->method('buildRefundParams')->willReturnArgument(0)` (pass the params through unchanged, exactly matching real destination-mode behavior, which is what every one of these tests' bookings actually resolves to) to each of the 12 affected mock instances across the 7 files. All 169 tests in `tests/Unit/Bookings/` pass again.

Eighth, ninth, and tenth sites wired same revision — closes out ALL FOUR of this file's own refund call sites. `chargeAddonFields()` 's own REFUND section (sibling to Section 7.2k's already-wired CHARGE section, same method, same "group refunds by PaymentIntent id" shape) and `deletePaxAddons()` (the pax-removal addon refund path — both the standalone "delete only add-ons" endpoint and the "Update Booking" modal's explicit-field-ids flow route through it) both wired identically to `refundAddonsByBookingId()` above: resolve the original `transactions` row via `transaction_number` / `charge_type='addons'` , read `payment_mode` , route through `buildRefundParams()` , dispatch `reverseOperatorTransfer()` when deferred. `deletePaxAddons()` additionally caps its Stripe refund at what the PaymentIntent actually still has available (net of prior refunds) — the reversal correctly uses that SAME capped amount, not the raw pre-cap total, proven directly by a dedicated test.

`processAddonFinancialTransactions()` 's own REFUND action (private, sibling to its already-wired CHARGE action from Section 7.2l, Addon Edit Charge) completes the set — the riskiest of the four given this method's own extensive prior real-money-bug history (per-item PI-offset/cascade math, six separate bugs found and fixed across earlier sessions). One genuine subtlety caught before shipping: this method's refund amount actually sent to Stripe (`$stripeAmount`) includes NAT's own platform fee AND is capped against the PaymentIntent's own Stripe-side availability — neither of which matches what the operator's payout ever actually received. Used `$groupTotal` (the PRICE-only portion, uncapped) for the `reverseOperatorTransfer()` amount instead — the SAME value this method's own ledger-write two lines below already trusts as "the operator's real share" — rather than the Stripe-facing `$stripeAmount` , which would have over-reversed relative to what `transferToOperator()` ever actually paid out at charge time.

SAME MOCKING HAZARD CAUGHT AGAIN, PROACTIVELY THIS TIME — 3 MORE PRE-EXISTING TESTS IN 3 MORE FILES

Wiring `processAddonFinancialTransactions()`'s REFUND action broke `RefundOffsetMultiPiAndRefundTypeTest` and `UpdatePaxAddonsOffsetAttributionTest` the same "Undefined array key" way as the previous callout — both mock `PaymentGatewayInterface` via PHPUnit's `createMock()` without a `buildRefundParams()` stub. Fixed the same way. Having learned the lesson, proactively swept the WHOLE `tests/` tree afterward for every remaining `createMock(PaymentGatewayInterface::class)` occurrence that expects `refundPayment()` to actually be called (not `expects($this->never())`, which needs no stub since neither method fires) and found two more at-risk mocks that happened not to be exercised by this specific edit yet but would break the next site touched — `RestoreBookingAddonRefundTest` and `ChangePeopleAddonServiceTest` — fixed both pre-emptively rather than waiting to discover them via a future failure.

4 new tests across two files, `tests/Feature/Payments/DeferredTransferAddOnsRefundActionFlowTest.php` (`chargeAddonFields()` 's REFUND section — no pre-existing test existed for it at all, confirmed by grepping the whole `tests/` tree for its distinguishing `refund_items` request key, zero matches, so no mocking-hazard risk there) and `tests/Feature/Payments/DeferredTransferDeletePaxAddonsFlowTest.php` — all four passed FIRST try. `processAddonFinancialTransactions()` 's own REFUND action got `tests/Feature/Payments/DeferredTransferAddonEditChargeRefundFlowTest.php` (2 tests) — one assertion mistake caught and fixed before finalizing (asserted the reversal amount in dollars instead of the cents scale `reverseOperatorTransfer()` actually passes to the gateway, a test bug, not a production one).

Eleventh site wired same revision: `App\Services\Booking\CustomChargeBookingService::refundCustomChargeFromCancel()` — called by all three `CancelBooking/*` services whenever a cancelled booking has a paid custom charge. Confirmed by reading the whole file: no partner-package handling anywhere, so only the non-partner `reverseOperatorTransfer()` dispatch applies. This method already had its own pre-existing lookup resolving the exact original `transactions` row (`charge_type='custom_charge'`) for an unrelated purpose (its `application_fee_id`) — extended that SAME query to also select `payment_mode` rather than adding a second lookup.

Amount subtlety, reasoned through rather than assumed: for a "100% Refund," Stripe is refunded `$cardRefund` (price + NAT's own fee together), but `transferToOperator()` 's forward payout to the operator was always price-only (`platform_fees` subtracted out, Section 7.3) — so the reversal uses `$amount` (the price-only figure, already computed by the method's own pre-existing switch statement) instead of `$cardRefund` , the same "don't reverse more than the operator ever actually received" principle already applied to the addons file's own `processAddonFinancialTransactions()` site just above.

2 new tests, `tests/Feature/Payments/DeferredTransferCustomChargeRefundFlowTest.php` (both passed after one trivial type-strictness assertion fix — dollars-scale float vs an int literal, same class of test-only mistake as several earlier sites, not a production bug). Proactively checked the whole `tests/` tree for any pre-existing mock at risk of this session's now-familiar mocking hazard before running anything — none found referencing this method with a `refundPayment()` expectation, so no fixes needed this time.

Twelfth and thirteenth sites wired same revision — closes out `EditBookingService.php` , the second-to-last file on the whole Phase 4 survey. This file has two distinct refund call sites, both reached from the "Delete Pax from Group Booking" flow (shared by the direct "Edit Booking" page and Change People's own pax-removal path): the standalone partial-refund branch directly inside the public `deletePaxFromGroupBooking()` entrypoint, and a private helper, `refundProcessStripe()` (FIVE `refundPayment()` call sites across its full/100%-refund branches, each with an admin-selectable-transaction variant and a default-transaction variant). Confirmed by reading both in full: zero partner-package handling anywhere in this file's refund logic — only the non-partner `reverseOperatorTransfer()` dispatch applies, same as every addon/custom-charge site.

Cleaner resolution than most sites: unlike every prior site, the surrounding code here already fetches full, unrestricted Eloquent `Transaction` models (not raw DB rows with a limited column list) for its own existing purposes — so `payment_mode` was already directly accessible with zero query changes for most of the five sites; only the "100% Refund + admin-selected-transaction" sub-case needed its existing column-restricted lookup (`->first(['id', 'amount'])`)

extended to also select `payment_mode`. Same "reuse the price-only figure already computed" amount principle as the addons/custom-charge sites: `refundProcessStripe()`'s own `$total` variable (ticket-cost only, fees excluded — matching "Full Refund minus Fees"/"100% Refund" naming precisely) is what gets reversed, never the fee-inclusive `$Refund100`.

2 new tests, `tests/Feature/Payments/DeferredTransferDeletePaxFromGroupBookingFlowTest.php`, targeting `refundProcessStripe()`'s simplest branch (full refund, no admin-selected transaction) via PHP reflection — the private method is called directly rather than through the substantially larger, genuinely orthogonal `deletePaxFromGroupBooking()` entrypoint (200+ lines of group-rate-recalculation/contacts logic before the refund section is even reached), the same "test the specific thing you changed" scoping precedent as Switch Package/Edit Discount/Custom Charge earlier this session. Both tests passed after the now-familiar dollars-vs-cents assertion fix. Confirmed via grep that NO pre-existing test anywhere references this file at all — zero mocking-hazard risk, the second site all session with a completely clean proactive check. The partial-refund branch inside `deletePaxFromGroupBooking()` itself is wired identically but not independently tested — its own reversal amount (a single admin-typed dollar figure with no separately-computed price/fee split available anywhere in the surrounding code, unlike every other site) is used as-is, relying on `reverseOperatorTransfer()`'s own pre-existing over-reversal guard (Phase 0/1, already tested) as the safety net rather than inventing a fake split.

> **SIGNIFICANT NEW FINDING, THIS REVISION — A PHASE 2/3 ADDENDUM, NOT JUST PHASE 4**
>
> Investigating `TourBookingRepository.php`'s (the last remaining file) own refund call sites required first tracing which methods each one lives in. Its `editDiscountRefund()` — the REFUND counterpart of `createEditDiscountPaymentIntent()`, wired for V3 back in Section 7.2i — exists in both the V3 file *and* the LEGACY (non-V3) file. Checking the legacy file's own `createEditDiscountPaymentIntent()` (its charge side) turned up a real, previously-unnoticed gap: it was never wired at all — no `PaymentModeResolver` reference anywhere, no `payment_mode` threaded into `storePaymentInTransaction()`, no `transferToOperator()` dispatch. Confirmed the legacy repository is genuinely still live traffic, not dead code — `App\Http\Controllers\API\V1\TourBooking\TourBookingController` (the mobile API's older, V1 version) routes directly to it. This means every legacy (API V1) Edit Discount charge on a 2027+ booking would have silently stayed on destination mode forever, the exact same class of gap Section 7.2m/7.2o already found and fixed for the V1 *new-booking/3DS* flows — Edit Discount's own V1 equivalent had simply never been checked until now. Fixed this revision: wired legacy's `createEditDiscountPaymentIntent()` to mirror V3's own fix exactly (confirmed the legacy method has no partner-package handling either, matching its own refund side — replicated that gap rather than inventing new logic), then wired both legacy's and V3's own `editDiscountRefund()` refund sides. This finding does not, on its own, mean other legacy creation sites are ALSO unwired — it was traced specifically for Edit Discount, prompted by investigating this one refund site; a full audit of every other legacy creation method is explicitly NOT done this revision and is flagged as a real, open follow-up below.

Fourteenth through sixteenth sites wired this revision: `editDiscountRefund()` in both files. V3's version already had working partner-package logic (`reverseTransfer()` for the 3rd party, mode-independent, unchanged) — but `partner_fees` is hardcoded to 0 for edit-discount charges (the same finding from the charge side, Section 7.2i), so a partner booking never had a separate selling-commission transfer to claw back here; only the non-partner `reverseOperatorTransfer()` applies in both files. The reversal amount mirrors `createEditDiscountPaymentIntent()`'s own forward `transferToOperator()` convention (`new_total` minus `service_commission`), not the fee-inclusive figure sent to Stripe — the same "reuse the price-only figure" principle established across every prior addon/custom-charge/EditBookingService site.

8 new tests across two files: `tests/Feature/Payments/DeferredTransferEditDiscountRefundFlowTest.php` (V3, 2 tests) and `tests/Feature/Payments/DeferredTransferEditDiscountLegacyFlowTest.php` (legacy, 3 tests — charge side AND both refund-side cases). Both refund tests needed a genuinely different technique than every prior site: `editDiscountRefund()` unconditionally calls `$this->applyDiscount(...)` internally at its end (substantial, orthogonal discount-calculation/history-writing logic already flagged out of scope back in Section 7.2i/7.2q) — a Mockery partial mock wrapping the real instance was tried FIRST and found NOT to work (Mockery cannot intercept a class's own internal `$this->method()` self-call this way, confirmed by a genuine test failure before switching approaches), so a real anonymous PHP subclass overriding only `applyDiscount()` was used instead — standard virtual dispatch correctly routes the internal call to the override, letting the REAL, unmodified `editDiscountRefund()` run end to end.

Seventeenth and eighteenth sites wired this revision: `refundCustomCharge()` in both files — the "refund a custom charge directly" action, genuinely distinct from `CustomChargeBookingService::refundCustomChargeFromCancel()` (the cancel-booking variant already wired, Section 7.4). Structurally identical between legacy and V3 (confirmed by reading both in full before touching either — a useful habit after the Edit Discount discovery above), and confirmed zero partner-package handling in either. THREE `refundPayment()` call sites per file (a `full_refund` branch and two `100_refund` sub-branches), all fixed the same way: extended the method's own pre-existing `$paymentTrans` lookup to also select `payment_mode`, then reversed `$customChargeAmount` (the PRICE-only figure the method's own `full_refund` branch already sends to Stripe unmodified) rather than the fee-inclusive `$refund100` the `100_refund` branches use — the same "reuse the price-only figure" principle as every other custom-charge/addon/EditBookingService site this session.

4 new tests across two files, `tests/Feature/Payments/DeferredTransferRefundCustomChargeFlowTest.php` (V3) and `tests/Feature/Payments/DeferredTransferRefundCustomChargeLegacyFlowTest.php` (legacy) — all four passed FIRST try, zero pre-existing tests found referencing `refundCustomCharge()` at all (confirmed by grep before writing anything), so no mocking-hazard risk. Confirmed by testing that direct property assignment on a real `Illuminate\Http\Request` instance (e.g. `$request->custom_charge_ids = [...]`) correctly satisfies this method's own `isset($request->custom_charge_ids)` checks — PHP resolves a directly-assigned dynamic property before falling back to the framework's `__isset()` magic method, so this is a genuine, safe test-construction shortcut, not a fragile trick.

Nineteenth and twentieth sites wired this revision — the most sophisticated partner-fee-spillover logic found anywhere in `TourBookingRepository.php`: `refund()` in both files, the admin booking-detail "Refund Booking" action (distinct from Cancel Booking's own `PartialRefundCancelService`, Section 7.4, and every other site above). V3's version has genuine partner-package handling well beyond `editDiscountRefund()`'s: the identical `netTicketCost` / `netPartnerCommission` spillover math as `PartialRefundCancelService::cancel()`, but computed PER TRANSACTION via `TransferPaymentService::getNetPartnerAmountsForTransaction()` rather than per-booking, across THREE independent `refundPayment()` call sites: two nested inside the same `check[]` / `partial_refund[]` loop (a specific partial amount entered against one transaction, and "refund the entire remaining balance on this transaction"), plus a third booking-level "100%/Full Refund" branch using the single original completed transaction and the per-booking equivalents `getNetPartnerAmountsForBooking()` / `getNetPartnerFees()`. All three sites dispatch `reverseSellingCommissionTransfer()` for the partner case using the SAME already-computed spillover figure (alongside the pre-existing mode-independent `reverseTransfer()` for the 3rd party, unchanged), and the surrounding code's own already-validated `$partial` / `$remainingAmountForTrans` / `$request->partial_refund_value` for the non-partner case — confirmed these are genuinely PRICE-ONLY (ticket-cost-only) figures already, not fee-inclusive, because the method's own outer guard validates the SUM of all partial-refund entries against `$totalTicketCostAmount` (`BookingService::calculateRemainingTotalTicketCost()`'s `netAmount`) — the same net-of-fees concept

`transferToOperator()` 's own forward payout convention uses, so no new split needed inventing. Legacy's version, confirmed by reading it in full, has ZERO partner-package handling anywhere (matching the same legacy-vs-V3 divergence Edit Discount surfaced) — only the non-partner `reverseOperatorTransfer()` dispatch applies to legacy's three sites.

5 new tests across two files: `tests/Feature/Payments/DeferredTransferPartialRefundBookingFlowTest.php` (V3, 3 tests — non-partner deferred, a genuine partner spillover proving the SAME split-proof pattern as `PartialRefundCancelService` 's own test [$95 requested against a $90 net-ticket-cost/$10 Partner-Fee split spills exactly $5 into the selling tenant's commission], and a destination-mode regression) and `tests/Feature/Payments/DeferredTransferPartialRefundBookingLegacyFlowTest.php` (legacy, 2 tests). All five passed FIRST try. Proactively grepped the whole `tests/` tree before writing anything and found one genuine pre-existing caller of this exact method, `tests/Unit/Bookings/BookingsTest.php` (`testCanRefundCashBooking` / `testCannotRefundMoreThanPaidAmount`) — confirmed both exercise only the Cash path (`is_cash` set), which this method gates its entire Stripe/reversal block behind, so neither test ever reaches the code this session touched; no mocking-hazard fix needed, confirmed by re-running that file too. Only Site 1 (a specific partial amount against one transaction) was independently tested — Site 2 (refund the entire remaining balance on a transaction) and Site 3 (the booking-level 100%/Full Refund branch) are wired identically and reasoned through, the same "don't manufacture a second real payment just to reach a structurally-identical branch" judgment call already made for every prior multi-site method this session.

Twenty-first through twenty-third sites wired this revision: `editBooking()` in both legacy and V3 `TourBookingRepository.php` — by far the largest method touched in this whole survey (~2,020 lines in V3 alone), covering THREE distinct admin actions inside one method: "Update Permit Fee" (`action_type == 'update'` , 2 sites), "Delete Pax / Ticket Refund + Weather Refund" (`action_type == 'delete'` , 4 sites — a `full_refund` and a `100_refund` branch, each with a checked-transaction and an unchecked-transaction variant), and "Delete Pax / Partial Refund" (1 site) — 7 sites in V3, 8 in legacy (legacy's own `100_refund` branch has an extra sub-case: rather than hard-failing when the selected transaction's remaining balance is insufficient like V3 does, legacy quietly CAPS the refund to what's available, a genuine pre-existing behavioral divergence between the two files, left untouched). V3's Permit Fee and Delete Pax sites already had partner-package handling (a flat, unconditional `reverseTransfer()` for the whole permit-fee/ticket-cost amount — no spillover computation, unlike `refund()`) — added the non-partner `reverseOperatorTransfer()` dispatch mutually exclusive on `is_partner_booking` , reusing the SAME pre-existing amount variable (`$amountTotalrefundpermitfees` for Permit Fee, the already-computed `$ticketCost` for Delete Pax) the partner path already trusts — the established "reuse what's already there" principle applied once again. The Delete Pax "Partial Refund" site DOES have the netTicketCost spillover shape (matching `refund()` 's Sites 1/2) but with no separate partner-fee-spillover tracking — replicated that narrower shape rather than inventing a `reverseSellingCommissionTransfer()` call the surrounding code has no equivalent for.

Legacy confirmed to have ZERO partner-package handling anywhere in `editBooking()` (a targeted grep for `is_partner_booking` / `isPartnerBooking` across the whole method came back empty) — the same divergence already found for `editDiscountRefund()` / `refundCustomCharge()` / `refund()` . Legacy's Delete Pax sites reuse `$total` (the ticket-cost-only figure) rather than the fee-inclusive `$Refund100` for the reversal — the same "amount subtlety" already established for `EditBookingService.php` 's own twin "Full Refund" vs. "100% Refund" naming. One structural fix needed for legacy's Permit Fee checked-transactions site: the pre-existing code fetched its `$TSDetails` lookup (for `application_fee_id`) only AFTER the Stripe refund call — reordered that lookup to run BEFORE the refund so `payment_mode` could be resolved from it first, a pure reordering with no behavioral change (the query itself doesn't depend on the refund's result).

6 new tests across three files: `tests/Feature/Payments/DeferredTransferEditBookingPermitFeeFlowTest.php` (V3, 2 tests — the Permit Fee "else"/unchecked-transaction branch), `tests/Feature/Payments/DeferredTransferEditBookingDeletePaxFullRefundFlowTest.php` (V3, 2 tests — the Delete Pax "Ticket Refund"/unchecked-transaction branch), and `tests/Feature/Payments/DeferredTransferEditBookingPermitFeeLegacyFlowTest.php` (legacy, 2 tests). A genuine, pre-existing legacy quirk surfaced while writing the legacy test — NOT a regression from this session's changes, confirmed by reproducing it identically on the untouched destination-mode case too: legacy's permit-fee loop reads `$checkPermitfee` via two different key conventions within the SAME method (`"{rate_for}.{index}"` in most places, but the raw `$check` -array numeric index in one spot) — the test request simply populates both keys rather than the production code being changed. Only the simplest (unchecked-transaction) variant of each multi-branch site was independently tested; the checked-transaction variants, the 100_refund sub-branches, and the Partial Refund site are wired identically and reasoned through, the same scoping judgment call made for every other large multi-site method this session. Proactively grepped the whole `tests/` tree beforehand and found two genuine pre-existing callers of `editBooking()` (`DeletePaxInsufficientTransactionBalanceTest` , `EditBookingPermitFeePayLaterTest`) — read both in full and confirmed neither reaches the Card-payment code this session touched (one exercises only the pre-refund validation-throw guards, the other only non-Card payment types); re-ran both anyway to confirm zero regression.

Twenty-fourth and twenty-fifth sites wired this revision — closing out `TourBookingRepository.php` entirely: `deletePeopleChangePeople()` (private) in both legacy and V3, the shared delete-pax engine behind the "Change People" admin flow (`changePeopleBooking()`) — structurally a near-twin of `editBooking()` 's own "Delete Pax" action wired last revision, just reached from a different entrypoint. Same six-site shape in V3 (`full_refund` checked/unchecked, `100_refund` checked-capped/checked-uncapped/unchecked, plus a Partial Refund site with the `netTicketCostForTrans` spillover shape) with pre-existing partner logic (`reverseTransfer()` , gated on `$ticketCost`) — added the non-partner `reverseOperatorTransfer()` dispatch reusing that same `$ticketCost` variable, mutually exclusive on `is_partner_booking` . Legacy again confirmed to have ZERO partner-package handling (targeted grep, zero matches) — only `reverseOperatorTransfer()` applies to its six sites, reusing `$total` (ticket-cost-only) for the reversal amount, the same "amount subtlety" convention established throughout this file. One structural nuance specific to this method: the transaction lookup used for reversal (`$checkedTrans`) can legitimately resolve to `null` when no matching row exists (a genuine pre-existing possibility this method already defends against via a fallback chain to the loop's own `$transaction`) — `payment_mode` resolution follows the identical fallback chain rather than assuming a non-null result.

4 new tests across two files: `tests/Feature/Payments/DeferredTransferDeletePeopleChangePeopleFlowTest.php` (V3, 2 tests) and `tests/Feature/Payments/DeferredTransferDeletePeopleChangePeopleLegacyFlowTest.php` (legacy, 2 tests) — all four passed FIRST try. Since this method is private, called directly via PHP reflection (`ReflectionMethod::setAccessible(true)`) rather than through its own real entrypoint, `changePeopleBooking()` — a ~3,250-line dispatcher whose full add/remove-pax/discount business logic is a substantial, orthogonal undertaking, the identical scoping call already made for `EditBookingService::refundProcessStripe()` earlier this session (a pre-existing test for the OTHER "Change People" flow, `DeferredTransferChangePeopleFlowTest.php` , independently reaches the same conclusion in its own doc-comment). Proactively grepped the whole `tests/` tree beforehand and found zero pre-existing tests calling `deletePeopleChangePeople()` directly (three files reference it only in comments) — no mocking-hazard risk. Legacy needed one extra request field V3 doesn't (`$data['total_tourist']` , the booking's ORIGINAL tourist count for its own "delete all" guard

— V3 computes this from the DB itself instead), confirmed by reading legacy's own validation logic before writing its test rather than assuming parity with V3's request shape.

This closes out `TourBookingRepository.php` entirely for Phase 4 — every one of its 49 `refundPayment()` call sites across both legacy and V3, spanning five distinct method groups (`editDiscountRefund()`, `refundCustomCharge()`, `refund()`, `editBooking()`, `deletePeopleChangePeople()`), is now wired and tested. Combined with every other file closed out in earlier revisions (`PosRepository`, all three `CancelBooking/*` services, Switch Package, `TourBookingAddonRepository.php`, `CustomChargeBookingService.php`, `EditBookingService.php`), the only work remaining anywhere in Phase 4 is `DisputeService::handleDisputeLoss()`'s mode branch (Section 7.5).

Explicitly not wired this revision: all twenty-five wired methods' multi-transaction/installment loop branches beyond what's described above (the fix is applied and reasoned through for every site, but exercising each loop branch independently would mean manufacturing a second real card payment/installment on the same booking — judged disproportionate for proving a pattern already proven repeatedly on the simpler branch); a full audit of legacy's OTHER creation methods (new-booking, make-payment, combo-cart, etc.) for the same "never wired" gap found for Edit Discount — explicitly flagged, not attempted this revision; `DisputeService::handleDisputeLoss()`'s mode branch (Section 7.5), the last item on the Phase 4 list.

7.5 Dispute handling shipped

Done — the final Phase 4 site. `DisputeService::handleDisputeLoss()` now branches on mode, resolved from the ORIGINAL transaction's own locked-in `payment_mode` column (via `$dispute->transaction`, set at dispute-creation time by `findAssociatedTransaction()`, with a direct fallback lookup by the dispute's own `payment_intent` for the rare case that association missed) — never re-derived from the booking's current tour date, the same core principle (Section 5.2) applied at every other site this whole project. Destination-mode charges are untouched: still read `$charge->transfer` exactly as before. Deferred-mode charges carry no `transfer_data` at all, so there is no automatic transfer on the charge to expand — instead, look up the separate `primary_operator` Transfer already recorded in the `transfers` ledger (Phase 0/1's `transferToOperator()`) and reverse it via the already-shipped `TransferPaymentService::reverseOperatorTransfer()`, reversing whatever is still outstanding (netting out any prior partial-refund reversals) rather than a fixed pre-computed figure — the same principle used at every other Phase 4 refund site.

Deliberately non-partner-only, matching the destination-mode branch's own existing scope — confirmed via a dedicated architecture investigation of `TransferPaymentService`'s own class-level design comment and the original charge-creation code: `transferToOperator()` is never called for a partner-package booking in ANY mode (destination or deferred) — the selling tenant's commission and the 3rd party's ticket cost are handled entirely separately by `transferSellingCommission()` / `transferToPartner()`, both pre-existing, mode-independent, and untouched by this whole project. So a partner booking's dispute loss finds no `primary_operator` payout row in deferred mode either, and is skipped exactly the same way the destination-mode branch already skips today (its own "no transfer found on charge" case) — extending dispute handling to reverse a partner booking's own split transfers is a separate, pre-existing gap this project does not introduce or fix. One deliberate, minor asymmetry: `reverseOperatorTransfer()` doesn't return the Stripe reversal object (it writes its own Reversed row into `transfers`, the authoritative ledger for deferred-mode transfers throughout this project), so `stripe_reversal_id` is left unset for deferred-mode dispute losses — `reversed_amount` / `lost_at` are still populated on the `Dispute` record either way, for consistent reporting regardless of mode.

3 new tests, `tests/Feature/Payments/DeferredTransferDisputeLossFlowTest.php` — a non-partner deferred-mode case (reverses the correct transfer, records a Reversed payout row, updates the Dispute record), a partner deferred-mode case proving the "no payout found, skip" path leaves nothing touched, and a destination-mode regression proving the pre-existing branch is completely unchanged. All three passed FIRST try. Proactively grepped the whole `tests/` tree beforehand — zero pre-existing references to `handleDisputeLoss()` anywhere, no mocking-hazard risk. Re-ran the ENTIRE pre-existing dispute test suite (254 tests across every Dispute-related file: webhook processing, evidence submission, controllers, notifications, analytics) as an extra check given how heavily-tested `DisputeService.php` already is — all still pass, zero regressions.

This is the last item in Phase 4 — Phase 4 (the entire refund/dispute cutover) is now COMPLETE.

7.6 Webhook additions not needed

Removed from scope. This section originally proposed `transfer.created` / `transfer.reversed` / `transfer.failed` handlers to reconcile an asynchronous sweep job. Per Decision 1 (resolved — synchronous transfer, no delay), there is no asynchronous gap to reconcile: `transferToOperator()` knows whether its own `createTransfer()` call succeeded or failed in the same PHP call, inline (Section 7.3). No webhook changes are needed for this feature.

7.7 Card Reader / Terminal shipped, partial

Creation side done for every existing-booking context; the not-yet-created-booking case is done too (Section 7.2t). See Section 7.2g for the full writeup: `CardReaderRepository::getPaymentIntent()` now resolves the mode from the booking (and, for a combo cart, every sibling booking sharing its `cart_id`) referenced in `metadata.booking_id`. This section's original note about `transferToOperator()` being "new territory" here turned out to be based on an incomplete picture — tracing the actual JS confirms `CardReaderRepository` needs no transfer-dispatch code of its own at all: every existing-booking Card Reader context reuses the same confirmation code Card Online already uses for that context (already wired in Sections 7.2a-f). The New Booking and POS pages' Card Reader tabs, which charge a booking that doesn't exist in the database yet, now send the selected tour date(s) directly as `metadata.tour_dates` (Section 7.2t) instead of falling back to destination mode.

7.8 Reporting medium — data layer shipped, admin UI pending

`PaymentBreakdownRepository` (the admin financial report) currently trusts ledger columns like `service_commission` / `fee_amount` as ground truth and does no live Stripe reconciliation — it will keep working unchanged as long as those columns are still populated correctly by the write path. Since transfers are now synchronous (Decision 1), there is no "pending vs. transferred" state to surface after all — a `transfers` row with `transfer_type='primary_operator'` is either `Completed` or `Failed` the instant the booking is confirmed, simplifying this item considerably from the original draft. The addition worth having: surfacing `Failed` rows specifically, so admins have a way to notice and manually retry a payout that failed at charge time (Section 11).

Data layer done, user-directed, admin UI deliberately deferred to a later round. An investigation first established the real shape of the gap: exactly two methods in `TransferPaymentService.php` can ever write `transfer_status='Failed'` — `transferToOperator()` (`primary_operator`) and `transferSellingCommission()` (`selling_commission`); every other transfer-writing method hardcodes `Completed` / `Reversed` and can never fail this way. Both already caught the Stripe exception and logged it via `Log::error()` — but never persisted the message anywhere, so a report built directly off this table could show "Failed" but never *why*. Also confirmed: `PaymentBreakdownRepository`, the home this section originally sketched, is actually a poor structural fit — it's a chart-heavy aggregate JSON dashboard, not a list-of-problem-rows page; the much closer analog already in this codebase is `DisputeController` / `disputes/index.blade.php` (a paginated list of Stripe-related problem rows with a resolve action and a nav badge), which the eventual admin UI should clone instead.

Shipped now: a new tenant migration (`2026_09_15_000001_add_failure_tracking_to_transfers_table.php`, `Schema::hasColumn()`-guarded per column so it's safe to re-run, applied to all 13 local tenant databases) adds five columns to `transfers` — `failure_reason` (text, populated going forward) and an accountability trail for the future "mark resolved" admin action, `resolved_at` / `resolved_by` / `resolved_by_user_type` / `resolution_notes`, the last two mirroring this same table's own `created_by` / `created_by_user_type` column shapes exactly (no FK constraint, same enum value set) rather than inventing a new convention. `App\Models\Transfer`'s `$fillable` updated to match. Both `catch (Throwable $e)` blocks in `transferToOperator()` / `transferSellingCommission()` now write `$e->getMessage()` into `failure_reason` alongside the existing log call — a two-line change per method, no other call site in the file needed touching since none of them can ever produce `Failed`. Pre-existing historical `Failed` rows (there are none yet in any local tenant — confirmed directly, since no tenant has a real 2027+ booking yet) stay `NULL`; the future admin UI will show those as "reason not recorded" rather than backfilling a reason no one has.

2 new/extended regression tests in `tests/Unit/TransferPayment/TransferToOperatorTest.php`: the pre-existing `test_stripe_failure_is_recorded_as_failed_and_does_not_throw` gained an assertion that `failure_reason` now captures the exact simulated exception message; a new `test_selling_commission_stripe_failure_is_recorded_with_failure_reason` proves the same for the other capable method. Both verified via the revert-confirm-restore discipline (stashed the two changed app files, confirmed `failure_reason` came back `NULL` against the real assertion message, restored). Still needed, deliberately deferred: the actual admin-facing report — new repository query, controller, route/permission (`view-failed-transfers`, gated for both Admin and Tour Operator staff per the user's own call), Blade view, and nav badge count, all modeled on the Disputes stack identified above. 345 tests passing (344 + 1), zero orphaned rows.

Same-day follow-up, user question: should a REVERSAL failure (cancel/refund) get the same `failure_reason`/ledger treatment as a forward-transfer failure? Traced precisely: `reverseOperatorTransfer()` / `reverseSellingCommissionTransfer()` have no `try/catch` at all around their Stripe call — unlike the forward methods — so a failure there propagates straight out, uncaught, with no ledger row ever written. It hits the calling method's own `DB::beginTransaction()` / `catch` wrapper (every refund/cancel site has one), which rolls back, logs once via `Log::error()`, and returns an error to the admin. Decided: NOT the same gap, deliberately left as-is. A forward-transfer failure is silent and delayed — the booking looks fully confirmed to everyone, so without a ledger record the missed payout is invisible until someone happens to check Stripe directly. A reversal failure is neither: because of this same round's reverse-then-refund reordering, nothing has happened yet when it fails, so the whole action aborts cleanly, the admin sees the error in the same request, and the booking simply stays in its unchanged, still-active state — which is itself the visible signal that it needs another look. Building durable tracking for it would also require a genuine workaround (a second DB connection outside the enclosing transaction, purely to survive its rollback) for a failure mode that isn't actually hiding anything. No code changed for this item — a considered-and-declined decision, not a gap.

7.9 Known adjacent bug — fixed

Done. `TransferPaymentService::transferToPartner()` (and every other `Transfer::create()` call in the same file — `updatePartnerTransfer()`, `transferBetweenConnectedAccounts()`, `transferToNewPartner()`) hardcoded `'currency' => 'usd'` instead of `currencyFormation(code: true)` — latent for any Canadian-dollar tenant. Fixed across all four call sites in the same change that added `transferToOperator()`, since the new method needed the correct pattern anyway and the file was already open.

7.10 Per-tenant kill switch shipped

Done — resolves Decision 4. Every one of the ~20 PaymentIntent-creation call sites already funnels through `PaymentModeResolver::resolve()` / `resolveForSingleDate()` (Section 5), so the kill-switch is checked once, inside those two methods, rather than at each call site individually — a call site cannot forget the gate because it never implements the gate itself. When the tenant's `DEFERRED_TRANSFER_ENABLED` setting is off, or the row is missing entirely (an un-migrated tenant), the resolver returns `DESTINATION` unconditionally, without even inspecting the tour date — the same fail-closed, safe-by-default posture as every other guard shipped in this project.

Mechanism, deliberately reused rather than invented: a plain row in the tenant-scoped `settings` table (key `DEFERRED_TRANSFER_ENABLED`, migration `2026_09_13_000001_add_deferred_transfer_enabled_setting.php`), read via the pre-existing `hasSetting()` helper (`app/helpers.php`) — the exact same mechanism the pre-existing `AUTO_PAYOUT` toggle already uses. Chosen over a new central `tenants` table column for two reasons: it keeps the toggle correctly scoped to the tenant DB rather than the central one (matching every other per-tenant feature flag in this codebase), and — once wired into the admin UI (below) — it's editable through the exact same screen and save mechanism `AUTO_PAYOUT` already uses, no separate settings page needed.

Correction, caught by the user testing it live, not by this project's own review: `Admin\SettingController::edit()` DOES fetch every `settings` row generically (`Setting::orderBy('order_number')->get()`) for a tenant domain — but the VIEW that renders them, `resources/views/users/admin/setting-add-edit.blade.php`, is NOT generic. It renders each settings "card" via its own hardcoded `@if(in_array($setting->key, [...]))` whitelist array — one array per visual group — so a brand-new key is fetched by the controller but silently never appears anywhere on the page unless it's ALSO added to the matching whitelist. `AUTO_PAYOUT` only ever appeared because it was explicitly added to the plain-checkbox "General" group's array at some point — not because the page is generic, an assumption this section's first draft made without checking the view. Fixed: added `DEFERRED_TRANSFER_ENABLED` to that same whitelist array (same group as `AUTO_PAYOUT` — a plain toggle, no special markup needed, confirmed by reading the group's own per-item rendering: it's a generic labeled checkbox once whitelisted, with only one unrelated key in that group needing a special-cased label), and added a human-readable label ("2027+ Deferred Transfer") to the controller's existing `display_name` match-map (the same place `CUSTOM_FIELDS` is remapped to "ADD-ONS") rather than showing the raw constant name. Confirmed the save path needed no changes: `settings.js`'s change handler and the controller's `update()` both key off the

checkbox's numeric `id` / `name` attribute generically, already shared by every other setting in this group. Compiled views cleared so the fix takes effect without a deploy.

Global vs. per-tenant, decided: per-tenant, because every tenant has its own Stripe Connect account with its own capability/restriction state (Section 11 already flags "connected account restricted" as a real failure mode) — a per-tenant gate contains a problem to the one tenant that has it instead of forcing an all-or-nothing choice for every tenant, and matches how tenants will actually open their 2027 sales calendars on their own separate schedules rather than in one shared go-live moment.

Rollout state as shipped: migration applied to all 10 live local tenant databases (three additional local schemas — `ahbb_old`, `dixies_latest`, `dixies_new`, `kens_old` — are stale physical snapshots, not registered tenants, and were correctly skipped by `tenants:migrate`). Defaults OFF everywhere the migration runs; `dixies` was then explicitly flipped ON, since it's the tenant this whole feature's 139-test suite already runs against — matching the real-world "the pilot tenant is already opted in, everyone else waits for an explicit flip" shape a staged rollout is meant to have. Nothing else changed behavior for any other tenant: since no tenant has a 2027-dated booking yet, defaulting OFF has zero live impact today: it simply requires a deliberate per-tenant enable before that tenant's first real 2027+ charge.

4 new tests, `tests/Feature/Payments/DeferredTransferKillSwitchFlowTest.php`: disabled forces `DESTINATION` for a single 2027 date and for a combo cart containing one 2027 row among 2026 ones; enabling it doesn't disturb the underlying 2027-01-01 boundary rule (both a 2027 and a 2026 date still resolve correctly); a missing settings row (simulating an un-migrated tenant) fails closed to `DESTINATION`. A real test-authoring bug caught and fixed, not a production one: the first draft's `tearDown()` restored the setting via a bare SQL `update()` — which silently affects zero rows once the "missing row" test has deleted it entirely, permanently losing the setting for every test that ran afterward in the same PHPUnit process. Since `PaymentModeResolverTest` (Section 10) doesn't set the flag itself and assumes `dixies` already has it enabled, this one bug alone caused 8 of its assertions to fail the first time the full curated suite ran — diagnosed by reproducing it in isolation (passed alone, failed as part of the full suite, pointing straight at cross-test state leakage) rather than assumed. Fixed by deleting-then-reinserting the FULL original row in `tearDown()` instead of trusting an `update()` against a row that might no longer exist. `PaymentModeResolverTest` itself updated to initialize tenancy (it previously declared itself dependency-free and skipped tenancy setup entirely, correctly at the time) now that the resolver it tests reads the tenant `settings` table on every call. 143 tests passing (139 + 4), zero orphaned rows.

7.11 Destination-charge transfer description/metadata shipped

Real gap found by the user testing live, not by this project's own review: viewing a transfer in a connected account's own Stripe dashboard showed no description and no metadata at all. Investigated by first re-verifying this codebase's own explicit `Transfer::create()` calls (all six, in `TransferPaymentService` — `transferToPartner()`, `updatePartnerTransfer()`, `transferBetweenConnectedAccounts()`, `transferToNewPartner()`, `transferToOperator()`, `transferSellingCommission()`) — confirmed by reading every one that description/metadata are already present and have been for a long time (git history shows `transferToPartner()`'s own description/metadata predate this entire 2027 project). Confirmed empirically, not just by code review: created a real transfer against a real test-mode Stripe account with the exact same param shape this codebase already sends, then retrieved it back — description and metadata both came back exactly as sent. These six call sites needed no fix.

Root cause, also confirmed empirically: created a real destination charge (the model virtually every booking uses today — `transfer_data.destination` + `application_fee_amount`, Section 4.1) against the same test account and inspected the result. The Charge object correctly carried the description/metadata the PaymentIntent was created with — but the Transfer object Stripe creates *automatically* as a side effect of that charge came back with `description: null`, `metadata: []`, every time. This is a genuine Stripe platform behavior, not a bug in any of this codebase's own code: Stripe's `transfer_data` parameter only accepts `amount` / `destination`, with no way to set the resulting transfer's own description or metadata at creation time. The only way to enrich it is a follow-up `Transfer::update()` call once the transfer's id is known — and since virtually every booking today still settles via a destination charge (deferred-transfer mode is barely enabled anywhere yet, Section 7.10), this gap affected the overwhelming majority of live transfers, not an edge case.

First implementation attempt, corrected same revision by the user before finalizing: initially centralized this at the `payment_intent.succeeded` Stripe webhook (`StripeWebhookController::handleStripePaymentIntentSucceeded()`), reasoning that unlike `transferToOperator()` / `transferSellingCommission()` (which must never rely on a webhook, to avoid a double-transfer race on real money movement), pure metadata attachment is idempotent and side-effect-free, so a webhook's "fires for every succeeded PaymentIntent" property seemed like a safe, minimal-footprint win. The user corrected this: this app's `payment_intent.succeeded` webhook plumbing (the `metadata.section` / `app`-gated dispatch it's built around) was developed specifically for the mobile application's own payment flows, and the application is currently closed — so leaning on Stripe webhook delivery at all, for anything, cannot be trusted as a reliable mechanism right now, regardless of how the code reads. This is exactly the kind of operational/infrastructure fact no amount of code reading reveals — corrected immediately rather than defended.

Implementation, rebuilt around a queued Laravel Job instead — centralized in ONE place, but a place this project's own architecture already trusts and already relies on synchronously, not Stripe's webhook delivery: `App\Services\Booking\TransactionService::storePaymentInTransaction()` and its three siblings (`storeAddonsPaymentInTransaction()`, `storeCustomChargeInTransaction()`, `storeTipInTransaction()`) — the same four shared methods this whole project's `payment_mode` column already threads through ~40+ callers, the one genuinely common choke point every Card confirmation flow in the codebase passes through synchronously, no webhook involved. Added a small private gate, `maybeAttachDestinationTransferMetadata()`, called from all four (both branches of `storePaymentInTransaction()` — the fresh-insert path and the "flip an Initialized reservation to Completed" path) — when the payment is Card, currently DESTINATION mode, and has a real PaymentIntent id, it dispatches `App\Jobs\AttachDestinationTransferMetadataJob::dispatch($paymentIntentId)`, a plain `ShouldQueue` job on this app's own Redis-backed queue (`App\Services\Payments\AttachDestinationTransferMetadata::run()` underneath — same core Stripe logic as before: retrieve the PaymentIntent with `charges.data.transfer` expanded, skip if there's no transfer (deferred mode) or it already has description/metadata (idempotency, not correctness), else `PaymentGatewayInterface::updateTransfer()` with the PaymentIntent's own already-rich description/metadata). This keeps the checkout/confirmation HTTP response fast (the job runs in the background, not inline) while guaranteeing delivery for every real confirmed Card payment — it depends on nothing Stripe has to send back to this app.

A real cross-cutting test-safety issue found and fixed while wiring this: this local environment's `QUEUE_CONNECTION=database`, so any `ShouldQueue` job dispatched during a test run inserts a REAL row into the tenant's `jobs` table rather than being a harmless no-op — and since dozens of pre-existing

`DeferredTransfer*Test` files' own "destination-mode regression" cases exercise exactly the Card+DESTINATION path this new dispatch now fires on, every one of them would have started leaving orphaned `jobs` rows (and, worse, risked a real background queue worker later picking one up and hitting the live Stripe API with a fake test `payment_intent_id`). `tests/TestCase.php` already globally fakes `Storage` / `Mail` for every test for the same class of reason — added `Queue::fake()` to that same list, the same fix a handful of individual test files (e.g. the combo booking card flow tests) had already needed ad hoc for a different, pre-existing job. Confirmed zero rows land in the `jobs` table after a full regression run.

12 new tests: `tests/Unit/Payments/AttachDestinationTransferMetadataTest.php` (6) — attaches correctly on a bare automatic transfer; no-ops on an empty payment-intent id, on a charge with no transfer (deferred mode), on a transfer that already has description/metadata (retry guard), and on a PaymentIntent with no charge at all; falls back to a generic description when the PaymentIntent has none; a gateway failure is caught and logged, never thrown. Plus `tests/Unit/Booking/TransactionServiceAttachTransferMetadataDispatchTest.php` (5, via reflection on the private gate — the same "test the specific thing changed" precedent already used elsewhere in this project for a private method) proving the dispatch itself: fires for Card+DESTINATION (case-insensitively on the "via" string), does NOT fire for Cash, for deferred-transfer mode, or when there's no payment-intent id to act on. Proactively confirmed zero pre-existing tests reference either new class/job directly (no mocking-hazard risk); the two pre-existing `StripeWebhookForeignEventTest` failures encountered while testing the (now-abandoned) webhook approach were re-confirmed pre-existing and unrelated via a stash-and-rerun. 155 tests passing (143 + 12), zero orphaned rows (checked both the usual booking-related tables and, newly, the `jobs` table).

Second real gap, found by the user testing live, same revision: a REAL booking's connected-account "Payments" view (a `py_`-prefixed dashboard entry) still showed no description/metadata, even after the fix above. Traced via the user's own screenshots and confirmed empirically: this `py_` entry is `Transfer::$destination_payment` — Stripe's documented field naming "the id of the payment the destination account received for the transfer" — and it is a genuinely SEPARATE object from both the Transfer and the originating Charge, readable/writable ONLY via a request made WITH the connected account's own context (`['stripe_account' => $destinationId]`; every plain `Charge::retrieve()` / `Payout::retrieve()` attempt without that context returned "No such X"). Confirmed via two real bookings side by side: a DESTINATION-mode transfer's `destination_payment` already inherits the originating charge's description/metadata automatically (via the transfer's own `source_transaction` link back to that charge — a genuine Stripe behavior, confirmed by checking a real transfer's `source_transaction` field) — no fix needed there. A DEFERRED-mode transfer, however, is a bare, standalone `Transfer::create()` call with no `source_transaction` at all (nothing to inherit from), so its own `destination_payment` stays permanently blank — confirmed on the exact real booking (#348609) the user reported, both before and after a manual fix.

Fixed, centralized into ONE shared implementation used by BOTH transfer paths: added `PaymentGatewayInterface::updateCharge()` (a thin wrapper on `$this->stripe->charges->update()`) and a new public method, `AttachDestinationTransferMetadata::attachDestinationPaymentCharge($transfer, $destinationId, $description, $metadata)`, reusing the SAME description/metadata already computed for the transfer itself — no new shape invented. Called from two places: (1) all six of `TransferPaymentService`'s explicit `Transfer::create()` sites (deferred-mode transfers — `transferToPartner()`, `updatePartnerTransfer()`, `transferBetweenConnectedAccounts()`, `transferToNewPartner()`, `transferToOperator()`, `transferSellingCommission()`), reading `$transfer->destination_payment` directly off the object `Transfer::create()` already returns — no extra Stripe round-trip needed; (2) `AttachDestinationTransferMetadata::run()`'s own destination-mode path, as a defensive backstop for the (apparently rare, per the finding above) case where a destination charge's automatic inheritance doesn't apply — reading `destination_payment` off `updateTransfer()`'s own response, since a NESTED expand (`charges.data.transfer`) returns a trimmed transfer representation that omits this field (confirmed empirically), while `Transfer::create()` / `Transfer::update()`'s own top-level responses always include it.

7 new test methods across two files (18 additional assertions): 5 more in `tests/Unit/Payments/AttachDestinationTransferMetadataTest.php` (the shared helper updates the charge when `destination_payment` is present, skips cleanly when absent or when the destination id is empty, omits the description key when null, catches and logs a gateway failure) plus 1 extending `run()`'s own end-to-end test to also assert the destination_payment charge gets updated, and 1 new test in `tests/Unit/TransferPayment/TransferToOperatorTest.php` proving `transferToOperator()` itself enriches the destination_payment charge too, not just the transfer row. Verified end to end against the real bookings that surfaced this: ran the actual (unmocked) code against booking #348609's real PaymentIntent — both real Stripe objects (the transfer AND its destination_payment) now correctly carry matching description/metadata, confirmed by direct retrieval, not assumed. Also caught and cleaned up a self-inflicted side effect from this investigation's own empirical probing (a stray `test_probe` metadata key temporarily left on booking #348608's real transfer while confirming Stripe's `Transfer::update()` response shape) before finalizing — Stripe metadata updates MERGE into existing metadata rather than replacing it wholesale, a subtlety worth remembering for any future manual diagnostic work against a real object. 162 tests passing (155 + 7), zero orphaned rows.

7.12 Tenant identifier prefix on Stripe-facing descriptions built, disabled pending client confirmation

Requested directly by the user: since every tenant shares one platform Stripe account, a description like "Operator transfer for Booking #348610" gives no way to tell, from the Stripe dashboard alone, which tenant it belongs to. Asked for a short tenant label prefixed onto these descriptions — e.g. "Dixies - Operator transfer for Booking #348610" / "Kens - Operator transfer for Booking #348610".

Implementation, final form: a new helper, `tenantDescriptionPrefix()` (`app/helpers.php`, the same `function_exists()`-guarded convention as every other helper in the file) — always the tenant's own id, uppercased (e.g. `"DIXIES - "`), deliberately not `tenants.name`: that column is populated for some tenants with a real company name but confirmed NULL for others (including `dixies` / `kens`), so the id — always present — is used unconditionally rather than only as a fallback. Applied to all 11 Stripe-facing `'description'` strings in `TransferPaymentService.php` (the six transfer-creation sites and five reversal sites) and to `AttachDestinationTransferMetadata`'s own fallback description (Section 7.11).

An intermediate draft, and a real bug caught in it, before the user settled on the simpler final design above: a first draft instead preferred `tenants.name` when set, falling back to `ucfirst($tenant->id)` — that draft's docblock documented exactly this, but its body actually called `strtoupper($currentTenant->id)` unconditionally, ignoring `tenants.name` entirely; docblock and body disagreed, a real bug caught by a dedicated test rather than by inspection alone, and fixed at the time to match the docblock (name-preference restored). The user then directly edited the helper back to the plain `strtoupper($currentTenant->id)` form — the simpler always-uppercase-id design is what shipped; the docblock and test were updated to match this final version instead of the intermediate one.

Decided, this revision: ship the helper and every call site, but keep the prefix itself OFF for now, pending client confirmation. `tenantDescriptionPrefix()` currently returns `''` unconditionally via a single early-return line at the top of its body (commented to say why) — every call site above already routes

through this helper, so re-enabling the prefix everywhere it's used will be a one-line change (deleting that early return) once confirmed, with no call sites to revisit.

1 test, `tests/Unit/Helpers/TenantDescriptionPrefixTest.php`, asserting the current disabled behavior (`tenantDescriptionPrefix()` returns `''`) rather than the real prefix logic, since that logic is switched off for now — once re-enabled, this test should go back to asserting the id-fallback case directly (the name-preference branch was deliberately left uncovered even before the disable decision, since exercising it would require mutating the real central `tenants.name` row, which no permission was given to do for a test). 163 tests passing (162 + 1), zero orphaned rows (this test performs no writes at all).

8. Data Model Changes

All new columns are tenant schema changes (`database/migrations/tenant/`), since bookings, transactions, and transfers are tenant-scoped data. All three migrations below guard against re-runs and are written with `up()` / `down()` pairs.

TABLE	CHANGE	STATUS
`transactions`	`payment_mode` enum(`'destination'` , `'deferred_transfer'`) NOT NULL default `'destination'` , added `after('is_reallocated')`	Shipped. Migration `2026_09_07_000001_add_payment_mode_to_transactions_table.php` , `Schema::hasColumn()` -guarded. Applied to the local `dixies` tenant DB and all 12 other local tenant DBs; confirmed all 17,447 existing `dixies` transaction rows correctly backfilled to `'destination'` .
`transfers`	`transfer_type` enum(`'partner_split'` , `'primary_operator'`) NOT NULL default `'partner_split'` , added `after('charge_type')`	Shipped. Migration `2026_09_07_000002_add_payout_type_to_payouts_table.php` (its own filename and column name predate the Section 7.2u rename below — kept as-is, since already-applied migrations are never edited after the fact), `Schema::hasColumn()` -guarded. Decision 3 (below) is resolved: the existing table was extended rather than creating a new one, exactly as recommended.
`transfers`	`transfer_type` enum extended to add a third value, `'selling_commission'` (default stays `'partner_split'`)	Shipped. Follow-up migration `2026_09_07_000003_add_selling_commission_to_payouts_payout_type_enum.php` (same historical-filename note as above) — a separate migration rather than editing the already-applied one, guarded via an `information_schema.COLUMNS` check on the enum's own definition string. Applied to all 13 local tenant DBs.

Both `Transaction` and `Payout` Eloquent models were updated to add the new column(s) to `$fillable` .

9. Phased Rollout Plan

Simplified from the original 6-phase draft now that Decision 1 removes the scheduled-job/webhook-reconciliation machinery entirely — there is less to build, and "staged verification of a sweep job" is no longer a phase of its own.

PHASE	DELIVERABLE	STATUS
0 — Foundation	`payment_mode` column, `transfer_type` discriminator, `PaymentModeResolver`	Done. Section 7.1 / Section 8.
1 — Transfer engine	`TransferPaymentService::transferToOperator()` / `reverseOperatorTransfer()` (non-partner) and `transferSellingCommission()` / `reverseSellingCommissionTransfer()` (partner-package selling tenant), currency-hardcode fix, `transfer_type` filter hardening across existing lookups	Done. Section 7.3 / 7.9. 10 tests passing.
2 — Wire the gate	Branch `buildConnectedPaymentData()` on mode (Section 7.2, shipped); update all 20 PaymentIntent-creation call sites (Appendix A.1) to resolve mode and persist it to `transactions.payment_mode`; update `CardReaderRepository` (Section 7.7)	9 distinct creation code paths fixed, covering 12 features plus Card Reader, Payment Links, and POS (Sections 7.2a-n, 7.2r) — new-booking, Make Payment, combo cart, the shared `createStripePaymentIntentEditBooking()` helper (Change People, Change Payment Method, Switch Package), `CardReaderRepository::getPaymentIntent()` (Section 7.2g), the shared `createStripePaymentIntentCommon()` helper (Section 7.2h — Edit Discount, Custom Charge, Tip, Add-Ons, Addon Edit Charge), mobile API v2's `createPaymentIntent()` (Section 7.2m, Appendix A.1 site #20), `PaymentLinkService::createPaymentLink()` (Section 7.2n, Appendix A.1 site #21 — a Checkout Session, not a raw PaymentIntent), and `StripeRepository::stripeCreateStripePaymentIntentForPos()` → `StripeUtils::createStripePaymentIntentPos()` (Section 7.2r, Appendix A.1 site #1). Tour-guide gratuity (`create_tip`) confirmed out of scope per Section 3, correctly untouched. Now also done (Section 7.2t): the New Booking/POS pages' Card Reader tabs, which needed new request fields carrying the selected tour date(s) from those pages' JS since no booking exists yet to resolve a date from.
3 — Wire the transfer call	Add `transferToOperator()` / `transferSellingCommission()` calls, gated on `payment_mode='deferred_transfer'`, at each of the same synchronous confirmation call sites already used by `transferToPartner()` (Section 6.6's 19-site list) plus the two live webhook handlers that need the same fallback addition	15 of 19 done and end-to-end tested (Sections 7.2a-p) — `createBookingCard()`, `paymentNewBooking()`, `paymentCartBookings()`, `createPaymentIntent()` (V3 web), `changeHistoryPaymentType()`, `SwitchTourPackageCardService::completeSecureCardPayment()` (a real destination-column bug found and fixed here — see Section 7.2n's callout), `tipPayment()`, `createEditDiscountPaymentIntent()`, `customChargeCreatePaymentIntent()` (whose raw-manual-entry creation shape was also fixed but not independently tested), `chargeAddonFields()`'s PAYMENT section, `processAddonFinancialTransactions()`'s CHARGE action, mobile API v2's `packageBookingTour()` and `packageBooking3dsPayment()` (same destination-column bug found and fixed a second time here), Payment Links' `bookingUpdate()` / `handleStripePayment()`, and `createComboBookingCard()` (plus a bonus fix to its sibling creation method `createPaymentIntentForCart()`, whose own separate zero-partner-handling gap was replicated for deferred mode, not fixed). Partial: `SwitchTourPackageCardService::run()`'s payment branch is also wired but reasoned-through rather than independently tested; mobile API v2's, Payment Links', and `createPaymentIntentForCart()`'s own creation sides are wired but reasoned-through, not independently tested; both Switch Package entrypoints' refund branches, and both add-ons methods' own separate REFUND sections, remain outstanding. `applyDiscount()`'s Card branch (Appendix A.4 site #10, the last one on that list) is now fully investigated (Section 7.2q) and deliberately left unwired — it is unreachable from either of Edit Discount's two normal entrypoints (both already fully covered via `createEditDiscountPaymentIntent()`) and only reachable through a narrow, already-broken mobile-API edge case with no reliable PaymentIntent id to resolve a locked-in mode from at all — wiring it would require re-deriving mode from the current tour date, violating this plan's core "never re-derive" principle (Section 5.2). Appendix A.4's 19-site list is now fully accounted for — every site is either wired and tested, wired-but-reasoned-through (Switch Package's `run()`), correctly out of scope (Cash/Invoiced-only), or this one investigated-and-declined site (Section 7.2q) — none remain simply unexamined. Note: POS's confirmation, `PosRepository::cardPaymentForPos()` (Section 7.2r), is wired and tested too, but was never part of this 19-site count — it has no `transferToPartner()` call to catalogue in the first place (no partner-package handling exists in that method today), the same reason `createPaymentIntentForCart()` isn't on this list either.
4 — Refund/dispute cutover	`buildRefundParams()` helper (Section 7.4) rolled out across ~100+ call sites; `DisputeService::handleDisputeLoss()` mode branch (Section 7.5)	PHASE 4 COMPLETE. `PaymentGatewayInterface::buildRefundParams()` shipped and unit-tested (4 tests). Twenty-five refund call sites wired and tested end-to-end, spanning `PosRepository`'s "Refund Order" flow, all three `CancelBooking/*` services, Switch Package's refund branch, all four of `TourBookingAddonRepository.php`'s own refund sites, `CustomChargeBookingService.php`, both of `EditBookingService.php`'s refund sites, and — closing out `TourBookingRepository.php` entirely — `editDiscountRefund()`, `refundCustomCharge()`, `refund()`, `editBooking()`, and `deletePeopleChangePeople()` in BOTH the legacy and V3 files. Plus, closing out Phase 4 itself: `DisputeService::handleDisputeLoss()`'s mode branch (Section 7.5, 3 tests) — deferred-mode dispute losses now reverse the separate `primary_operator` Transfer via the already-shipped `reverseOperatorTransfer()`, since a deferred-mode charge carries no `transfer_data` for the pre-existing destination-mode logic to expand. Significant new finding surfaced while wiring Edit Discount: legacy (API V1)'s own `createEditDiscountPaymentIntent()` charge side had NEVER been wired at all — confirmed API V1 is still live traffic — fixed as a Phase 2/3 addendum before its refund side could be wired meaningfully. Every file and site originally surveyed for Phase 4 is now fully accounted for: `PosRepository`, all three `CancelBooking/*` services, Switch Package, `TourBookingAddonRepository.php`, `CustomChargeBookingService.php`, `EditBookingService.php`, all 49 `refundPayment()` sites across both `TourBookingRepository.php` files, and `DisputeService`. Wiring the add-ons sites caught and fixed a real mocking-hazard regression across 12 pre-existing tests in 9 files total (Section 7.4's callouts). Remaining, standalone (not part of Phase 4): a full audit of legacy's OTHER creation methods for the never-wired gap found for Edit Discount, explicitly flagged and not yet attempted.

5 — Reporting	Data layer: `failure_reason` capture on `transfers`. Admin UI: surface `Failed` primary-operator/selling-commission transfers for manual follow-up, modeled on the Disputes stack, not `PaymentBreakdownRepository` (Section 7.8)	Data layer done — migration shipped and applied to all 13 tenants, `failure_reason` now captured at the point of failure. Admin UI (controller/route/view/nav badge) not started — deliberately deferred to a later round per the user's own choice.
6 — Verification & go-live	Stripe test-mode end-to-end pass (Section 10); tenant-level or global kill-switch setting (Decision 4) as a rollback lever; deploy	Kill-switch done (Section 7.10) — per-tenant `DEFERRED_TRANSFER_ENABLED` setting, defaults OFF, applied to all 10 live local tenants, explicitly enabled for `dixies` only. Stripe test-mode end-to-end pass and deploy remain not started.

NO MIGRATION OR BACKFILL REQUIRED

Because no 2027-dated booking exists yet, there is nothing to convert and nothing to dual-support during a transition window. The `payment_mode` column being immutable per-booking (Section 5.2) still matters for every booking created from Phase 2 onward, since a booking's tour date can be rescheduled after payment and the column must keep reflecting what the *original charge* actually was — steady-state design hygiene, not a migration concern.

10. Testing Strategy

- Done: `PaymentModeResolver` unit tests — boundary date, mixed string/ `DateTimeInterface` input, combo-cart mixed-year arrays, null handling (12 tests).
- Done: `TransferPaymentService::transferToOperator()` / `reverseOperatorTransfer()` and `transferSellingCommission()` / `reverseSellingCommissionTransfer()` tests — card success, cash (no Stripe call), Stripe failure handling, partial reversal, over-reversal guard, partner-split/primary-operator cross-contamination regression, the partner-package flow at both a non-zero and a zero Partner Fee, and selling-commission reversal including its no-op case (10 tests). Gateway calls mocked via `$this->app->instance(PaymentGatewayInterface::class, ...)`, matching the project's existing test convention — no live Stripe calls made.
- Done: `StripePaymentGateway::buildConnectedPaymentData()` mode-branching tests — default/explicit destination mode unchanged, deferred mode omits `transfer_data` / `application_fee_amount`, `on_behalf_of` /statement descriptor preserved correctly in both modes, the no-card-payments-capability case (5 tests). Uses Stripe SDK's own `ApiRequestor::setHttpClient()` test seam rather than a live call.
- Done: the primary new-booking flow (Section 7.2a) is now verified end-to-end, not just by regression. `tests/Feature/Payments/DeferredTransferNewBookingFlowTest.php` (2 tests) calls the real `createBookingCard()` against a real `CreateABooking`-created booking dated 2027-06-15, mocking only the Stripe gateway, and asserts on real DB state: `transactions.payment_mode` persisted correctly, `transferToOperator()` actually calling `createTransfer()` with the tenant's real connected-account id and the right cents amount, the matching `transfers` row, and the absent-metadata regression case (falls back to destination mode, no transfer call). Also confirmed via the pre-existing `MakePaymentSameLoginDifferentIpTest` (4 tests) and `ApplicationFeeCentsRoundingTest` (6 tests), which exercise adjacent code and still pass unchanged.
- Done: the Make Payment flow (Section 7.2b) is likewise verified end-to-end via `tests/Feature/Payments/DeferredTransferMakePaymentFlowTest.php` (1 test) — real `stripeCreatePaymentIntentForMakePayment()` then real `paymentNewBooking()`, asserting the PaymentIntent metadata carries the mode at creation AND that it's correctly persisted/acted on at confirmation. This test is what caught the reservation-row bug described in 7.2b — proof that the dedicated-test requirement isn't a formality.
- Done: the combo cart flow (Section 7.2c) is likewise verified end-to-end via `tests/Feature/Payments/DeferredTransferComboCartFlowTest.php` (1 test) — real `stripeCreatePaymentIntentForMakeComboPayment()` then real `paymentCartBookings()` against two real bookings (one 2026-dated, one 2027-dated) sharing a cart, proving the Section 5.3 combo-cart rule end to end: the 2026 item is still persisted as deferred_transfer, and each item gets its own independent transfer.
- Done: the Change People flow (Section 7.2d) is verified via `tests/Feature/Payments/DeferredTransferChangePeopleFlowTest.php` (2 tests) — deliberately scoped to call the two touched methods directly (`createStripePaymentIntentEditBooking()`, `createPaymentIntent()`) rather than through the full ~3,250-line `changePeopleBooking()` dispatcher, since that method's own business logic is substantial and orthogonal to this change. Both passed on first try.
- Done: the Change Payment Method flow (Section 7.2e) is verified via `tests/Feature/Payments/DeferredTransferChangePaymentTypeFlowTest.php` (1 test, passed first try) — a real Cash booking switched to Card, confirming the existing `transactions` row (updated via direct Eloquent assignment, not `storePaymentInTransaction()`) correctly persists `payment_mode`, and that the first-ever operator transfer for that booking fires correctly.
- Done, partially tested: the Switch Package flow (Section 7.2f). `tests/Feature/Payments/DeferredTransferSwitchPackageFlowTest.php` (1 test, passed first try) verifies `completeSecureCardPayment()`'s non-partner case end to end — a purpose-built new package with a real connected-account id and a 2027-06-20 tour date, asserting persisted `payment_mode`, the `createTransfer()` call's destination/amount, and the resulting payout row. `run()`'s payment-branch dispatch (all 4 partner-status combinations) is wired the same way but not independently tested — exercising it safely would mean simulating its much larger surrounding discount/remaining-amount/rate-diff calculation, judged disproportionate for this revision. Phase 4 update: `run()`'s refund branch is now wired too (also reasoned-through, not independently tested, same complexity reason) — a genuinely different reversal shape requiring a `package_id`-based payout lookup rather than the `transaction_id`-based one every other Phase 4 site used; `completeSecureCardPayment()` confirmed to have no refund branch at all, nothing further to wire there.
- Done: Card Reader's shared creation endpoint (Section 7.2g/7.7) — resolves mode correctly for every existing-booking context (needs no confirmation-side changes of its own, since it reuses Card Online's already-wired code), and fixed a real stray-`application_fee_amount` bug along the way. The New Booking/POS pages' own Card Reader tabs (no booking exists yet to resolve a tour date from) are ALSO now done (Section 7.2t) — new JS-supplied `tour_dates` metadata closes the gap. 7 tests total in `tests/Feature/Payments/DeferredTransferCardReaderFlowTest.php` (4 original + 3 added for this), all passing.
- Done, partially tested: POS (in-person point of sale) checkout (Section 7.2r, Appendix A.1 site #1) — `StripeRepository::stripeCreateStripePaymentIntentForPos()` resolves mode across every cart package's own tour date (reasoned through, not independently tested — same combo-cart shape already proven elsewhere); `PosRepository::cardPaymentForPos()` confirmed and tested end to end via `tests/Feature/Payments/DeferredTransferPosFlowTest.php` (1 test, passed first try) — no partner-package handling exists in that method today, so only the non-partner `transferToOperator()` dispatch applies. This closes out Appendix A.1's site #1, the last explicitly-tracked creation-side gap from this section's own follow-up list above.
- Done: booking-level Tip (Section 7.2h) — `tests/Feature/Payments/DeferredTransferTipFlowTest.php` (1 test, passed after fixing a real pre-existing `tipBookingDetails()` bug — a missing `infants` column that broke every web tip payment, not just 2027+ ones). Also fixed the shared `createStripePaymentIntentCommon()` creation helper, covering Edit Discount/Custom Charge/Add-Ons/Addon Edit Charge's creation sides too, though their own confirmation sides remain unwired.
- Done: Edit Discount (Section 7.2i) — `tests/Feature/Payments/DeferredTransferEditDiscountFlowTest.php` (1 test, passed first try), calling `createEditDiscountPaymentIntent()` directly rather than the full `applyDiscount()` /history-writing chain.
- Done: Custom Charge (Section 7.2j) — `tests/Feature/Payments/DeferredTransferCustomChargeFlowTest.php` (1 test, passed first try). Also fixed a second, independent raw-manual-card-entry creation shape found in the same method, not independently tested (no known live caller reaches it today).
- Done: Add-Ons (Section 7.2k) — `tests/Feature/Payments/DeferredTransferAddOnsFlowTest.php` (1 test, passed first try); full 31-test pre-existing add-ons regression suite re-run and still passes.
- Done: Addon Edit Charge (Section 7.2l) — `tests/Feature/Payments/DeferredTransferAddonEditChargeFlowTest.php` (1 test, passed first try). This completes all five features sharing the Section 7.2h creation helper. The same 31-test add-ons regression suite re-run and still passes.

- Done: Mobile API v2 new-booking checkout (Section 7.2m) — `tests/Feature/Payments/DeferredTransferMobileApiV2FlowTest.php` (1 test, passed after two small fixture-shape fixes). Creation side (`createPaymentIntent()`) reasoned-through, not independently tested.
- Done: Payment Links (Section 7.2n) — `tests/Feature/Payments/DeferredTransferPaymentLinkFlowTest.php` (1 test, passed first try). Also caught and fixed a real, previously-shipped bug in Switch Package (Section 7.2f) while tracing this flow's own destination-column logic — see that section's callout.
- Done: Mobile API v2's `packageBooking3dsPayment()` (Section 7.2o) — `tests/Feature/Payments/DeferredTransferMobileApiV2ThreeDsFlowTest.php` (1 test, passed after adding an expectation for a second, separate `retrievePayment()` call made by the pre-existing saved-card feature).
- Done: Combo new-booking Card (Section 7.2p) — `createComboBookingCard()` (site #5) wired and tested; its sibling creation method `createPaymentIntentForCart()` also fixed (bonus, not on the A.4 list) but not independently tested. `tests/Feature/Payments/DeferredTransferComboBookingCardFlowTest.php` (1 test, passed first try).
- Investigated fully, deliberately left unwired (Section 7.2q): `applyDiscount()`'s Card branch (Appendix A.4 site #10, the last one on that list). Traced every caller and confirmed it is unreachable from Edit Discount's two normal entrypoints (both already fully covered elsewhere) — only reachable via a narrow mobile-API "second click" edge case that has no reliable PaymentIntent id to resolve a locked-in mode from, meaning wiring it would require re-deriving mode from the current tour date and violating this plan's core "never re-derive" principle. This closes out Appendix A.4's 19-site list — every site is now either wired, correctly out of scope, or investigated-and-declined for a stated reason.
- Phase 4 underway: `PaymentGatewayInterface::buildRefundParams()` shipped (Section 7.4), 4 unit tests. Five real sites wired and tested end to end: `TicketRefundService::refund()` (`tests/Feature/Payments/DeferredTransferTicketRefundFlowTest.php`, 2 tests) and `WeatherRefundService::refund()` (`tests/Feature/Payments/DeferredTransferWeatherRefundFlowTest.php`, 4 tests — 2 original + 2 added later for the gross-vs-net amount bug fix and the reverse-then-refund reordering, see the Revision History) — both dispatched from `PosRepository::handleRefundJobs()`'s Card branch, both no-partner-handling — plus all three `CancelBooking/*` services, dispatched from the separate "Cancel Booking" flow: `TicketRefundCancelService::cancel()` (`tests/Feature/Payments/DeferredTransferTicketRefundCancelFlowTest.php`, 3 tests), `WeatherRefundCancelService::cancel()` (`tests/Feature/Payments/DeferredTransferWeatherRefundCancelFlowTest.php`, 5 tests — same later additions as its sibling above), and `PartialRefundCancelService::cancel()` (`tests/Feature/Payments/DeferredTransferPartialRefundCancelFlowTest.php`, 6 tests — 3 original + 3 added later for the gross-vs-net amount bug fix across all 5 of its own call sites, see the Revision History) — a non-partner case proves `reverseOperatorTransfer()` alone; a partner case proves BOTH the pre-existing mode-independent `reverseTransfer()` (3rd party) AND the new `reverseSellingCommissionTransfer()` (selling tenant) fire with exactly the right amounts (including a correctly-SPLIT spillover case in `PartialRefundCancelService`'s own test); a destination-mode case proves the regression path is unchanged. All three `CancelBooking/*` services are now fully accounted for. Done, same session: the gross-vs-net `reverseOperatorTransfer()` amount audit was extended to every remaining caller in `TourBookingRepository.php` (legacy+V3), `TourBookingAddonRepository.php`, `EditBookingService.php`, `CustomChargeBookingService.php`, and `SwitchTourPackageCardService.php` — 5 distinct genuinely-broken patterns found and fixed across 15 call sites, `DisputeService::handleDisputeLoss()` and `CustomChargeBookingService.php` confirmed already correct with no changes needed; see the Revision History for the full breakdown. Done, same session: the reverse-then-refund reordering hardening (piloted on `WeatherRefundService` / `WeatherRefundCancelService`) was extended to the 17 "simple" sites across `TicketRefundService.php`, `PartialRefundCancelService.php`, `TourBookingAddonRepository.php`, legacy `TourBookingRepository.php`'s `editDiscountRefund()`, V3's `refundCustomCharge()` else-branch, and `EditBookingService.php` — a survey of all 57 sites in these files found 28 were already correctly ordered (a pleasant surprise), 1 (`DisputeService.php`) has no paired refund call at all, and 11 remain "complicated" (partner-booking coupling or an intervening `DB::commit()`), deliberately deferred to a separate round per the user's own sequencing choice; see the Revision History for the full breakdown. Done, same session: the remaining 11 complicated sites were finished too — the `DB::commit()` question resolved by confirming those commits guarded nothing but plain reads and simply keeping the commit immediately after `refundPayment()` in its new position; the partner-coupled sites moved their whole `reverseSellingCommissionTransfer()` / `reverseOperatorTransfer()` block as one unit. This closes out reverse-then-refund hardening for all 57 surveyed sites. Still needed: independently testing the handful of duplicate/reasoned-through sites flagged across both rounds (see the Revision History).
- Per project convention, tests run against the real `dixies` tenant DB with no transaction rollback — every new test wraps `CreateABooking`'s `create()` / `cleanup()` in `try { } finally { }`. Tests that write directly to `transfers` (outside the fixture) must also clean that table up themselves in `tearDown()`, as the shipped tests do — confirmed zero orphaned rows left behind after a full local test run.
- Done: the regression pass this bullet originally flagged as "still needed" was completed as part of Phase 4 itself — rolling out `buildRefundParams()` to `TourBookingAddonRepository.php` /all three `CancelBooking/*` services/ `RefundBooking/WeatherRefundService.php` surfaced a real mocking-hazard regression across 12 pre-existing tests in 9 files (Section 7.4's own callouts), all fixed; the existing 31/42-test add-ons and cancel/weather-refund suites all still pass.
- Done: per-tenant kill-switch (Section 7.10, Decision 4) — `tests/Feature/Payments/DeferredTransferKillSwitchFlowTest.php` (4 tests) proves `PaymentModeResolver` fails closed to destination mode when the tenant's `DEFERRED_TRANSFER_ENABLED` setting is off or missing entirely, for both a single date and a combo cart, and that enabling it doesn't disturb the underlying date-boundary rule. `PaymentModeResolverTest` (above) updated to initialize tenancy, since the resolver it tests now reads the tenant `settings` table on every call — it previously declared itself dependency-free and correctly skipped tenancy setup.

11. Risk Register

RISK	IMPACT	MITIGATION
Combo cart spans a 2026 and 2027 line item	Medium — ambiguous settlement mode for the whole PaymentIntent	Section 5.3 "any row ≥ 2027 forces deferred mode" rule, shipped in `PaymentModeResolver` and regression-tested; the transfer engine iterates per destination, so the 2026 line item still gets paid correctly, just via an explicit `Transfer::create()` instead of Stripe's automatic split
Booking's tour date changes after payment (reschedule/switch-package), crossing the 2026/2027 boundary	Medium — could desync mode from current date if re-derived live	`payment_mode` stored immutably at creation time (5.2); reschedule flows never rewrite it (Phase 2, not yet built, must preserve this)
Dispute-loss handler silently no-ops or errors for deferred-mode charges (no `$charge->transfer`)	Resolved	Explicit mode branch in `DisputeService::handleDisputeLoss()` (7.5) — built and tested (3 tests), confirmed against live code during this revision's audit, not just this row's own prose
A synchronous transfer fails right after the charge succeeds (e.g. connected account restricted)	Medium — customer charged, operator not paid until manually retried	Handled inline today: `transferToOperator()` / `transferSellingCommission()` catch the failure, log it, and record a `Failed` ledger row with a real `failure_reason` (Section 7.3/7.8, data layer shipped) rather than throwing — this is strictly better visibility than a destination charge offers today, where an equivalent failure has no ledger trace at all; still needs an admin-facing surface to actually notice `Failed` rows (Section 7.8, admin UI not yet built). The mirror-image case — a *reversal* (cancel/refund) failing — was deliberately NOT given the same treatment: see Section 7.8's own callout for why that's a different, already self-evident failure mode.
Two confirmation paths (sync client callback + async webhook) could race and double-transfer	High if unguarded — double payout, real money	Inherited for free from the existing `transactionCount==0` idempotency check both paths already share (Section 6.6); confirmed wired correctly for the one site done so far (Section 7.2a) — the same guard must wrap every remaining site in Phase 3
Operator-transfer and partner-split transfer interaction on the same 2027+ booking, including the risk of double-paying the selling tenant the ticket-cost portion	Resolved — see Section 6.7/Decisions 5 & 6	Corrected design: `transferSellingCommission()` sends the selling tenant only their own Partner Fee (never the ticket-cost-inclusive remainder `transferToOperator()` would have sent), so nothing needs reconciling back out of their account. Regression-tested, including the zero-Partner-Fee case.
`CardBookingService::run()`'s payout lookup has no `transfer_type` filter (Section 7.2a)	Low today (coincidentally correct for both new row types), fragile	Found during Section 7.2a's wiring, not fixed — flagged as its own small follow-up before this code is trusted further
The one wired flow (7.2a) had no dedicated end-to-end feature test at first, only regression coverage	Resolved	`DeferredTransferNewBookingFlowTest` (2 tests) now exercises the real code path with real DB assertions — see 7.2a/Section 10
A confirmation flow's pre-created placeholder ("reservation") `transactions` row never sets `payment_mode` at all, silently defeating the gate forever for that flow (found in Make Payment, Section 7.2b)	Resolved	Fixed for Make Payment's reservation insert (and its canceled-reservation-reuse branch). Every other confirmation call site was subsequently checked for this same shape across Sections 7.2c-7.2u and the dedicated legacy-creation-method audit (Section 7.2s) — this is exactly the class of bug a dedicated end-to-end test catches and a code read alone does not, and the project's own testing discipline (a real feature test per site, not just a code read) is what caught it here the first time and would catch it again if it recurred
A confirmation site's own destination lookup silently reads the wrong `stripe_destination_id` column (package-level instead of operator-level, or vice versa), producing a false-positive "Completed" payout row with no Stripe transfer ever actually made (found in Switch Package's `completeSecureCardPayment()`, Section 7.2f, caught while wiring Payment Links, Section 7.2n)	Was High until caught — the customer's charge still succeeds either way, so nothing looks wrong from outside; only the operator's payout silently never arrives, with a ledger row falsely claiming success	Fixed; the shipped test's own package fixture had accidentally masked this exact bug by setting the package-level column to the same value as the operator-level one — fixed the test too, and verified with a real before/after regression proof (reverted the code fix alone, confirmed the test now fails, restored the fix, confirmed it passes again). Every remaining site must have its destination-column source (operator- vs package-level) explicitly verified against real data, not assumed from a variable name alone — this is now a second confirmed instance of a "looks right, reads the wrong column" class of bug (see the Section 7.2b `paymentNewBooking()` /Section 7.2m `packageBookingTour()` missing-alias gaps below), on top of the pre-created-placeholder-row class already found in Make Payment.
Refund helper rollout across 100+ call sites introduces a regression in a rarely-hit refund branch	Medium	Phase 4 ships after Phases 2-3 are stable; full regression matrix (Section 10) before go-live; each call site's old behavior is preserved exactly for `'destination'` mode
CLAUDE.md's stale file paths mislead a future engineer picking this plan up	Low	This document uses verified real paths throughout (Section 2 callout); recommend a separate small PR correcting CLAUDE.md

12. Open Decisions Needed

DECISION 1 — RESOLVED: TRANSFER FIRES IMMEDIATELY, SYNCHRONOUSLY

Confirmed: the operator's share moves the instant the charge is confirmed, same timing as a destination charge today — no delay, no scheduled job, no "hold until tour date/cancellation window" behavior. The engine is built accordingly (Section 7.3).

DECISION 2 — RESOLVED: COMBO-CART MODE RULE

"Any row ≥ 2027 forces the whole cart to deferred mode" (Section 5.3) is implemented in `PaymentModeResolver` and regression-tested. No objection raised — proceeding as shipped.

DECISION 3 — RESOLVED: LEDGER TABLE SHAPE

The existing `transfers` table was extended with a `transfer_type` discriminator (Section 8) rather than creating a new table. Shipped.

DECISION 4 — RESOLVED: PER-TENANT KILL-SWITCH

Per-tenant, chosen over global specifically because every tenant has its own Stripe Connect account with its own capability/restriction state — a per-tenant gate contains a problem to the one tenant that has it rather than forcing an all-or-nothing choice for everyone, and matches how tenants will actually open their 2027 calendars on separate schedules. Shipped (Section 7.10): a tenant-scoped `DEFERRED_TRANSFER_ENABLED` setting, same mechanism as the pre-existing `AUTO_PAYOUT` toggle, checked once inside `PaymentModeResolver` so no call site can bypass it. Defaults OFF everywhere; explicitly enabled for `dixies` only.

DECISION 5 — RESOLVED: OPERATOR + PARTNER-SPLIT INTERACTION

Business-confirmed design (Section 6.7): for a partner-package booking, `transferToOperator()` is not called. Instead `transferSellingCommission()` sends the selling tenant only their configured Partner Fee, and `transferToPartner()` runs additionally, completely unchanged, sending the 3rd party's ticket cost independently. Regression-tested in `TransferToOperatorTest`'s partner-package tests, including the zero-Partner-Fee case.

DECISION 6 — RESOLVED: NO TICKET-COST DOUBLE-DISBURSEMENT

An earlier draft of this plan (before the correction above) would have sent the selling tenant the full remainder via `transferToOperator()`, which really would have disbursed the ticket cost twice with nothing reconciling it back out. The corrected design in Decision 5 sends the selling tenant only their own Partner Fee — never the ticket-cost portion — so this risk doesn't arise under the new model. (Whether today's *existing* destination-charge-based flow has the same underlying question is a separate, pre-existing matter this plan doesn't need to resolve to proceed, since the new model is built correctly from the start.)

13. Appendix A — Full Call-Site Inventory

A.1 PaymentIntent creation

#	FILE : LINE	TRIGGER
1	`app/Utils/StripeUtils.php:61` shipped, partial	POS (browser) checkout — `createStripePaymentIntentPos`. Wired (Section 7.2r): mode resolved via `PaymentModeResolver::resolve()` across every cart package's own tour date; confirmation (`PosRepository::cardPaymentForPos()`) tested end-to-end. Creation side reasoned-through, not independently tested. The New Booking/POS pages' own Card Reader tabs (no tour-date plumbing for a not-yet-created booking) are now ALSO done (Section 7.2t).
2	`app/Utils/StripeUtils.php:329`	Cart checkout (multi-booking) — `createStripePaymentIntentCart`
3	`app/Utils/StripeUtils.php:546` shipped	Shared by FIVE features (Section 7.2h) — Edit Discount, Custom Charge, Tip, Add-Ons, Addon Edit Charge — `createStripePaymentIntentCommon`. All five confirmation sides are now wired (Sections 7.2h-7.2l).
4	`app/Utils/StripeUtils.php:639`	Shared `createStripeIntent`: new booking, make-payment, combo-payment, edit-booking
5	`app/Repositories/TipCenter/GratuityRepository.php:1328` out of scope	Tour-guide "Tap to Pay" tip — investigated fully (Section 7.2s): confirmation (`afterTipPayment()`) writes a `gratuities` row with `booking_id = NULL`, no tour date to resolve a mode from at all; matches Section 3's original tour-guide-gratuity exclusion
6	`app/Services/Booking/StripeService.php:1318` out of scope	Guide-dashboard "Create Tip" — same finding as row 5 (confirmation `makeTipPayment()`, same `booking_id = NULL` shape)
7	`app/Repositories/TourBooking/TourBookingRepository.php:1953`	Legacy cart card-checkout (confirm:true flow)
8	`TourBookingRepository.php:11675`	Mobile-app card reader, cart context
9	`TourBookingRepository.php:11686`	Mobile-app card-present, cart context
10	`TourBookingRepository.php:11727`	Mobile-app card reader, single-booking
11	`TourBookingRepository.php:11738`	Mobile-app card-present, single-booking
12	`TourBookingRepository.php:17220`	Web custom-charge new PaymentIntent branch
13	`app/Repositories/TourBooking/V3/TourBookingRepository.php:2369`	V3 cart card-checkout
14	`V3/TourBookingRepository.php:14494`	V3 mobile-app card reader
15	`V3/TourBookingRepository.php:14509`	V3 mobile-app card-present
16	`V3/TourBookingRepository.php:20179`	V3 web custom charge
17	`app/Repositories/Pos/CardReaderRepository.php:325` shipped	Admin Terminal, `connect=true` (direct charge, unused path) — mode-aware as of Section 7.2g, though this branch never runs today
18	`CardReaderRepository.php:327` shipped	Admin Terminal, `connect=false` (live destination-charge path) — wired Section 7.2g; resolves mode from `metadata.booking_id`, or (Section 7.2t) from a JS-supplied `metadata.tour_dates` when no booking exists yet (New Booking/POS pages) — falls back to destination only when neither is present
19	`app/Services/Network/Stripe/StripeService.php:50`	Cross-tenant Network booking — no Connect params, out of scope
20	`app/Http/Controllers/API/V2/TourBookingController.php:1941` shipped	Mobile API v2 new-booking checkout — wired Section 7.2m (creation reasoned-through; confirmation, `packageBookingTour()`, end-to-end tested)
21	`app/Services/Booking/PaymentLinkService.php:257,296` shipped	Emailed Payment Link — Checkout Session `payment_intent_data` — wired Section 7.2n; creation reasoned-through, confirmation (`handleStripePayment()`) end-to-end tested

A.2 Refund call sites (representative — full grep list available on request)

FILE	REPRESENTATIVE LINES
`app/Services/RefundBooking/TicketRefundService.php`	216-221, 291-296
`app/Services/RefundBooking/WeatherRefundService.php`	195-196
`app/Services/Booking/ChangePeopleAddonService.php`	1555-1556
`app/Services/CancelBooking/PartialRefundCancelService.php`	307-308, 348-349, 415-416, 449-450, 482-483
`app/Services/Booking/EditBookingService.php`	533-534, 788, 795, 821-822, 829-830, 838-839
`app/Services/Booking/StripeService.php`	1233-1234
`app/Repositories/TourBooking/TourBookingRepository.php`	~5030-5036 and 21 more sites
`app/Repositories/TourBooking/V3/TourBookingRepository.php`	21 sites, parallel structure

A.3 Dispute & transfer reversal

FILE : LINE	ROLE
`app/Services/Dispute/DisputeService.php:643–741`	`handleDisputeLoss()` — reads `$charge->transfer`, reverses it; needs the mode branch (7.5)
`app/Services/TransferPayment/TransferPaymentService.php`	`transferToPartner()` / `reverseTransfer()` — the reusable templates; `transferToOperator()` / `reverseOperatorTransfer()` and `transferSellingCommission()` / `reverseSellingCommissionTransfer()` now added alongside them (Section 7.3, shipped)
`app/Http/Controllers/StripeWebhookController.php:207–214`	Dispute webhook dispatch — unaffected by this plan (Section 7.6: no `transfer.*` handling needed)

A.4 `transferToPartner()` call sites — the exact wiring points for `transferToOperator()`/`transferSellingCommission()` (Section 6.6)

#	FILE : LINE	METHOD	GATE CONDITION TODAY
1	`V3/TourBookingRepository.php:1112`	`createBooking()`	`$isPartnerPackage==1`, Cash
2	`V3/TourBookingRepository.php:1827`	`createComboBooking()`	same, Cash
3	`V3/TourBookingRepository.php:2227` shipped	`createPaymentIntent()`	`$data['isPartnerPackage']==1`, Card — wired Section 7.2d
4	`V3/TourBookingRepository.php:2616` shipped	`createBookingCard()`	same, Card — the synchronous confirmation entrypoint quoted in 6.6; wired Section 7.2a
5	`V3/TourBookingRepository.php:2797` shipped	`createComboBookingCard()`	same, Card — wired Section 7.2p; its sibling creation method `createPaymentIntentForCart()` (zero partner handling of its own, not on this list) fixed alongside it
6	`V3/TourBookingRepository.php:11897`	`changePeopleBooking()`	`$isPartnerBooking==1`, Cash — out of scope, Cash never touches deferred-transfer mode
7	`V3/TourBookingRepository.php:14965` shipped	`paymentNewBooking()`	`$isPartnerBooking==1`, destination id from already-loaded `$tourBooking[0]->stripe_destination_id` — wired Section 7.2b
8	`V3/TourBookingRepository.php:15380`	`paymentCartBookings()`	Invoiced/Cash branch — out of scope
9	`V3/TourBookingRepository.php:15501` shipped	`paymentCartBookings()`	Card branch — wired Section 7.2c
10	`V3/TourBookingRepository.php:19323` investigated, declined	`applyDiscount()`	Card branch — has the `partner_destination_id` bug noted in 6.6. Traced fully (Section 7.2q): unreachable from Edit Discount's two normal entrypoints (both go through site #11 instead); only reached via a mobile-API "second click" edge case with no reliable PaymentIntent id to resolve mode from — wiring it would mean re-deriving mode from the current tour date, violating Section 5.2's core principle. Deliberately left unwired.
11	`V3/TourBookingRepository.php:19468` shipped	`createEditDiscountPaymentIntent()`	`$tourBooking->is_partner_booking==1` — wired Section 7.2i
12	`API/V2/TourBookingController.php:857` shipped	`packageBookingTour()`	`$bookingData->is_partner_booking==1` — wired Section 7.2m
13	`API/V2/TourBookingController.php:1195` shipped	`packageBooking3dsPayment()`	same shape — wired Section 7.2o; found+fixed the same destination-column bug as Section 7.2n's callout, in this method's own separate local query
14	`ExtendBooking/ExtendBookingInvoicedService.php:500`	—	Invoiced/Cash — out of scope
15	`Booking/PaymentLinkService.php:648` shipped	`handleStripePayment()`	`is_partner_booking==1` — wired Section 7.2n
16	`SwitchTourPackage/SwitchTourPackageInvoicedService.php:329`	—	`is_partner_package==1 && $netAmount>0` — Invoiced, out of scope
17	`SwitchTourPackage/SwitchTourPackageCashService.php:420`	—	same, Cash — out of scope
18	`ExtendBooking/ExtendBookingCashService.php:533`	—	Cash — out of scope
19	`SwitchTourPackage/SwitchTourPackageCardService.php:497` shipped	`run()`	Card — wired Section 7.2f (reasoned-through, not independently tested for the partner-package combinations). Phase 4: this method's separate refund branch also now wired (Section 7.2f), same reasoned-through-only status — a package-id-based reversal lookup, genuinely different from every other Phase 4 site.

Every one of these currently gates on a partner-package condition; Phase 3 adds a parallel, independent gate at the same 19 sites (plus the webhook fallbacks below) on `payment_mode=='deferred_transfer'`: when the existing partner-package condition is false, call `transferToOperator()`; when it's true, call `transferSellingCommission()` instead, and `transferToPartner()` continues to run exactly as it does today, unchanged (Decision 5, resolved, Section 6.7). `transferToOperator()` and `transferSellingCommission()` are mutually exclusive per booking — never both.

A.5 The six `payment_intent.succeeded` webhook handlers (Section 6.6, Finding 2)

HANDLER	WRITES TRANSACTIONS?	LIVE TODAY?
`StripePaymentIntentSucceededForNewBooking`	Yes — single & combo	Yes
`StripePaymentIntentSucceededForMakePayment`	Yes, or updates a pending `Initialized` row	Yes
`StripePaymentIntentSucceededForDiscountPayment`	Via `TransactionService::storePaymentInTransaction()`	Yes
`StripePaymentIntentSucceededForTip`	No — `tour_booking_history` only	Yes (irrelevant to operator transfers)
`StripePaymentIntentSucceededForPaymentLink`	No	No — entire body is a commented-out call
`StripePaymentIntentSucceededForTapToPayTip`	No — `gratuities` table	No — `metadata->section` is never set due to a missing `config/constants.php` key

14. Appendix B — Stripe Connect Glossary

TERM	MEANING
Destination charge	A PaymentIntent created on the platform account with `transfer_data.destination` set; Stripe automatically splits and transfers funds to the connected account the moment the charge settles. This is what the platform uses today for every booking.
Separate charges and transfers	Stripe's own name for the target pattern: the PaymentIntent has no `transfer_data`, the full amount stays on the platform's balance, and a completely separate `Transfer` object moves funds to the connected account whenever the platform decides to call it.
application_fee_amount	On a destination charge, the slice of the charge the platform keeps; the rest goes to the connected account automatically. Meaningless once `transfer_data` is removed — the platform's cut is instead just "whatever isn't transferred."
on_behalf_of	Marks the connected account as the entity the charge is "for," used to show their name/statement descriptor on the customer's card statement and shift some card-present liability. Can be used with or without `transfer_data` — this plan recommends keeping it in deferred mode to preserve statement-descriptor behavior.
transfer_group	A free-text tag that ties a charge and its later, separate transfer(s) together for reconciliation — already generated today (`generateTransferGroup()`) and simply needs to be reused as the join key for deferred transfers.
reverse_transfer / refund_application_fee	Refund-time flags that only make sense against a destination charge: they tell Stripe to also pull back the automatic transfer and/or the application fee. Meaningless (and must not be sent) for a charge that never had `transfer_data` in the first place.
Direct charge	A third pattern (present but unused in this codebase's `connect=true` branch) where the PaymentIntent is created directly against the connected account via a `stripe_account` request header, rather than on the platform account at all. Not part of this plan.

2027 Deferred Transfer Migration Plan

Prepared September 7, 2026 · Revised September 13, 2026 (independent 100%-completeness audit; Decision 4 resolved and shipped — per-tenant DEFERRED_TRANSFER_ENABLED kill-switch, Section 7.10; destination-charge transfers' missing description/metadata found (live, by the user, on a real booking) and fixed via a queued Job, not a webhook — then a second related gap, the connected account's own separate destination_payment record, found and fixed too (Section 7.11), both verified against the real booking that surfaced them; the Stripe-facing `payout_amount` metadata key/description wording, deliberately left unrenamed by Section 7.2u's original pass, revisited at the user's request and renamed to `transfer_amount`/"transfer" across all 8 files that set it, then the same treatment extended to `TransferPaymentService.php`'s own 118 internal "payout"-named variables (collision-checked against the file's existing `$transfer` Stripe-response variables first) — deliberately not extended to the other 7 shared files' own internal variables, a materially broader scope flagged to the user rather than assumed (Section 7.2u addendum) — that scope note was itself then reversed days later, extending the rename to all 10 files the `payoutAmount` token actually spans (69 replacements, Section 7.2u); a tenant-identifier prefix for Stripe-facing descriptions was also built (`tenantDescriptionPrefix()`, Section 7.12), but is currently disabled pending client confirmation; a real production bug (booking #348614) — `WeatherRefundService`/`WeatherRefundCancelService` reversing the operator's Transfer for the wrong (gross, not net) amount — was found, fixed at all 8 affected call sites, and hardened further by reversing the Transfer before the Stripe refund rather than after, piloted on these same two files (Section 7.4); the same audit then extended to `PartialRefundCancelService.php`'s own 5 call sites, confirmed genuinely broken (including against real discount-transaction fee data), and fixed with a "ticket-cost-first" reversal model — reverse from the operator up to the remaining ticket cost only, any excess absorbed by NAT's own fee, never the operator — corrected mid-implementation by the user from an initial proportional-split draft; the SAME audit was then swept across every other `reverseOperatorTransfer()` caller in the codebase (8 more files), finding and fixing 5 more distinct instances of the identical bug across 15 call sites in `TourBookingRepository.php` (legacy+V3), `TourBookingAddonRepository.php`, `EditBookingService.php`, and `SwitchTourPackageCardService.php`, with `DisputeService.php`/`CustomChargeBookingService.php` confirmed already correct; the reverse-then-refund reordering hardening (piloted on WeatherRefund*) was then extended to the 17 "simple" sites across those same files, of a 57-site survey that found 28 already correctly ordered, 1 not applicable, and 11 "complicated" ones (partner-booking coupling or an intervening DB::commit()) deliberately deferred to a separate round at the user's own choice; that separate round then finished all 11 complicated sites, closing out reverse-then-refund hardening for the full 57-site survey. Phase 5 (Reporting, Section 7.8) then began: its data layer shipped — a new failure_reason column (plus a resolved_at/resolved_by/resolution_notes accountability trail for the admin UI still to come) captured at the exact point transferToOperator()/transferSellingCommission() already catch a Stripe failure, migration applied to all 13 tenants — with the admin-facing report itself (modeled on the Disputes stack, not PaymentBreakdownRepository, per this same investigation) deliberately deferred to a later round. 345 tests passing, up from 139)